

Friedrich-Alexander-Universität Erlangen-Nürnberg

**Lehrstuhl für Multimediakommunikation und
Signalverarbeitung**

Prof. Dr.-Ing. André Kaup

Master Thesis

**Investigating the Impact of
Point Cloud Upsampling on
3D Object Detection Performance**

Jiahui Shen

September 2026

Supervisor: Marina Ritthaler, M.Sc.



Lehrstuhl für Multimediakommunikation
und Signalverarbeitung
Prof. Dr.-Ing. André Kaup



Friedrich-Alexander-Universität
Erlangen-Nürnberg

Master Thesis
for
Mr. Jiahui Shen

**Investigating the Impact of Point Cloud Upsampling on 3D Object Detection
Performance**

**Untersuchung des Einflusses von Punktwolken-Upsampling auf die Leistungsfähigkeit der
3D-Objekterkennung**

A big topic of research in the field of point cloud enhancement is the upsampling of point clouds. The goal of the upsampling methods is to create a more dense and uniform point cloud. The idea is to do upsampling as a first step before other tasks like shape classification and object detection are performed. Despite the upsampling task being seen as a preprocessing step, most works only evaluate their method by comparing the resulting point cloud with the ground truth using metrics like Chamfer Distance and point-to-surface distance.

Therefore, this thesis aims to investigate, which improvements can be achieved in object detection tasks when upsampling is done in a preprocessing step. For this analysis, multiple pipelines of upsampling methods and object detection networks will be build and compared with each other. Hereby, deep learning based and optimization based upsampling are assessed comparatively.

The research examines the extent to which upsampling methods can enhance object detection. Additionally, it investigates, whether the classic evaluation metrics on point cloud upsampling are good indicators for the improvements the methods can achieve in the object detection performance. We also want to explore, which type of upsampling method provides the most benefit.

As part of this work, a thorough literature review is required to examine the state-of-the-art upsampling and object detection techniques. In addition, the thesis is expected to provide clear and comprehensive documentation of the research, methodology, and findings.

Start: 01.03.2026
End: 01.09.2026

A handwritten signature in blue ink, reading 'A. Kaup', is written over a horizontal line. Below the line, the text 'Prof. Dr.-Ing. A. Kaup' is printed in a black, sans-serif font.

Declaration

I confirm that I have written this thesis unaided and without using sources other than those listed and that this thesis has never been submitted to another examination authority and accepted as part of an examination achievement, neither in this form nor in a similar form. All content that was taken from a third party either verbatim or in substance has been acknowledged as such.

Place, Date

Jiahui Shen

Contents

Kurzfassung	V
Abstract	VII
Symbols and Notations	IX
Abbreviations and Acronyms	XI
1 Introduction	1
2 Fundamentals and Related Work	5
2.1 Point Clouds and 3D Perception	5
2.1.1 Definition and Properties of Point Clouds	5
2.1.2 Differences Between Object-Level and Scene-Level Point Clouds	8
2.2 Point Cloud Classification	9
2.2.1 Problem Formulation	9
2.2.2 PointNet	10
2.2.3 PointNet++	10
2.3 3D Object Detection from Point Clouds	13
2.3.1 PointRCNN	13
2.3.2 CenterPoint	16
2.4 Point Cloud Upsampling	18
2.4.1 Edge-Aware Resampling (EAR)	19
2.4.2 PU-Net	20
2.4.3 PU-GCN	21
2.4.4 PU-EdgeFormer	24
2.4.5 PDANS	25
2.5 Evaluation Metrics	26
2.5.1 Geometric Metrics	27
2.5.2 Classification Metrics	29
2.5.3 Detection Metrics	29
3 Idea and Concept	31
3.1 Overall Concept	31
3.2 Research Hypotheses	32
3.3 Evaluation Pipeline	33
3.4 Experimental Comparison	34
3.4.1 Line A - Densification of the Original Observation	34
3.4.2 Line B - Recovery after Controlled Sparsification	34
3.5 Fairness Protocol	35

3.6	Cross-Domain Interpretation	36
4	Experimental Setup	37
4.1	System Overview and Final Experiment Matrix	37
4.2	Software and Hardware Environment	38
4.3	Data Preparation	39
4.3.1	ModelNet40	39
4.3.2	KITTI	41
4.4	Downsampling Procedure	43
4.5	Upsampling Method Integration	43
4.5.1	Shared Input and Coordinate Interfaces	44
4.5.2	EAR	45
4.5.3	PU-Net	46
4.5.4	PU-GCN	47
4.5.5	PU-EdgeFormer	48
4.5.6	PDANS	48
4.6	Ratio Normalisation and Point-Count Verification	49
4.7	KITTI Format Conversion and Intensity Transfer	50
4.8	Detector Integration	50
4.8.1	PointRCNN Integration	50
4.8.2	CenterPoint Integration	52
4.9	Classifier Integration	53
4.10	Geometric and Task Evaluation Protocol	54
4.10.1	ModelNet40 geometric evaluation	54
4.10.2	ModelNet40 classification evaluation	56
4.10.3	KITTI detection evaluation	57
4.10.4	Geometry-task relationship	57
4.11	Detector Retraining with PU-GCN Inputs	58
4.12	Executed Diagnostic and Mechanism Experiments	61
4.12.1	Patch Construction and Normalisation	61
4.12.2	Detector Input Budgets and Generated-Point Dose	62
4.12.3	ModelNet40 Controls and Input Reception	64
4.13	Reproducibility and Validity Checks	64
5	Results and Evaluation	67
5.1	ModelNet40: Classification and Geometric Fidelity	67
5.1.1	Densifying the Original Observation	67
5.1.2	Recovery after Fourfold Sparsification	68
5.1.3	Category-Dependent Classification Response	72
5.1.4	Genuine-Sampling Controls and Point Reception	77
5.1.5	Equal-Cardinality Geometry	79
5.1.6	Multi-Method Point-Cloud Evidence	82
5.2	KITTI: Frozen Detection and Detector Adaptation	86
5.2.1	Three-Epoch Full-Validation Comparison	86

5.2.2	PointRCNN	86
5.2.3	CenterPoint and Class Dependence	87
5.2.4	Class, Difficulty, and Projection Results	89
5.2.5	PU-GCN Inputs and Detector Retraining	89
5.3	KITTI Integration and Mechanism Diagnostics	97
5.3.1	Earlier Multi-Method Evidence	97
5.3.2	Patch Locality and Normalisation	102
5.3.3	The Effective Detector Input	104
5.3.4	Selection, Sampling Variance, and Negative Controls	105
5.3.5	Completed Input and Selection Interventions	108
5.3.6	Object-Level Losses and Recoveries across the Validation Set	112
5.3.7	Where Matched Objects Are Lost	114
5.3.8	Whole-Scene Multi-Method Comparisons	118
5.3.9	Local Surfaces and Measurement Preservation	120
5.3.10	Matched Detection Failures and Positive Counterexamples	122
5.4	Joint Discussion	127
6	Conclusion and Outlook	131
6.1	Findings from the Completed Experiments	131
6.2	Limits of the Evidence	132
6.3	Implications and Future Work	133
	List of Figures	135
	List of Tables	137
	Bibliography	141
	Curriculum Vitae	145

Kurzfassung

Die Hochabtastung von Punktwolken erhöht die Anzahl räumlicher Abtastpunkte. Ein Nutzen für nachgelagerte Wahrnehmungsaufgaben lässt sich jedoch nicht allein aus geometrischen Abständen ableiten. Diese Arbeit untersucht die Verdichtung einer ursprünglichen Eingabe sowie die Rekonstruktion nach einer Reduktion auf ein Viertel der Punkte. Auf ModelNet40 werden EAR, PDANS, PU-Net, PU-GCN und PU-EdgeFormer mit separat trainierten PointNet++-Klassifikatoren und geometrischen Referenzen gleicher Punktanzahl verglichen. Auf KITTI werden PointRCNN und CenterPoint untersucht. Eine PU-GCN-Eingabe- und Detektoranpassungsstudie über drei Epochen umfasst 20 Auswertungen. Anschließend wird das Training mit Eingaben unter Erhalt der beobachteten Punkte verlängert. Jede Hauptauswertung verwendet 3.769 Validierungsframes.

Bei der protokollierten, über Batches gemittelten Genauigkeit übertrifft keine getestete Methode in ModelNet40-Linie A die beste Gesamtgenauigkeit der ursprünglichen Darstellung mit 1.024 Punkten von 91,95%. Nach der Ausdünnung verbessert PU-Net diese Genauigkeit von 90,85% auf 91,27%, während PU-GCN in beiden Linien den niedrigsten mittleren Chamfer-Abstand erreicht. Auf KITTI verschlechtern direkte PU-GCN-Eingaben die Ergebnisse der unveränderten Detektoren. Der Erhalt gemessener Punkte und die Anpassung der Detektoren verbessern die Ergebnisse, übertreffen jedoch nicht die jeweiligen Referenzen. Das verlängerte Training verbessert alle vier Detektor-Linien-Kombinationen weiter und erfüllt das festgelegte Plateaukriterium für die Car-Validierungsmetrik. Dennoch verbleiben Rückstände von 3,31 bis 11,28 AP-Punkten gegenüber den jeweiligen Linienreferenzen. Das PU-GCN-Netz selbst verwendet weiterhin seine veröffentlichten Gewichte.

Diagnostische Versuche zu Patch-Normalisierung, Punktverarbeitung und Voxelbelegung zeigen, warum gespeicherte Punktdichte und wirksame Modelleingabe verschieden sind. Die Aussagekraft wird durch nur einen Trainingsseed, die Auswahl des besten Klassifikators anhand des Testsatzes, ungleiche Trainingsbudgets der Detektoren und die Auswahl ihrer Checkpoints anhand desselben Validierungssatzes begrenzt. Geometrische Qualität allein reicht somit nicht aus, um einen Nutzen für nachgelagerte Aufgaben nachzuweisen.

Abstract

Point-cloud upsampling increases the number of spatial samples, but its benefit for downstream perception cannot be inferred from geometric distance alone. This thesis evaluates densification of an original input and recovery after fourfold sparsification on two complementary tasks. The ModelNet40 study compares EAR, PDANS, PU-Net, PU-GCN, and PU-EdgeFormer using separately trained PointNet++ classifiers and equal-cardinality geometric references. The KITTI study evaluates point-based PointRCNN and voxel-based CenterPoint, with a 20-arm PU-GCN input and three-epoch detector-adaptation matrix, followed by extended observed-first training. Each main evaluation covers 3,769 validation frames.

Using the recorded batch-aggregated accuracy, no tested method improves ModelNet40 Line A best overall accuracy over the original 1,024-point representation at 91.95%. After sparsification, PU-Net raises best accuracy from 90.85% to 91.27%, while PU-GCN achieves the lowest mean Chamfer distance in both lines. On KITTI, direct insertion degrades frozen detection. Preserving measured points and adapting the detectors recover performance but leave deficits relative to the corresponding baselines. Extended adaptation further improves all four detector-line conditions and meets the specified Car-validation plateau criterion, but still leaves deficits of 3.31–11.28 AP points relative to the line-specific references. PU-GCN itself retains its released pretrained weights throughout this later study.

Patch-normalisation, point-reception, and voxel-occupancy diagnostics show why stored density differs from effective task input. The findings are bounded by single-seed classifier training, test-based best-checkpoint selection, unequal detector-training budgets, and detector checkpoint selection on the reported validation set. The combined evidence establishes a limited object-level recovery benefit and partial scene-level recovery, while showing that geometric fidelity alone is insufficient to establish downstream utility.

Symbols and Notations

Symbols are defined at their first use. The following list collects the principal quantities; method-specific indices are defined locally.

Point Sets and Scan Geometry

$\mathcal{P}, \mathcal{X}$	Input point sets
$\mathbf{p}_i, \mathbf{x}_i$	Three-dimensional coordinates
ν_i	Reflectance associated with a measured point
N_s, M_s	Original and sparse point counts in frame s
$\mathcal{R}$	Geometric reference point set

Networks and Detection

ϕ_θ, f_θ	Learned feature map and classifier
$\mathcal{N}_j(R)$	Radius- R neighbourhood of centroid j
$y_m, \hat{y}_m$	True and predicted class of test object m
$B_{\text{pred}}, B_{\text{gt}}$	Predicted and ground-truth boxes

Upsampling and Evaluation

U_m	Upsampling method m
$\mathcal{C}, \mathcal{Y}$	Candidate and strict output sets
S_4, A_d	Cardinality operator and downstream adapter
$\boldsymbol{\mu}, s$	Patch centroid and normalisation radius
Δ	Difference from the stated reference

Abbreviations and Acronyms

AP	Average precision
BEV	Bird’s-eye view
CAD	Computer-aided design
CD	Chamfer distance
CPU	Central processing unit
CUDA	Compute Unified Device Architecture
DDIM	Denoising diffusion implicit model
EAR	Edge-Aware Resampling
EMD	Earth Mover’s Distance
FAU	Friedrich-Alexander-Universität Erlangen-Nürnberg
FPS	Farthest Point Sampling
GPU	Graphics processing unit
HD	Hausdorff distance
HPC	High-performance computing
IoU	Intersection over union
LiDAR	Light Detection and Ranging
mAcc	Mean class accuracy
MD5	Message-Digest Algorithm 5
MLP	Multilayer perceptron
MSG	Multi-scale grouping
NUC	Local non-uniformity coefficient used in the evaluation
OA	Overall accuracy
P2F	Point-to-face distance
PCA	Principal component analysis
RCNN	Region-based convolutional neural network
RPN	Region proposal network
SHA-256	Secure Hash Algorithm, 256-bit
SSG	Single-scale grouping
XYZI	Cartesian coordinates with intensity

Chapter 1

Introduction

As a principal data source for perception in autonomous driving, Light Detection and Ranging (LiDAR) provides direct and precise range measurements of the three-dimensional environment [1]. The measurement process is, however, inherently non-uniform. Because a LiDAR sensor emits rotating laser beams at a fixed angular resolution, two adjacent beams are always separated by a constant angle: close to the sensor the beams lie near one another, whereas their separation grows steadily with distance. An object of a given size therefore receives progressively fewer laser returns the farther away it is located. As a consequence, LiDAR point clouds are dense in the near range and sparse in the far range, and they additionally suffer from occlusion and from an insufficient number of points on small objects. The density of the points in turn affects both the classification of an object and the localisation of its three-dimensional bounding box. In the KITTI detection benchmark [1, 2], objects that are more distant or occluded and thus carry fewer points are harder to detect.

Precisely because the detection difficulty is caused by a shortage of points, an idea suggests itself: to make the points denser, that is, to generate a denser and more uniform point cloud from the sparse input, in the hope of recovering the object surface and its local structure. In other words, before the data enter the classification, segmentation or detection module, an independent module could generate additional points that lie on the observed surfaces and thereby recover part of the lost performance — without replacing the sensor and without retraining the downstream detector. Point cloud upsampling provides exactly such methods. Covering both object-level and scene-level problems, this thesis compares different upsampling methods, comprising classical geometric-optimisation approaches, represented by edge-aware resampling [3], and deep-learning-based approaches [4, 5, 6, 7]. Whether a method that performs better on geometric metrics also performs better in downstream classification and detection, however, remains open. This leads to the fundamental question addressed in this thesis.

To examine this relationship, it is necessary to distinguish geometric reconstruction quality from downstream task performance. Geometric evaluation in point cloud upsampling uses distances to reference points or surfaces and measures of sampling uniformity [4, 5]. These include Chamfer distance (CD), Hausdorff distance (HD), point-to-surface distance and uniformity measures. They can reflect changes in surface boundaries and local point distribution, so these properties are not exclusive to detector evaluation. However, such geometric scores do not directly evaluate a downstream network’s predicted classes or bounding boxes. PointRCNN uses foreground classification and proposal refinement, whereas CenterPoint predicts object centres and box attributes [8, 9]. Whether an improvement in geometric reconstruction also improves these predictions therefore needs to be evaluated on the downstream task.

The automotive setting poses a further difficulty when applying networks trained on sampled object surfaces to LiDAR scenes. PU-Net, for example, learns from point patches sampled from 3D models [4]. This differs from the partial observations obtained by a vehicle-mounted LiDAR sensor [1, 2]. In this thesis, the integration of the selected patch-based networks includes local patch extraction and normalisation. In the later PU-GCN pipeline, predicted patches are rescaled and translated back to LiDAR coordinates before merging and detector-input construction (Section 4.5.1). Normalisation is therefore reversed before the downstream task; it does not itself imply a permanent change in scene scale. The integration experiments examine coordinate-transformation errors and the effects of patch construction and merging (Section 4.12). This allows implementation effects to be examined separately from the behaviour attributed to the upsampling method.

This thesis therefore investigates upsampling under controlled output point counts and explicitly specified downstream training conditions. The central question is whether upsampling recovers useful surface evidence and improves downstream classification or detection. A related question is how changes in geometric quality correspond to changes in these task outcomes. The study compares CD and HD with downstream performance and considers the recorded point-to-face (P2F) distances as a separate surface diagnostic. The different normalisation used for this diagnostic is specified in Section 5.1.5.

To address these questions, this thesis makes the following contributions.

1. **A strict, cardinality-controlled evaluation protocol.** Within each input line, upsampling methods produce the same target number of output points for a given input. Original and sparse baselines retain their native point counts. Direct

outputs are distinguished from observed-first KITTI inputs, which concatenate all measured input rows with selected predicted rows. Preservation applies to the constructed point-cloud file in that condition; direct outputs do not impose this requirement. Later detector sampling or voxelisation can still limit which points enter the network (Sections 4.6 and 4.8).

2. **A two-line experimental design.** Line A tests whether densifying the original input improves performance over that original input. Line B tests recovery after deliberate fourfold downsampling, using the sparse input as the baseline and the original input as a recovery reference. Results are reported separately because the lines start from different observations and address different performance questions.
3. **A cross-task evaluation framework.** The same protocol is applied both to object-level shape classification on ModelNet40 [10] with PointNet++ [11] and to scene-level 3D detection on KITTI — with the point-based detector PointRCNN [8] and the voxel-based detector CenterPoint [9]. The two detectors consume the input in structurally different ways, which makes it possible to distinguish the effect of the generated geometry itself from the effect of a particular detector architecture; the object-level branch, in turn, provides a controlled surface-sampling setting in which to examine whether the same behaviour also occurs outside real sensor scenes.
4. **Task-level evaluation.** Geometric metrics are used together with the classification accuracy of PointNet++ and the detection Average Precision (AP) of PointRCNN and CenterPoint.
5. **An explicit treatment of the input budget.** This thesis formalises the observation that a detector accepts only a bounded amount of input, so that adding points beyond that budget forces a selection. The protocol includes configurations that separate this effect from the geometric quality of the generated points.
6. **The integration process itself as an object of study.** The spatial locality of the patches, unique support, coverage and normalisation are explicitly defined, measured and audited rather than taken for granted, so that conclusions about a method do not inadvertently become conclusions about its wrapper code.

To describe this evaluation design consistently, the experiments distinguish between *task branches* and *input lines*.

Two task branches are studied:

- classification with PointNet++ on ModelNet40, and
- detection with PointRCNN and CenterPoint on KITTI.

Each task branch contains two input lines: Line A densifies the original observation, whereas Line B first degrades the observation by a deterministic fourfold reduction and then attempts to recover it. Both lines are specified formally in Section 3.4. Throughout this thesis the terminology is therefore used uniformly as follows: *task branch* refers to ModelNet40 or KITTI, and *input line* refers to Line A or Line B.

The primary detection comparison keeps the downstream detectors frozen. A later, separate experiment adapts PointRCNN and CenterPoint to PU-GCN inputs while retaining a fixed PU-GCN upsampler. This is detector adaptation, not retraining of the upsampling network. The object-classification branch uses separately trained PointNet++ SSG classifiers under the common training recipe described in Chapter 4; it is distinct from this later detector-adaptation experiment.

The following chapters describe the foundations, experimental design and results of this evaluation. Chapter 2 introduces point clouds, LiDAR perception and the networks used for classification and detection. It explains the upsampling methods and their mathematical formulation. It also defines the geometric and task metrics used in the evaluation. Chapter 3 develops the concept: the research hypothesis, the evaluation pipeline, the two comparison lines, the fairness protocol, and the challenges that had to be solved. Chapter 4 documents the experimental setup at the level of detail required for reproduction: data preparation, downsampling, the integration of each upsampling method, the format conversions, the detector and classifier configurations, the computation of the geometric metrics, and the verification procedures. Chapter 5 presents and discusses the results. Chapter 6 summarises the findings and outlines the direction in which an upsampling method would have to be developed in order to be genuinely useful for downstream 3D perception.

Chapter 2

Fundamentals and Related Work

This chapter discusses the concepts and mathematical background needed to understand point cloud upsampling and its use in three-dimensional perception. It begins with point representation and sampling, then explains classification and detection before introducing the upsampling methods. The final section defines geometric and task metrics. The purpose is to explain how the methods work and how their outputs should be interpreted; the particular datasets, training settings and evaluation configurations are introduced in Chapter 4.

2.1 Point Clouds and 3D Perception

Point clouds provide spatial observations; classification and detection assign semantic meaning to those observations. In object classification, a classifier receives the points of one segmented object and predicts one category [12]. In three-dimensional object detection, a detector receives a scene and predicts object instances, their categories, confidence scores and oriented bounding boxes [8,9]. A detector must therefore localise objects as well as recognise them. The distinction connects the point representation introduced below to the classification networks in Section 2.2 and the detectors in Section 2.3. Upsampling supplies denser coordinates to either task; it does not itself assign an object label or a detection box [4,5].

2.1.1 Definition and Properties of Point Clouds

A point cloud is an unordered finite set of discretely sampled points in three-dimensional space [12]. When only geometric coordinates x_i , y_i and z_i are used, a point cloud consisting of N points is written as

$$\mathcal{P} = \{\mathbf{p}_i\}_{i=1}^N, \quad \mathbf{p}_i = (x_i, y_i, z_i)^\top \in \mathbb{R}^3. \quad (2.1)$$

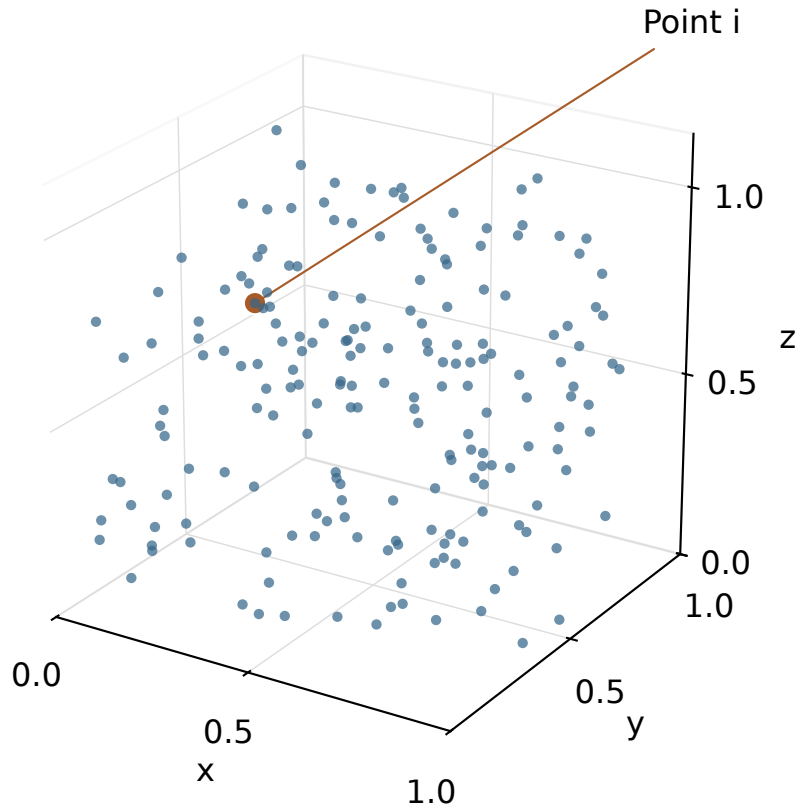


Figure 2.1: A synthetic random set of 192 points in the unit cube, used only to illustrate three-dimensional coordinates and irregular point spacing. All points are shown; the highlighted point is one member of the same set. Coordinates are dimensionless. This is a conceptual example, not an object sample or an experimental result.

Here N is the number of stored points, i indexes a point, and (x_i, y_i, z_i) are its Cartesian coordinates. The superscript T denotes transpose, so that $\mathbf{p}_i$ is a column vector; $\mathbb{R}^3$ is three-dimensional real coordinate space. A point gives a location, not a small solid object. Connections, faces and a surface normal are not supplied by the coordinate set itself. A mesh, in contrast, explicitly records connections between vertices [12].

Figure 2.1 illustrates this definition with a random point set. The example has no object category and no experimental meaning: it shows only positions in a coordinate system. The highlighted point has three coordinates just like every other point. The apparent distance between points in a projected image must not be confused with their full three-dimensional distance.

If the sensor additionally provides reflectance, colour or normal vectors, the spatial coordinates can be stored together with an attribute vector as

$$\mathbf{u}_i = [\mathbf{p}_i^\top, \mathbf{a}_i^\top]^\top \in \mathbb{R}^{3+d_a}. \quad (2.2)$$

Here $\mathbf{a}_i \in \mathbb{R}^{d_a}$ contains d_a attributes, and $\mathbf{u}_i$ is the extended record. This notation keeps the spatial point $\mathbf{p}_i \in \mathbb{R}^3$ separate from its features. An XYZI record, for example, contains (x_i, y_i, z_i, ι_i) with measured reflectance ι_i [1]. Distances between positions use the first three coordinates; adding reflectance to a Euclidean distance without a defined scaling would mix different quantities.

Point clouds possess, first of all, the property of being *unordered*. Writing the records as rows of a matrix $\mathbf{U} \in \mathbb{R}^{N \times (3+d_a)}$ is a storage convention and does not express a natural ordering. For a set-level output such as a classification score, a function f must satisfy

$$f(\{\mathbf{p}_1, \dots, \mathbf{p}_N\}) = f(\{\mathbf{p}_{\pi(1)}, \dots, \mathbf{p}_{\pi(N)}\}) \quad (2.3)$$

for every permutation π of the indices $1, \dots, N$. This is permutation invariance. For a per-point output, such as foreground segmentation, reordering the inputs should reorder the outputs in the same way; this is permutation equivariance. A shared pointwise mapping followed by symmetric aggregation implements the invariant case in PointNet [12]. Section 2.2.2 explains how the network uses this construction.

Point clouds often exhibit *irregular sampling*. The representation does not require a fixed image-like lattice, and local distances and neighbourhood sizes can vary substantially [11]. Measured point clouds are also often *incomplete*, because a sensor records visible surfaces subject to occlusion and acquisition limitations [1]. Unordered storage, irregular density and incomplete coverage are different properties. A randomly reordered cloud preserves its geometry; removing points can change both its sampling density and its coverage. These distinctions lead to two different observation processes.

An object-level cloud represents one segmented shape; surface sampling from a mesh can cover regions that would be hidden from a single viewpoint. A scene-level scan instead contains returns from several objects, ground and background. Surface coverage depends on the observation process, not on the definition of a point cloud [1, 10, 12]. The concrete object dataset, normalisation and scene data preparation are specified in Sections 4.3.1 and 4.3.2.

Table 2.1: Structural differences between object-level surface samples and LiDAR scenes. Surface completeness and normalisation describe common preparation conventions, not requirements of either representation [1, 8, 10, 12].

Comparison dimension	Object-level point cloud	LiDAR scene
Data source	Sampling of a CAD mesh surface	Real sensor returns
Spatial content	A single, already segmented object	Multiple objects, ground and background
Surface completeness	Can cover the complete available surface	Covers sensor-visible surfaces
Coordinate scale	Often centred and unit-normalised	Real metric coordinates
Sampling regularity	Controlled surface sampling	Determined by scan angles, range and occlusion
Primary task	Output of one object category	Output of multiple categories and 3D boxes
Dominant error consequence	Altered class-discriminative features	Altered class, centre, size and orientation

2.1.2 Differences Between Object-Level and Scene-Level Point Clouds

Table 2.1 compares the two observation processes. It describes common surface-sampling and sensor settings; it does not specify a particular experimental split or point count [1, 8, 10, 12].

In the last row, *error* refers to a distortion of the input geometry, such as missing surface support, displaced points or spurious added points. Its *consequence* is a possible change in the features used for a task. For classification, this can alter the evidence distinguishing categories. For detection, it can additionally affect the estimated centre, dimensions or orientation of an object [8, 12]. The row describes possible pathways from geometric distortion to a task error; it does not assert that every upsampling operation causes them.

The table therefore explains why the same geometric operation need not have the same task effect in both settings. A category label summarises one input shape; a detector must additionally separate instances and localise them within a scene [8, 12]. To understand where additional points enter those decisions, the next two sections follow

the processing chain from coordinates to class scores and then from scene features to three-dimensional boxes.

2.2 Point Cloud Classification

2.2.1 Problem Formulation

The input of point cloud classification is the point set $\mathcal{P}$ of a single object and the output is its category y [11, 12]. Here $y \in \{1, \dots, C\}$ and C is the number of classes. The subscript θ denotes learned parameters. A classification network f_θ produces the logits of C categories,

$$\mathbf{z} = f_\theta(\mathcal{P}) \in \mathbb{R}^C. \quad (2.4)$$

Here $\mathbf{z}$ is a vector of real-valued class scores, or logits. After softmax, the predicted probability $\hat{p}_c$ of class c is

$$\hat{p}_c = \frac{\exp(z_c)}{\sum_{j=1}^C \exp(z_j)}. \quad (2.5)$$

Here z_c is the score of class c , j indexes all candidate classes, and $\exp$ is the exponential function. The denominator makes the probabilities sum to one. Given the ground-truth label y , the cross-entropy objective used for classification [12] is

$$\mathcal{L}_{\text{cls}} = -\log \hat{p}_y, \quad (2.6)$$

where $\log$ is the natural logarithm and $\hat{p}_y$ is the probability assigned to the true class. A confident correct prediction has a small loss; assigning little probability to the true class gives a large loss. For M training samples the empirical risk is

$$\mathcal{L}_{\text{train}} = \frac{1}{M} \sum_{m=1}^M \mathcal{L}_{\text{cls}}^{(m)}. \quad (2.7)$$

The superscript (m) indexes a sample and $\mathcal{L}_{\text{train}}$ is its sample-averaged training objective. These equations express the classification objective of PointNet-family networks [11, 12]. The classification decision is $\hat{y} = \arg \max_c \hat{p}_c$. This formulation shows that upsampling yields a benefit only if it changes the features extracted by the network in such a way that the score of the true class is raised relative to the scores of the remaining classes. Defining the classification margin as

$$\Delta_{\text{cls}}(\mathcal{P}) = z_y - \max_{c \neq y} z_c, \quad (2.8)$$

a positive margin means that the true class has the largest score. Here z_y is the true-class logit and the maximum selects the strongest competing class; a zero margin is a tie. For a fixed classifier, a change in point count alone imposes no constraint on this margin. The margin can increase or decrease when the input changes. A change across zero can correct a previous classification error or introduce a new one. These statements follow from the decision rule; they are not additional measured results. The following architectures explain how the point coordinates become the scores used by that rule.

2.2.2 PointNet

PointNet processes an unordered point set directly through a shared pointwise mapping, symmetric aggregation and a classification head [12]. The shared multilayer perceptron (MLP) applies the same learned transformation to every point. Its output is a feature vector describing that point in a learned space. A channelwise max operation then retains the largest response in each feature channel across the whole set. Because the maximum is unchanged by reordering, the resulting global feature satisfies the permutation invariance in Equation (2.3). The classification head maps that feature to category scores.

PointNet also uses transformation networks to align input coordinates and intermediate features in learned spaces. The feature transformation is regularised towards an orthogonal matrix, which discourages severe distortion without imposing an exact rigid rotation [12]. These transformations complement the shared mapping and pooling; they do not introduce an ordering of the points.

The global maximum is determined by the points attaining the largest response in each channel. Qi et al. describe this property through a critical-point set [12]. Repeating an identical pointwise feature does not change a channel’s maximum. However, the shared mapping before pooling does not explicitly aggregate the relative positions of neighbouring points. PointNet++ addresses this limitation by applying local PointNet operations within nested metric regions [11]. This progression from global aggregation to local geometric aggregation connects the two architectures.

2.2.3 PointNet++

PointNet++ [11] learns local-to-global features progressively by means of hierarchical sampling, radius grouping and local PointNet aggregation. A set abstraction layer usually comprises the three steps of sampling, grouping and feature extraction.

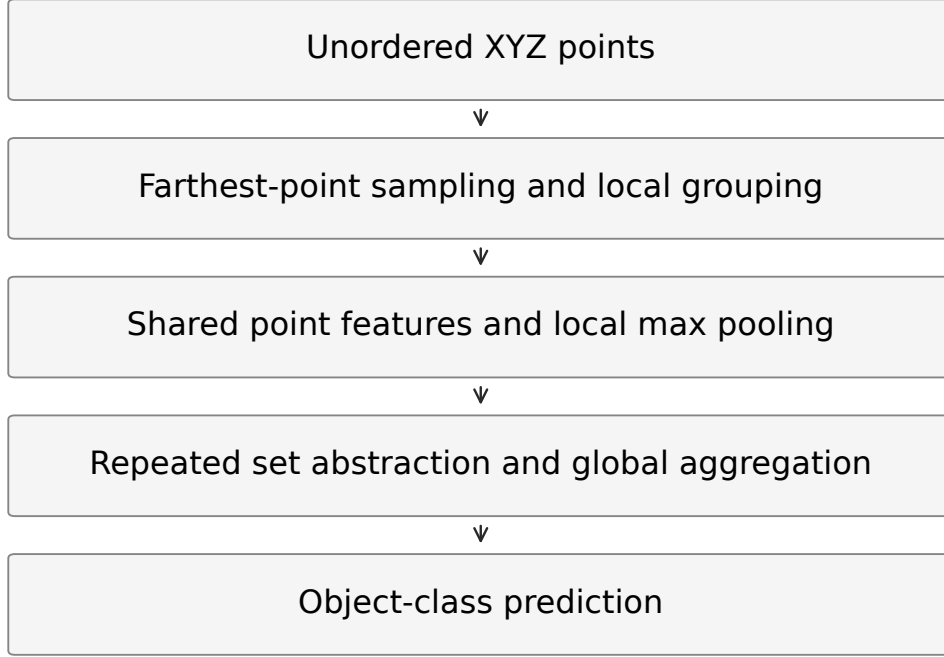


Figure 2.2: Sampling, local grouping and shared feature aggregation in PointNet++, redrawn from the published architecture [11].

Figure 2.2 connects these operations in the feature hierarchy.

The first step selects $M < N$ centroids by Farthest Point Sampling (FPS), as used in PointNet++ [11]. If the set of already selected centroids is $\mathcal{M}_t$, the next centroid is

$$\mathbf{c}_{t+1} = \arg \max_{\mathbf{p}_i \in \mathcal{P}} \min_{\mathbf{c} \in \mathcal{M}_t} \|\mathbf{p}_i - \mathbf{c}\|_2. \quad (2.9)$$

Here t counts selection steps, and $\mathcal{M}_t$ contains the centroids already selected. The inner minimum gives a candidate point’s distance to its nearest selected centroid. The outer maximum chooses the candidate with the largest such distance, namely the least-covered remaining point. Starting from an initial centroid, the procedure repeats until M centroids have been selected [11]. An initial-point rule and a rule for equal distances specify the selection completely. The operation selects existing points; it does not create new coordinates.

The second step constructs a local neighbourhood around each centroid $\mathbf{c}_j$. The radius grouping takes the form

$$\mathcal{N}_j(R) = \{\mathbf{p}_i : \|\mathbf{p}_i - \mathbf{c}_j\|_2 \leq R\}. \quad (2.10)$$

Here $R > 0$ is a radius in the coordinate units of the input; j indexes centroids. A k -nearest-neighbour query instead fixes the number k of neighbours and permits the neighbourhood radius to vary. The two alternatives are discussed in PointNet++ [11]. In order to obtain a translation-invariant local description, relative coordinates are obtained by subtracting the centroid,

$$\Delta \mathbf{p}_{ij} = \mathbf{p}_i - \mathbf{c}_j. \quad (2.11)$$

Unlike a k nearest neighbour query, the ball query of (2.10) guarantees a fixed metric scale, so that the learned features carry a consistent spatial scale in regions of differing density. For normalised coordinates this scale is relative to the normalisation radius, not a distance in metres.

The third step applies a shared mapping and a symmetric pooling within each local set, expressing the grouped points relative to their centroid and producing one feature vector per centroid according to

$$\mathbf{f}'_j = \max_{\mathbf{p}_i \in \mathcal{N}_j} \phi_\theta([\Delta \mathbf{p}_{ij}, \mathbf{f}_i]), \quad (2.12)$$

where ϕ_θ is a shared MLP with learned parameters θ , $\mathbf{f}'_j$ is the output feature, $[\cdot, \cdot]$ denotes concatenation, $\mathcal{N}_j$ is the ball-query set, and $\mathbf{f}_i$ is the point feature of the preceding layer; the first layer may use the coordinates alone or the coordinates together with the normal vectors. Stacking several set abstraction layers steadily reduces the number of centroids while enlarging the receptive field, and finally yields an object-level feature. In order to accommodate a non-uniform density, PointNet++ can employ Multi-Scale Grouping (MSG). Given several radii $R_1, \dots, R_L$, the local features are extracted separately and concatenated as

$$\mathbf{f}_j^{\text{MSG}} = \left\| \max_{\mathbf{p}_i \in \mathcal{N}_j(R_\ell)} \phi_{\theta_\ell}([\mathbf{p}_i - \mathbf{c}_j, \mathbf{f}_i]) \right\|_{\ell=1}^L. \quad (2.13)$$

Here L is the number of scales, ℓ indexes a scale, R_ℓ is its radius, and θ_ℓ denotes the parameters of that branch; $\|$ denotes feature concatenation. Single-scale grouping (SSG) uses one radius per layer, whereas multi-scale grouping (MSG) concatenates several radii [11]. A small radius attends to detail, whereas a large radius provides a wider context whenever the number of local points is insufficient.

2.3 3D Object Detection from Point Clouds

The preceding classifier maps one point cloud to one category. Detection extends this task to a scene containing several objects and background: it must decide which points support an object, distinguish instances, and estimate each object’s position and extent [8]. An oriented three-dimensional box can be written as

$$\mathbf{b} = (x_c, y_c, z_c, l, w, h, \psi)^\top, \quad (2.14)$$

where (x_c, y_c, z_c) is the centre, (l, w, h) are length, width and height, and ψ is yaw about the vertical axis. A detector returns

$$\hat{\mathcal{B}} = \{(\hat{\mathbf{b}}_j, \hat{c}_j, \hat{s}_j)\}_{j=1}^{n_{\text{det}}}. \quad (2.15)$$

Here n_{det} is the number of retained detections, j indexes one detection, $\hat{c}_j$ is its predicted class and $\hat{s}_j$ its confidence. The box notation is independent of a particular coordinate convention; the choice of vertical axis must be stated when writing its regression targets [8].

Both detection families considered below learn features, classify candidate objects and regress box parameters. They differ in the representation from which candidates are formed. PointRCNN predicts proposals from foreground points; CenterPoint predicts centre peaks from a spatial feature map [8, 9]. Figure 2.3 shows where these processing paths diverge. The distinction matters for upsampling because adding points changes the input to a sequence of sampling and aggregation operations, not directly the final boxes.

2.3.1 PointRCNN

PointRCNN is a two-stage, point-based detector. Its first stage uses a PointNet++ backbone to obtain pointwise semantic features, segments points into foreground and background, and generates proposals directly from foreground points. Its second stage pools the points around each proposal and refines that proposal in local coordinates [8]. This connects the local feature hierarchy of Section 2.2.3 to instance localisation.

Let $\mathbf{f}_i \in \mathbb{R}^F$ be the F -dimensional feature of point $\mathbf{p}_i$. A learned scalar mapping g_θ produces a foreground probability

$$\hat{q}_i = \text{sigmoid}(g_\theta(\mathbf{f}_i)), \quad \text{sigmoid}(u) = \frac{1}{1 + e^{-u}}. \quad (2.16)$$

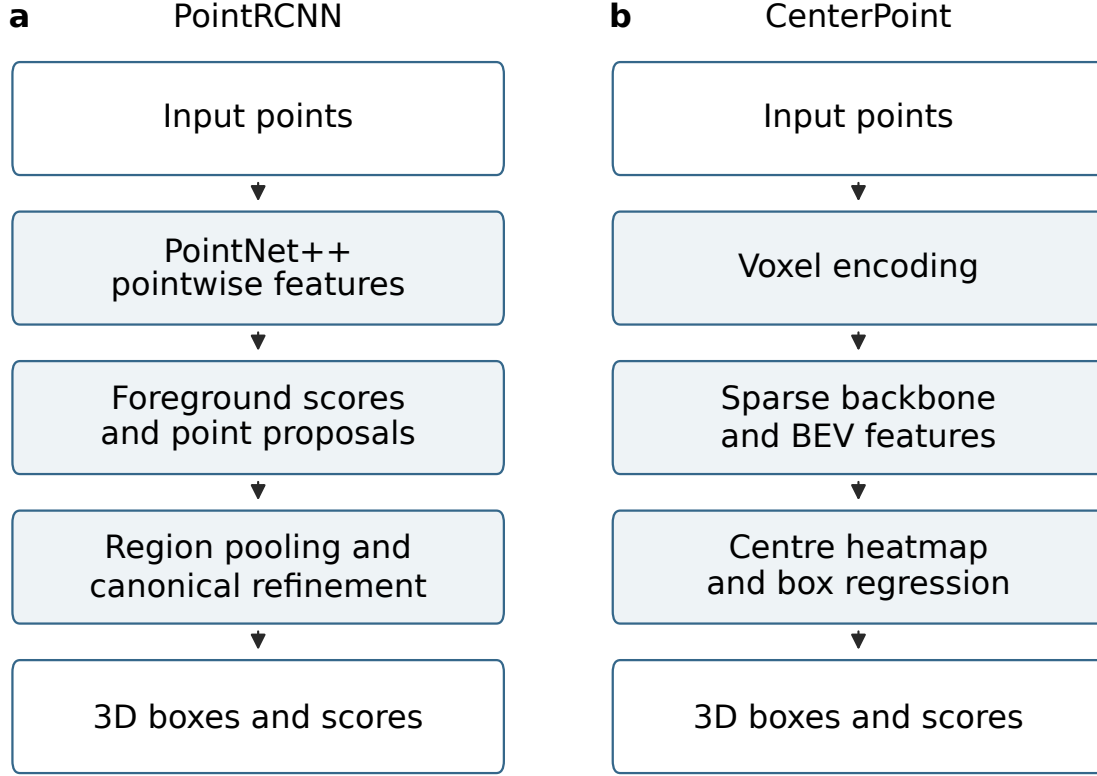


Figure 2.3: Processing paths of PointRCNN and a voxel-based CenterPoint configuration, redrawn from the method descriptions [8, 9]. The diagram shows the principal representations; it omits layer sizes and implementation-specific input limits.

PointRCNN derives foreground labels from annotated three-dimensional boxes and uses focal loss to address the foreground–background imbalance [8, 13]. For binary label $q_i \in \{0, 1\}$, define the probability of the correct binary class and its weighting as

$$\begin{aligned} p_{t,i} &= q_i \hat{q}_i + (1 - q_i)(1 - \hat{q}_i), \\ \alpha_{t,i} &= q_i \alpha_f + (1 - q_i)(1 - \alpha_f). \end{aligned} \quad (2.17)$$

The per-point focal loss is

$$\ell_{\text{seg},i} = -\alpha_{t,i}(1 - p_{t,i})^{\gamma_f} \log p_{t,i}. \quad (2.18)$$

Here $\alpha_f \in (0, 1)$ controls class weighting and $\gamma_f \geq 0$ controls suppression of easily classified examples. If the correct class already has probability close to one, the factor

$(1 - p_{t,i})^{\gamma_f}$ reduces its contribution. For $q_i = 0$ the formula uses $1 - \hat{q}_i$, so it supervises background as well as foreground. This is the binary focal-loss formulation of Lin et al.; it does not specify the hyperparameters of the experiments in this thesis [13].

A foreground point provides a local reference from which PointRCNN predicts the offset to the object centre, box dimensions and orientation. For horizontal position and yaw, PointRCNN combines a discrete bin classification with a continuous residual within the selected bin [8]. To express the centre-offset construction without mixing camera and LiDAR axes, let ζ denote either horizontal offset. For a horizontal offset $\zeta \in [-S, S)$ and bin width $\delta > 0$,

$$b^* = \left\lfloor \frac{\zeta + S}{\delta} \right\rfloor, \quad c_b = -S + (b + \tfrac{1}{2})\delta, \quad r^* = \frac{\zeta - c_{b^*}}{\delta}. \quad (2.19)$$

Here $S > 0$ is the half-width of the search range, b^* the target bin, c_b the centre of bin b , and r^* its dimensionless residual. The floor operation selects a discrete interval. With predicted bin $\hat{b}$ and predicted residual $\hat{r}_{\hat{b}}$, the offset is recovered as $\hat{\zeta} = c_{\hat{b}} + \delta \hat{r}_{\hat{b}}$. Angular regression uses the analogous construction over an angular interval. The original paper uses camera-frame x and z as horizontal axes; this does not make z horizontal in a LiDAR coordinate convention [8].

The bin label is supervised by cross-entropy and the residual by smooth L_1 regression. For scalar residual error e , the latter is

$$\text{smooth}_{L_1}(e) = \begin{cases} \frac{1}{2}e^2, & |e| < 1, \\ |e| - \frac{1}{2}, & |e| \geq 1. \end{cases} \quad (2.20)$$

It is quadratic near zero and linear for larger errors. Thus the network first learns a coarse location and then a finer correction, while other box terms such as dimensions are regressed separately [8]. Proposal filtering removes redundant candidates before the refinement stage.

The second stage pools point coordinates and features within an enlarged proposal region. For proposal centre $\mathbf{c}$ and yaw ψ , a z -up notation for its local coordinates is

$$\mathbf{p}_i^{\text{can}} = \mathbf{R}_z(-\psi)(\mathbf{p}_i - \mathbf{c}), \quad \mathbf{R}_z(\varphi) = \begin{bmatrix} \cos \varphi & -\sin \varphi & 0 \\ \sin \varphi & \cos \varphi & 0 \\ 0 & 0 & 1 \end{bmatrix}. \quad (2.21)$$

Here $\mathbf{R}_z(\varphi)$ rotates through angle φ around the vertical axis. Translation removes the proposal centre and inverse yaw aligns its heading. The refinement network then predicts residual box parameters and confidence from the aligned region. PointRCNN uses the corresponding transformation about the vertical axis of its working frame [8].

These stages explain two possible routes by which generated points can affect a point-based detector. They can change the pointwise features and foreground proposals, and they can change the points pooled for refinement. Moreover, if an implementation samples an input set $\mathcal{Y}$ to a budget $B_p \leq |\mathcal{Y}|$, it actually processes

$$\mathcal{P}_{\text{det}} = \mathcal{S}_{B_p}(\mathcal{Y}), \quad |\mathcal{P}_{\text{det}}| = B_p, \quad (2.22)$$

where $\mathcal{S}_{B_p}$ denotes the chosen sampling operator. PointRCNN uses input point sampling [8]; the consequence is that file-level cardinality need not equal the number of points consumed. The implemented full-frame sampling and the separate region-split controls are distinguished in Sections 4.8.1 and 4.12.2. The next detector aggregates points into spatial cells before forming object candidates, introducing a different kind of reduction.

2.3.2 CenterPoint

CenterPoint extends the centre-keypoint idea of CenterNet to three-dimensional detection [9, 14]. It predicts a class-specific centre heatmap in bird’s-eye view and regresses box attributes at the predicted centres. The centre head can operate with voxel or pillar encoders; voxelisation is a backbone choice rather than the definition of every CenterPoint variant [9]. The voxel-based path is explained here to show how point density and spatial occupancy differ.

With voxel dimensions $\mathbf{v} = (v_x, v_y, v_z)^\top$, where each entry is positive, and a lower spatial bound $\mathbf{o}$, a point is assigned to the grid cell

$$\mathbf{k}_i = \lfloor (\mathbf{p}_i - \mathbf{o}) \oslash \mathbf{v} \rfloor. \quad (2.23)$$

Here $\oslash$ denotes componentwise division and the floor acts on each coordinate. The integer vector $\mathbf{k}_i$ identifies a voxel. A voxel feature encoder combines the points within each occupied cell, and sparse convolutions propagate information across the occupied grid [15, 16].

For example, the learned voxel encoding used by VoxelNet combines each point’s features with its displacement from the voxel centroid before symmetric pooling [15].

Abstracting this operation,

$$\mathbf{f}_k = \max_{i:\mathbf{k}_i=k} \phi_\theta([\mathbf{u}_i, \mathbf{p}_i - \bar{\mathbf{p}}_k]). \quad (2.24)$$

The vector $\bar{\mathbf{p}}_k$ is the mean position in voxel k , $\mathbf{u}_i$ is the coordinate-and-attribute record from Equation (2.2), and ϕ_θ is a shared learned mapping. The maximum operates channelwise. Mean encoders instead average retained point records; these are alternative implementations, not the same operation as VoxelNet’s learned encoder. The spatial backbone converts the encoded grid into the bird’s-eye-view feature map used by the centre head [9].

For one annotated centre projected onto a feature grid at integer position $\boldsymbol{\mu}$, a Gaussian target has the form

$$H(\mathbf{u}) = \exp\left(-\frac{\|\mathbf{u} - \boldsymbol{\mu}\|_2^2}{2\sigma^2}\right). \quad (2.25)$$

Here $\mathbf{u}$ is a two-dimensional grid index, and $\sigma > 0$ determines the target’s spread in grid cells. The target is centred on a discrete grid location; a separate offset compensates for the continuous centre’s displacement from that location. When several objects share a class, their Gaussian targets are combined by an elementwise maximum. This heatmap construction originates in CenterNet and is used in CenterPoint [9, 14].

The centre head is trained with a variant of focal loss adapted to Gaussian targets. For target value H_{uv} and predicted value $\hat{H}_{uv}$ at grid position (u, v) and class channel c , a per-location term is

$$\ell_{uv} = \begin{cases} -(1 - \hat{H}_{uv})^a \log \hat{H}_{uv}, & H_{uv} = 1, \\ -(1 - H_{uv})^b (\hat{H}_{uv})^a \log(1 - \hat{H}_{uv}), & H_{uv} < 1. \end{cases} \quad (2.26)$$

The exponents $a, b \geq 0$ control downweighting, and the complete heatmap loss sums these terms with normalisation by the number of target centres. Locations around a Gaussian peak are downweighted as negatives, rather than treated as additional positive object centres [14]. This supervision differs from the binary foreground loss in PointRCNN.

At a detected peak, additional heads regress the centre offset, height, dimensions and orientation [9]. Orientation can be represented by $(\sin \psi, \cos \psi)$ and decoded by the two-argument arctangent, which avoids a scalar discontinuity where an angle wraps around. During training, the box attributes at annotated centres are supervised with

weighted L_1 regression terms alongside the centre-heatmap loss. Each weight scales the contribution of an attribute, such as size or orientation; the loss therefore supervises localisation as well as the presence of a centre [9]. At inference, the predicted attributes are decoded into metric boxes and redundant boxes are suppressed.

Several input points can occupy the same voxel. Added points may change the encoded feature of that cell without creating another occupied cell; the outcome depends on the encoder and point-selection rule [15]. This distinction explains how point-based and voxel-based representations process density differently. It is not a geometric accuracy metric or a claim that one detector is always more robust. With the downstream processing paths established, the next section explains how the additional points are generated.

2.4 Point Cloud Upsampling

Classification and detection produce semantic predictions from an input representation. Upsampling instead changes that representation: it infers a denser point set from sparse spatial support. Given $\mathcal{X} = \{\mathbf{x}_i\}_{i=1}^N$, a fixed-ratio upsampling model is written as

$$\mathcal{Y} = \mathcal{U}_\theta(\mathcal{X}; r) = \{\mathbf{y}_j\}_{j=1}^{rN}, \quad |\mathcal{Y}| = rN. \quad (2.27)$$

Here $r > 1$ is an integer expansion ratio, θ denotes the parameters of a learned model, and j indexes output points. This formulation follows the sparse-to-dense task used by PU-Net [4]; a geometric optimisation method need not have learned parameters. The output should represent the underlying surface, preserve its local structure and avoid excessive clustering. Cardinality alone cannot establish any of those properties.

It is not presupposed that $\mathcal{X} \subseteq \mathcal{Y}$. A coordinate regression network can predict the whole output anew, while a residual network can move every repeated input anchor [4, 6]. Thus even an architecture that begins by repeating input coordinates need not retain the original measurements exactly. Whether observations are explicitly kept is a separate property of an output construction.

The methods below illustrate five ways of addressing the ambiguity of sparse-to-dense reconstruction: geometric resampling, hierarchical feature expansion, graph-based expansion, graph attention and conditional diffusion. Related developments include adversarial distribution learning in PU-GAN, progressive patch refinement, and edge-supervised consolidation [17, 18, 19]. The following descriptions separate

the published method principles from the wrappers and downstream training choices introduced later in the thesis.

2.4.1 Edge-Aware Resampling (EAR)

EAR is an optimisation-based resampling method developed by Huang et al. to consolidate noisy point sets while preserving sharp features [3]. The central difficulty is that normals estimated close to a sharp edge can mix information from different surface sides. EAR therefore first resamples away from uncertain edge regions to obtain reliable oriented points. It then inserts points progressively towards the gaps near edges. This two-stage sequence is the reason for the term edge-aware resampling; it is more specific than simply smoothing a cloud and adding points.

Let $\mathbf{n}_i$ be a unit normal associated with point $\mathbf{p}_i$. EAR combines a spatial weight and a normal-similarity weight [3], giving

$$\kappa_p(d) = \exp(-d^2/\sigma_p^2), \quad \kappa_n(\mathbf{n}_i, \mathbf{n}_j) = \exp \left[- \left(\frac{1 - \mathbf{n}_i^\top \mathbf{n}_j}{1 - \cos \sigma_n} \right)^2 \right]. \quad (2.28)$$

Here $d = \|\mathbf{p}_i - \mathbf{p}_j\|_2$ is spatial distance, $\sigma_p > 0$ sets its neighbourhood scale, and $0 < \sigma_n < \pi$ is an angular bandwidth in radians. The dot product of unit normals is the cosine of their angular difference. Nearby points with compatible normals receive high weight; disagreeing normals are downweighted even when the points are close. Using $w_{ij} = \kappa_p(\|\mathbf{p}_i - \mathbf{p}_j\|_2) \kappa_n(\mathbf{n}_i, \mathbf{n}_j)$, the weighted normal update is

$$\bar{\mathbf{n}}_i = \frac{\sum_{j \in \mathcal{N}(i)} w_{ij} \mathbf{n}_j}{\sum_{j \in \mathcal{N}(i)} w_{ij}}. \quad (2.29)$$

The set $\mathcal{N}(i)$ contains spatial neighbours and $\bar{\mathbf{n}}_i$ is their weighted mean direction; a nonzero vector can be normalised when a unit direction is required. This is the bilateral normal-smoothing step, not the entire EAR algorithm. EAR alternates normal processing with locally anisotropic projection to move samples away from ambiguous edges, because normal smoothing alone can retain incorrect assignments [3].

For a new sample, EAR chooses a base position between existing points, selects a projection direction compatible with surrounding normals, and computes a projection distance towards the latent surface. Positions and normals influence all three decisions. Repeating this process fills sparse regions and approaches sharp features from better-supported regions [3]. The method does not require a trained generator, but its geometric neighbourhoods and normals must be meaningful. Sparse or mixed-surface

neighbourhoods therefore warrant inspection when the method is used outside the surface-scan setting of its original evaluation. Learned methods address the reconstruction ambiguity differently, through features estimated from training examples.

2.4.2 PU-Net

PU-Net is an early end-to-end network dedicated to point cloud upsampling. It consists of multi-level feature embedding, feature expansion and coordinate reconstruction [4]. Its encoder adapts the hierarchical feature extraction of PointNet++ to local surface patches. Because deeper levels contain fewer sampled locations, their features are interpolated back to the original points before aggregation.

For a point $\mathbf{x}$ and its three nearest feature locations $\mathbf{x}_j^{(\ell)}$ at level ℓ , PU-Net uses inverse-distance interpolation [4], expressed as

$$\tilde{\mathbf{f}}^{(\ell)}(\mathbf{x}) = \frac{\sum_{j=1}^3 w_j(\mathbf{x}) \mathbf{f}^{(\ell)}(\mathbf{x}_j^{(\ell)})}{\sum_{j=1}^3 w_j(\mathbf{x})}, \quad w_j(\mathbf{x}) = \frac{1}{\|\mathbf{x} - \mathbf{x}_j^{(\ell)}\|_2}. \quad (2.30)$$

Here $\mathbf{f}^{(\ell)}$ is the feature field on the sampled level. The expression applies at positive distances; an exactly coincident location takes the coincident feature rather than dividing by zero. Features from the different levels are projected to compatible channel dimensions and concatenated, giving $\mathbf{F} \in \mathbb{R}^{N \times F}$. The number F denotes the resulting feature width. Local detail and broader patch context are therefore made available to the expansion stage at every original point.

PU-Net uses r branches of independent pointwise convolutions, followed by a reshape [4], producing

$$\mathbf{F}' = \text{RS} \left[C_1^2(C_1^1(\mathbf{F})), \dots, C_r^2(C_r^1(\mathbf{F})) \right] \in \mathbb{R}^{rN \times F_2}. \quad (2.31)$$

Each C_m^1, C_m^2 is a learned 1×1 convolution in branch m . The bracket concatenates channels into an $N \times rF_2$ tensor; RS reshapes it to rN rows of width F_2 . Separate branch weights reduce the correlation between the new features. A shared coordinate head then maps each expanded row to three coordinates, $\mathbf{y}_{i,m} = \psi_\theta(\mathbf{f}'_{i,m})$, with i indexing an input feature and $m = 1, \dots, r$ its expanded features.

PU-Net uses Earth Mover’s Distance (EMD) for reconstruction. Let $\mathcal{Y}$ be the predicted point set and $\mathcal{R}$ its dense reference, with $|\mathcal{Y}| = |\mathcal{R}|$. The reconstruction loss is

$$\mathcal{L}_{\text{rec}} = \min_{\substack{\varphi: \mathcal{Y} \rightarrow \mathcal{R} \\ \varphi \text{ bijective}}} \sum_{\mathbf{y} \in \mathcal{Y}} \|\mathbf{y} - \varphi(\mathbf{y})\|_2. \quad (2.32)$$

Here $\mathbf{y}$ denotes an output point and $\varphi(\mathbf{y})$ its assigned reference point. The bijective map φ pairs every output with exactly one reference point, unlike nearest-neighbour Chamfer matching, which permits many outputs to share one reference [4, 20]. Equal cardinality is required by this discrete bijection. Dividing the sum by $|\mathcal{Y}|$ would give a mean variant with different numerical scaling.

Reconstruction alone can leave clustered outputs. PU-Net therefore adds the local repulsion term

$$\mathcal{L}_{\text{rep}} = \sum_{i=1}^{rN} \sum_{j \in \mathcal{N}_k(i)} -d_{ij} \exp(-d_{ij}^2/h^2), \quad d_{ij} = \|\mathbf{y}_i - \mathbf{y}_j\|_2. \quad (2.33)$$

Here $\mathcal{N}_k(i)$ contains the k nearest other output points and $h > 0$ controls the decay scale. The negative distance favours separation of very close neighbours, while the exponential limits the range over which the interaction is effective. The published loss combines reconstruction, weighted repulsion and parameter regularisation [4]. These terms are expressed in the coordinates used for training. Patch normalisation and its inverse therefore affect the scale of both reconstructed geometry and local separation. The next method changes the expansion operation itself by incorporating graph neighbours.

2.4.3 PU-GCN

PU-GCN proposes an Inception DenseGCN feature extractor and the NodeShuffle upsampling module [5]. Graph convolution represents local relations between points explicitly. For features $\mathbf{h}_i$ and neighbour indices $j \in \mathcal{N}_k(i)$, the EdgeConv construction used in this graph-convolution family is

$$\mathbf{e}_{ij} = \phi_{\theta}([\mathbf{h}_i, \mathbf{h}_j - \mathbf{h}_i]), \quad \mathbf{h}'_i = \max_{j \in \mathcal{N}_k(i)} \mathbf{e}_{ij}. \quad (2.34)$$

Here $\mathcal{N}_k(i)$ is the set of indices of the k neighbours of node i ; $\mathbf{h}_i$ and $\mathbf{h}_j$ are the centre and neighbour features. The learned shared mapping ϕ_{θ} transforms their concatenation into the edge feature $\mathbf{e}_{ij}$. Channelwise max aggregation produces the updated node

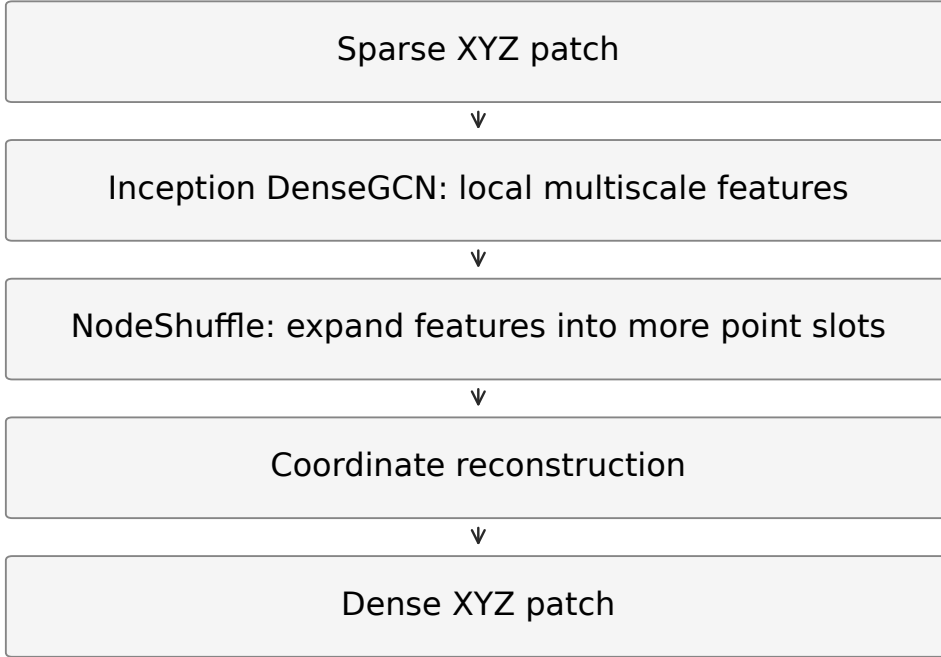


Figure 2.4: Graph feature extraction, NodeShuffle and coordinate reconstruction in PU-GCN, redrawn from the published method [5].

feature $\mathbf{h}'_i$ [21]. The centre feature preserves information about the point itself, while the feature difference describes its relationship to a neighbour. Neighbourhoods may be constructed in a coordinate or feature space; the space and graph rule are part of the architecture specification.

Inception DenseGCN combines densely connected graph layers with branches having different receptive fields. Dilated neighbourhoods let the branches reach different spatial extents without simply duplicating one local operation. Their features are fused with broader pooled information [5]. Dense connections retain intermediate features so that later processing can use both earlier local detail and deeper representations. Figure 2.4 shows how feature extraction leads to expansion and coordinate reconstruction.

The graph layers first embed coordinates and then combine features from different receptive fields. The block outputs are combined through residual connections and passed through a pointwise bottleneck before upsampling [5]. Denote the resulting feature of point i by $\mathbf{f}_i^{\text{multi}}$. Stacking these features as rows gives $\mathbf{F} \in \mathbb{R}^{N \times F}$, where the

scalar F is their channel width. Thus $\mathbf{F}$ is the processed output of feature extraction, not an independently supplied input.

NodeShuffle applies a graph convolution to $\mathbf{F}$ that expands each node’s channel width from F to rF ,

$$\mathbf{F}_{\text{exp}} = \text{GCN}_{\theta}(\mathbf{F}) \in \mathbb{R}^{N \times rF}. \quad (2.35)$$

The operator GCN_{θ} denotes the graph-convolution layer with learned parameters θ , and $\mathbf{F}_{\text{exp}}$ contains its expanded features. Periodic shuffling then rearranges groups of F channels into additional rows,

$$\mathbf{F}_{\text{shuffle}} = \text{PS}(\mathbf{F}_{\text{exp}}) \in \mathbb{R}^{rN \times F}. \quad (2.36)$$

The matrix $\mathbf{F}_{\text{shuffle}}$ contains one feature row per prospective output point. The operator PS is a deterministic rearrangement, not a learned coordinate displacement. The preceding graph convolution learns the new features; the following coordinate head converts the shuffled features into dense point positions [5]. PU-GCN trains this output with a squared Chamfer reconstruction loss, whose distance convention is distinguished from the unsquared metric in Section 2.5.

The construction makes neighbourhood selection relevant to transfer. For fixed k , a sparse region can have a much larger neighbourhood radius than a dense one. A graph that crosses unrelated surfaces presents different relations to the learned mapping. The graph construction therefore determines which local relationships reach the learned feature extractor.

Reconstruction training and downstream detector retraining have different targets. PU-GCN learns dense coordinates using a geometric reconstruction loss [5]. PointRCNN and CenterPoint learn object classification and box localisation through their detection losses [8,9]. With a fixed upsampler, its outputs can be supplied as detector-training inputs while the detection loss updates only the detector parameters. The saved point-cloud files are not regenerated by that optimisation.

The detector-side retraining studied in this thesis follows this separation: PointRCNN and CenterPoint are adapted to PU-GCN inputs, while the PU-GCN upsampling network retains its fixed checkpoint. Section 4.11 specifies the completed training conditions; Figure 4.3 identifies which network is updated.

Attention-based upsampling changes feature generation itself by combining local relationships with the wider patch context.

2.4.4 PU-EdgeFormer

PU-EdgeFormer combines local graph features with multi-head self-attention [6]. Its encoder stacks EdgeFormer units, followed by feature extension and coordinate reconstruction. To explain the modification, standard scaled dot-product attention first forms queries, keys and values from a feature matrix $\mathbf{H} \in \mathbb{R}^{N \times F}$ through learned projections,

$$\mathbf{Q} = \mathbf{H}\mathbf{W}_Q, \quad \mathbf{K} = \mathbf{H}\mathbf{W}_K, \quad \mathbf{V} = \mathbf{H}\mathbf{W}_V, \quad (2.37)$$

$$\text{Attn}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right) \mathbf{V}. \quad (2.38)$$

The learned matrices $\mathbf{W}_Q, \mathbf{W}_K$ produce d_k -dimensional query and key vectors; $\mathbf{W}_V$ produces value vectors. The dot product scores relationships between locations. Row-wise softmax turns each row into nonnegative weights summing to one, and multiplication by $\mathbf{V}$ gives a weighted feature combination. Scaling by $\sqrt{d_k}$ controls the magnitude of the dot products. Multiple heads perform these operations in separate learned subspaces, concatenate their outputs and apply an output projection [22].

The defining change in PU-EdgeFormer is that EdgeConv replaces the linear query, key and value transformations. In an architectural shorthand,

$$\mathbf{Q} = \text{EdgeConv}_Q(\mathbf{H}), \quad \mathbf{K} = \text{EdgeConv}_K(\mathbf{H}), \quad \mathbf{V} = \text{EdgeConv}_V(\mathbf{H}). \quad (2.39)$$

Each branch groups nearest neighbours, convolves the grouped features and max-pools them. The resulting query, key and value features therefore contain local relationships before attention combines information across the patch [6].

The feature-extension stage shuffles an $N \times F$ tensor into $rN \times (F/r)$ features and applies MLPs, giving a dense feature matrix $\mathbf{F}'$. If $\tilde{\mathbf{X}} \in \mathbb{R}^{rN \times 3}$ repeats each input coordinate r times, the coordinate reconstruction is

$$\mathbf{Y} = \tilde{\mathbf{X}} + \Phi_\theta(\mathbf{F}'). \quad (2.40)$$

The mapping Φ_θ predicts three-dimensional offsets, and the rows of $\mathbf{Y}$ form the output set [6]. Repeated anchors do not imply repeated outputs because their predicted offsets can differ. Equally, residual reconstruction does not guarantee exact retention of the measured coordinates. The method trains with squared Chamfer distance. Attention expands the available context within the input patch, but cannot establish whether

that patch corresponds to a coherent observed surface. Conditional diffusion takes a different route by generating the output through successive denoising steps.

2.4.5 PDANS

PDANS combines a conditional diffusion model with Adaptive Noise Suppression (ANS) and a TreeTrans decoder [7]. Its starting point is the denoising diffusion probabilistic model (DDPM), which learns to reverse a gradual Gaussian noising process [23]. The sparse point cloud conditions this reversal so that the generated dense points correspond to the supplied geometry.

Let $\mathbf{Z}_0 \in \mathbb{R}^{3rN}$ be the vectorised dense point coordinates, and let $t = 1, \dots, T$ index noising steps. A forward step is

$$q(\mathbf{Z}_t | \mathbf{Z}_{t-1}) = \mathcal{N}(\sqrt{1 - \beta_t} \mathbf{Z}_{t-1}, \beta_t \mathbf{I}), \quad (2.41)$$

where $\beta_t \in (0, 1)$ is the variance increment, $\mathbf{I}$ is the identity in $3rN$ dimensions, and $\mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$ denotes a Gaussian with mean $\boldsymbol{\mu}$ and covariance $\boldsymbol{\Sigma}$. With $\alpha_t = 1 - \beta_t$ and $\bar{\alpha}_t = \prod_{s=1}^t \alpha_s$, the state can be sampled directly as

$$\mathbf{Z}_t = \sqrt{\bar{\alpha}_t} \mathbf{Z}_0 + \sqrt{1 - \bar{\alpha}_t} \boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}). \quad (2.42)$$

The product $\bar{\alpha}_t$ is the remaining signal fraction after t steps; the product index s runs over the forward steps up to t , and $\boldsymbol{\epsilon}$ is standard Gaussian noise. These are the DDPM forward equations also used to introduce PDANS [7, 23].

If $\mathbf{c}$ denotes the conditioning representation of the sparse input and requested expansion, a learned reverse step is

$$p_\theta(\mathbf{Z}_{t-1} | \mathbf{Z}_t, \mathbf{c}) = \mathcal{N}(\boldsymbol{\mu}_\theta(\mathbf{Z}_t, t, \mathbf{c}), \sigma_t^2 \mathbf{I}). \quad (2.43)$$

Here $\boldsymbol{\mu}_\theta$ is the learned reverse mean and σ_t^2 the reverse variance. The conditioning vector $\mathbf{c}$ encodes the sparse input and requested expansion. In the noise-prediction parameterisation, $\boldsymbol{\epsilon}_\theta$ is the learned function that estimates the noise present at step t , given the noisy coordinates and their conditioning. It is trained through the mean-squared objective

$$\mathcal{L}_{\text{diff}} = \mathbb{E}_{\mathbf{Z}_0, t, \boldsymbol{\epsilon}} [\|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\mathbf{Z}_t, t, \mathbf{c})\|_2^2]. \quad (2.44)$$

The expectation averages over training shapes, sampled times and sampled noise; $\mathbf{Z}_t$ is constructed by Equation (2.42). At inference the process starts from noise and repeat-

edly estimates a less noisy state. Unlike a direct coordinate head, generation therefore depends on a sequence of denoising updates. The squared noise-prediction loss supervises the denoising process. Geometric metrics instead assess the final generated coordinates, as described in Section 2.5 [7, 23].

The ANS module constructs neighbourhoods for the input points and uses attention to calculate weights for refining coordinates and features [7]. For centre $\mathbf{x}_i$ and neighbour $\mathbf{x}_{ij}$, write the coordinate-based attention score as

$$a_{ij} = \frac{\exp(s_{ij})}{\sum_{m \in \mathcal{N}_k(i)} \exp(s_{im})}, \quad s_{ij} = \frac{q_\theta(\mathbf{x}_i)^\top k_\theta(\mathbf{x}_{ij} - \mathbf{x}_i)}{\sqrt{d_q}}. \quad (2.45)$$

Here q_θ and k_θ are learned coordinate transformations, d_q is their channel dimension, and $\mathcal{N}_k(i)$ indexes the k neighbours. The query uses the centre coordinates and the key uses relative neighbour coordinates. With a learned linear feature transform γ_θ , the weighted neighbourhood representation is

$$\mathbf{w}_i = \sum_{j \in \mathcal{N}_k(i)} a_{ij} \gamma_\theta(\mathbf{f}_{ij}). \quad (2.46)$$

The vector $\mathbf{f}_{ij}$ is the neighbour feature. ANS uses learned activations of $\mathbf{w}_i$ to rescale both the centre coordinates and its features, then fuses the refined information. This is a learned suppression mechanism; the attention weights are not independently validated probabilities that a point is noise [7].

TreeTrans propagates features from coarser tree nodes to finer descendants using transformations of the current node and information from ancestor nodes. Local and global attention then combine the tree features with corresponding encoder features. The conditional and diffusion networks use these representations to guide dense-point generation [7]. ANS addresses unreliable input support, while TreeTrans improves how features at different levels enter the decoder. The resulting surface quality must still be checked against downstream predictions; a denoising objective does not directly optimise class labels or detection boxes. This brings the discussion to evaluation: the metrics must distinguish geometric fidelity, point distribution and task success.

2.5 Evaluation Metrics

The methods above produce point coordinates, whereas the downstream models produce labels or boxes. These outputs answer different questions and require different

metrics. Geometric evaluation compares a point set with a reference surface or reference samples. Classification and detection evaluation compare predictions with semantic annotations [5, 8, 12]. Their definitions are given here; the implemented aggregation rules and experimental reference construction are specified in Chapter 4.

2.5.1 Geometric Metrics

Let $\mathcal{Y}$ be a nonempty output point set and $\mathcal{R}$ a nonempty reference set. Define the distance from a point $\mathbf{y}$ to its nearest reference point as

$$d(\mathbf{y}, \mathcal{R}) = \min_{\mathbf{r} \in \mathcal{R}} \|\mathbf{y} - \mathbf{r}\|_2. \quad (2.47)$$

This nearest-neighbour construction is used in Chamfer and Hausdorff comparisons [5, 20]. It requires a common coordinate frame. If the coordinates are measured in metres, an unsquared distance is in metres; after unit-sphere normalisation it is dimensionless. Squaring the distance changes its units and its sensitivity to large errors.

A symmetric averaged, unsquared Chamfer distance is

$$\begin{aligned} \text{CD}(\mathcal{Y}, \mathcal{R}) = & \frac{1}{|\mathcal{Y}|} \sum_{\mathbf{y} \in \mathcal{Y}} \min_{\mathbf{r} \in \mathcal{R}} \|\mathbf{y} - \mathbf{r}\|_2 \\ & + \frac{1}{|\mathcal{R}|} \sum_{\mathbf{r} \in \mathcal{R}} \min_{\mathbf{y} \in \mathcal{Y}} \|\mathbf{r} - \mathbf{y}\|_2. \end{aligned} \quad (2.48)$$

The cardinality symbols $|\mathcal{Y}|$ and $|\mathcal{R}|$ denote the numbers of points. The first term measures how far outputs lie from the reference; the second measures how well the output covers the reference. Both directions are necessary: points concentrated on one small part of a surface can have a small first term while leaving much of the reference uncovered. This bidirectional construction follows the nearest-neighbour matching used for point-set reconstruction [5, 20]. Because the terms are averages, a small number of severe deviations can be hidden by many close matches.

Distance conventions must be distinguished explicitly. Fan et al. define the sum of squared nearest-neighbour distances, and PU-GCN uses averaged squared Euclidean distances in its Chamfer loss [5, 20]. Equation (2.48) defines an unsquared, averaged variant, so it must not be cited as numerically identical to those losses. Replacing each norm by its square produces the averaged squared variant; multiplying a reported value by a display factor does not convert one variant into the other.

The symmetric Hausdorff distance takes the worst nearest-neighbour error in either direction [24], with

$$\begin{aligned} h(\mathcal{Y}, \mathcal{R}) &= \max_{\mathbf{y} \in \mathcal{Y}} \min_{\mathbf{r} \in \mathcal{R}} \|\mathbf{y} - \mathbf{r}\|_2, \\ \text{HD}(\mathcal{Y}, \mathcal{R}) &= \max\{h(\mathcal{Y}, \mathcal{R}), h(\mathcal{R}, \mathcal{Y})\}. \end{aligned} \quad (2.49)$$

Here h is the directed distance; reversing its arguments exchanges output deviation and reference coverage. HD highlights isolated outliers and uncovered regions that may be diluted in CD. It is correspondingly sensitive to the worst reference error. PU-GCN reports HD alongside CD and P2F to characterise different geometric properties [5].

If a continuous reference mesh surface S is available, a mean unsquared point-to-surface distance is

$$\text{P2F}(\mathcal{Y}, S) = \frac{1}{|\mathcal{Y}|} \sum_{\mathbf{y} \in \mathcal{Y}} \min_{\mathbf{s} \in S} \|\mathbf{y} - \mathbf{s}\|_2. \quad (2.50)$$

Here $\mathbf{s}$ ranges over the surface, including face interiors, rather than only its vertices. Such mesh-based distances are used in point-cloud consolidation and upsampling evaluation [5, 19]. They reduce the influence of how densely a discrete reference is sampled. However, P2F is one-directional: a few points on the surface can obtain a small value without covering the whole surface. The mesh and output must share a coordinate frame and scale, and the mesh must represent the intended target geometry.

Geometric fidelity does not fully describe distribution. PU-Net uses local repulsion to discourage clustered predictions, while PU-GAN includes a separate uniformity objective [4, 17]. A basic local spacing quantity is

$$d_i^{\text{NN}} = \min_{j \neq i} \|\mathbf{y}_i - \mathbf{y}_j\|_2. \quad (2.51)$$

Here i and j index distinct stored rows, so repeated coordinates can produce zero spacing. Neighbour distances underlie the local repulsion and uniformity constructions in PU-Net and PU-GAN [4, 17]. A nearest-neighbour distribution, a disk-count uniformity statistic and a repulsion loss describe related but different properties. Their neighbourhood definition, scale and aggregation must be given before their numbers are compared. A metric abbreviated as a uniformity coefficient should not silently be replaced by another such statistic.

CD and HD are mathematically defined for nonempty sets of unequal cardinality; the discrete EMD bijection in Equation (2.32) is not. Equal point counts in a method comparison are an experimental control, not a condition needed to evaluate a nearest-neighbour minimum. Changing the sample count or reference density can change

coverage and matching distances, so both must be reported. There is no general theorem that adding arbitrary points improves the symmetric CD: additional outliers can worsen its output-to-reference term. The implemented equal-count protocol, reference construction and uniformity statistic are defined in Chapter 4.

2.5.2 Classification Metrics

Point-cloud classification studies report overall accuracy (OA) and mean class accuracy (mAcc) [12]. For M evaluated samples with true labels y_m and predicted labels $\hat{y}_m$, OA is

$$\text{OA} = \frac{1}{M} \sum_{m=1}^M \mathbb{I}[\hat{y}_m = y_m]. \quad (2.52)$$

The indicator $\mathbb{I}[\cdot]$ equals one when the condition holds and zero otherwise; m indexes an object, not an individual point. OA weights each evaluated object equally. For class c containing $M_c > 0$ objects,

$$\text{Acc}_c = \frac{1}{M_c} \sum_{m: y_m=c} \mathbb{I}[\hat{y}_m = c], \quad \text{mAcc} = \frac{1}{C} \sum_{c=1}^C \text{Acc}_c. \quad (2.53)$$

Here C is the number of evaluated classes. mAcc gives each class equal weight regardless of its sample count, whereas OA gives more aggregate weight to larger classes. They can therefore change by different amounts on an imbalanced evaluation set [12].

These equations define sample-level aggregation. Averaging batch accuracies without weighting by batch size gives a different quantity when batches differ in size. Similarly, averaging only the class entries present in each batch can differ from pooling all class predictions first. The actual evaluator must be read to determine which aggregation produced a reported result; the implementation used here is stated explicitly in Chapter 4. This distinction prevents a textbook definition from being substituted for an executed metric.

2.5.3 Detection Metrics

Detection evaluates class correctness together with localisation. For predicted and ground-truth boxes B_{pred} and B_{gt} , the three-dimensional intersection over union is

$$\text{IoU}_{3D} = \frac{\text{Vol}(B_{\text{pred}} \cap B_{\text{gt}})}{\text{Vol}(B_{\text{pred}} \cup B_{\text{gt}})}. \quad (2.54)$$

The operators $\cap$ and $\cup$ denote intersection and union, and Vol is volume. For non-degenerate boxes IoU lies between zero and one. Replacing the volumes by the areas of horizontal box footprints gives BEV IoU. A BEV overlap can remain high when vertical localisation is inaccurate, so BEV and full 3D evaluation should not be interchanged [8, 25].

Predictions are ranked by confidence and matched to eligible ground-truth instances according to class, overlap and benchmark rules. A matched prediction is a true positive (TP); an unmatched eligible prediction is a false positive (FP), and a missed eligible object is a false negative (FN). At a given score threshold,

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}. \quad (2.55)$$

The denominators count retained eligible predictions and eligible annotated objects, respectively. Duplicate detections cannot all match the same object as true positives; ignored annotations and empty denominators must be handled according to the benchmark implementation. Sweeping the score threshold produces a precision–recall curve [26].

Interpolated precision at recall ν is the upper envelope

$$p_{\text{interp}}(\nu) = \max_{\tilde{\nu} \geq \nu} p(\tilde{\nu}), \quad (2.56)$$

where $p(\tilde{\nu})$ is the precision achieved at an attained recall $\tilde{\nu}$. The maximum removes local upward fluctuations in precision [26]. Average precision summarises the interpolated precision across the recall positions specified by an evaluation protocol [25, 26]. Different sampling conventions need not yield the same numerical AP. The implemented 40-position convention, class thresholds and difficulty rules are specified in Section 4.10.3.

An individual scene image can illustrate a missed or recovered detection, but it does not determine AP for a complete evaluation set. Likewise, changing confidence ordering can affect AP even if some selected boxes appear similar. Qualitative examples and aggregate precision–recall evaluation therefore play complementary roles.

The geometric and task metrics quantify different outputs. Their relationship is examined through the comparison design in Chapter 3; the datasets and executed evaluation settings are specified in Chapter 4.

Chapter 3

Idea and Concept

Chapter 1 formulated the central question: whether a denser point cloud provides more useful evidence to a downstream perception model. Chapter 2 defined the geometric and task metrics used to examine this question. The present chapter converts those observations into a controlled experimental design. The design uses two data domains and three downstream networks, but applies the same logical separation between the input visible to the upsampling method, the generated point set, the point set supplied to the downstream model, and the final task metric.

3.1 Overall Concept

The metric definitions explain why a good geometric score is not itself a guarantee of better perception. CD averages nearest-neighbour discrepancies; PointNet++ pools learned features within sampled neighbourhoods; PointRCNN generates proposals from foreground points; and a voxel-based CenterPoint aggregates points into spatial cells [5, 8, 9, 11]. These operations do not optimise or measure the same quantity.

Several consequences follow from the definitions. Repeating coordinates can increase stored point count without extending spatial coverage. A small number of misplaced points can have little effect on an average geometric distance while changing a sampled neighbourhood or an occupied voxel. Conversely, an improved surface approximation may leave the predicted class unchanged when the classifier already assigns the correct class the largest score. These are possible mechanisms derived from the metrics and architectures, not universal empirical outcomes.

Reference quality also limits interpretation. A complete mesh supports point-to-surface evaluation, whereas a single sensor scan supplies only observed returns [1, 5]. Distance to that scan measures consistency with its samples; it cannot certify the unobserved surface. Geometry, measurement consistency and semantic task performance must therefore be kept distinct.

For the classification branch, PointNet++ is selected because its set abstraction layers explicitly sample centroids and aggregate local neighbourhoods [11]. Upsampling can change both the selected centroids and the points grouped around them. It can supply additional surface support, but near-duplicates need not extend coverage and displaced points can alter local geometry. This architectural sensitivity motivates the classifier choice; whether it improves or degrades classification is determined by the experiments.

The experimental object is the combination of an upsampling method and its integration protocol. Let $\mathcal{X}_s^{(b,l)}$ be the visible input for sample s , branch $b \in \{M, K\}$ (ModelNet40 or KITTI), and line $l \in \{A, B\}$. For method m , let U_m be its fixed inference operator; the method-native candidate set is

$$\mathcal{C}_{s,m}^{(b,l)} = U_m(\mathcal{X}_s^{(b,l)}). \quad (3.1)$$

The symbol $\mathcal{C}$ is used deliberately: before the common output protocol is applied, the method output is treated as a candidate point set. Method-native output cardinality, patch overlap, and representation-specific reconstruction are therefore not automatically equated with the final point cloud evaluated by the downstream task.

The two task branches address complementary settings. ModelNet40 provides mesh-sampled objects with known categories; KITTI provides partial scene observations containing foreground objects and background [1, 10]. This pairing makes it possible to examine geometric and classification behaviour within the object branch before considering transfer to scene-level detection. Agreement between the branches would support the finding across the tested settings. Disagreement must be interpreted with their different data preparation and downstream training regimes in view; it does not identify the data domain as the sole cause.

3.2 Research Hypotheses

The following hypotheses organise the completed comparisons. They state relationships to examine rather than conclusions assumed before evaluation. Their numerical evaluation is presented in Chapter 5.

H1 - Geometric improvement does not imply task improvement. A reduction in CD, HD, P2F, or a related geometric discrepancy is not expected to guarantee an increase in classification accuracy or 3D detection AP.

The implication

$$\Delta d_{\text{geom}} < 0 \implies \Delta \mu_{\text{task}} > 0 ? \quad (3.2)$$

is therefore treated as an empirical question rather than an assumption. Here d_{geom} denotes a geometric error, μ_{task} a task metric such as OA or AP_{R40} , and Δ the method value minus its comparison baseline.

H2 - Densification and recovery are distinct tasks. Adding points to an already available observation and trying to recover information from an artificially sparsified observation are not equivalent questions. A method may be neutral or harmful in the former case but useful in the latter, or vice versa. The two cases are therefore evaluated in separate comparison lines.

H3 - Point count is not equivalent to independent spatial evidence. Satisfying an exact fourfold cardinality requirement only proves that more coordinates are present. It does not prove that those coordinates occupy new, valid surface regions or improve the representation consumed by the downstream model. Duplicate or near-duplicate points can satisfy the count without increasing spatial coverage.

H4 - The downstream architecture mediates the effect of upsampling. PointNet++, PointRCNN, and CenterPoint process point clouds through different sampling and aggregation mechanisms [8,9,11]. This motivates the hypothesis that a change helping one architecture may not help another. Cross-model consistency is therefore treated as stronger evidence than an isolated gain in a single architecture.

H5 - Integration and input budgets can dominate the observed effect. Patch locality, normalisation, coordinate restoration, point subsampling, and voxelisation may change the effective dose of generated points. If an upsampled point cloud performs poorly, the experiment must distinguish a failure of the generated geometry from a failure induced by the wrapper or the downstream input adapter.

3.3 Evaluation Pipeline

The comparison separates four stages: the information visible to the upsampler, its generated candidates, the validated output, and the representation consumed by the task model. Geometry is measured on the saved output against a declared reference; task performance is measured after the downstream adapter. Linking these stages by sample identity allows a geometric change and a task change to be compared without assuming that every generated point reaches the model.

The two branches address different uses of upsampling. ModelNet40 asks whether a generated representation supports classification learning; each representation therefore has its own classifier. The initial KITTI comparison asks whether an existing detector benefits when upsampling is inserted before inference, so detector weights remain fixed. A separate adaptation comparison asks whether training the detector on PU-GCN inputs reduces the resulting mismatch. It changes the downstream detector while keeping the upsampler fixed. These training regimes must remain separate when the findings are combined.

Candidate generation is separated from evaluation. Reference surfaces, class labels and ground-truth boxes provide assessment targets, not additional observations for the upsampler. Chapter 4 specifies the transformations, point counts, checkpoints and training settings that implement this design.

3.4 Experimental Comparison

The two comparison lines distinguish adding points to an available observation from attempting to recover information after controlled sparsification. They share the same methods within each branch, but their baselines answer different questions. Table 4.1 in Chapter 4 specifies their executed point counts.

3.4.1 Line A - Densification of the Original Observation

Line A asks whether generated points provide useful marginal information when the original observation is already available. Its baseline is the native original input, without an extra resampling operation. The comparison therefore concerns the practical consequence of adding the upsampling stage, rather than a comparison between two artificially equalised versions of the baseline.

A negative Line A result cannot be attributed to deliberately removing observations beforehand. It means that the tested generator, integration and downstream model did not obtain a net task benefit from densifying the available input. It does not imply that every individual object or category deteriorates.

3.4.2 Line B - Recovery after Controlled Sparsification

Line B removes a fixed proportion of observations before upsampling. Every method receives the same prepared sparse input. Its immediate baseline is that sparse representation, so a gain measures recovery relative to the information actually available

to the generator. The original representation remains a separate pre-sparsification reference.

This reduction is a controlled stress condition, not a simulation of a particular lower-channel LiDAR sensor. Restoring the number of points does not restore the missing measurements themselves. The comparison therefore asks how much useful task information a method recovers, rather than assuming that an equal output count establishes recovery.

3.5 Fairness Protocol

Fairness requires the same visible observations, a declared output cardinality and a consistent evaluation within each comparison. Upsampling methods share the target expansion ratio; native baselines retain their original sizes. Geometry comparisons separately control the evaluated and reference counts. This distinguishes equal conditions among generated representations from an artificial enlargement of the baseline.

A common interface does not imply identical internal processing. PU-Net expands learned point features, PU-GCN uses graph features and NodeShuffle, PU-EdgeFormer uses edge-transformer features, PDANS uses diffusion-based generation, and the EAR-style baseline uses geometric resampling [3, 4, 5, 6, 7]. Their implementation-specific preprocessing must therefore be recorded alongside the common comparison rules. A modified inference path supports a conclusion about that integrated pipeline, rather than automatically about an unmodified published method.

The effective input depends on the downstream architecture. PointRCNN samples points, CenterPoint aggregates them into voxels, and PointNet++ groups local neighbourhoods [8, 9, 11]. Let $\mathcal{Y}_{s,m}$ denote the validated output for sample s and upsampler m . The representation consumed by downstream model d is

$$\tilde{\mathcal{Y}}_{s,m}^{(d)} = A_d(\mathcal{Y}_{s,m}), \quad (3.3)$$

where A_d is its input adapter. Equal saved cardinality need not imply equal distinct support after this operation. Adapter settings and diagnostic changes are consequently reported as part of the experimental protocol, rather than inferred from file size.

Evidence is interpreted at its actual scope. Complete dataset comparisons establish the main task outcomes; detector adaptation tests a separate training intervention; targeted diagnostics examine specific integration factors. Subset results do not replace full-validation results, and an adapted input must be compared with the corresponding

adapted control where one exists. Chapter 4 records which controls were completed and which sources of uncertainty remain.

3.6 Cross-Domain Interpretation

Object surfaces and LiDAR scenes differ in visibility, coverage and sampling. Scene occlusion and range-dependent support [1] make the transfer of object-oriented up-samplers an additional experimental question [4, 5, 6, 7]. Patch locality, restoration of metric coordinates and preservation of measured attributes can mediate that transfer. Their implementations belong to Chapter 4; the conceptual requirement is to distinguish these interface effects from the generated geometry itself.

Three comparisons connect the branches. Within each branch, Line A and Line B distinguish densification from recovery. Within ModelNet40, geometric ordering is compared with classification ordering. Across branches, the direction of the task response is compared while retaining the different training regimes and observation types. Overall accuracy and detection AP are not combined into a universal score.

Downstream reception and voxel occupancy provide intermediate measurements that may explain a task response. They do not by themselves identify a unique cause, and selected visual cases do not estimate how frequently an outcome occurs. The design thus separates available information, generated geometry, output assembly and downstream use. The next chapter specifies how those distinctions were implemented in the completed experiments.

Chapter 4

Experimental Setup

This chapter describes the implementation of the design defined in Chapter 3. It reports only configurations that were actually built, run, audited, or used as formal diagnostic controls. Mathematical derivations of the networks are not repeated; instead, the focus is on the information required to reproduce the data preparation, upsampling integration, downstream training or inference, and evaluation.

4.1 System Overview and Final Experiment Matrix

The implementation is organised as an offline pipeline. Input point clouds are prepared first and stored under a fixed manifest. Upsampling methods then generate outputs independently of the downstream evaluation. Method-native outputs are preserved, transformed back to the common coordinate system where required, and passed through the strict point-count validation. Only after those checks are complete are the files converted into the representation required by PointNet++, PointRCNN, or CenterPoint.

Separating these pipeline stages allowed detector-input fixes and metric corrections to be evaluated without rerunning every upsampling network and allowed failures in point generation to be distinguished from failures in detector preprocessing.

The high-performance computing (HPC) branch contains 12 primary classification conditions: the original and sparse baselines and five upsampling methods in each line. Each condition has its own PointNet++ training run. Two further mesh-reference conditions provide controls for directly sampled surface support. Their generation, training and evaluation are specified in Sections 4.3.1, 4.9 and 4.10. An input’s nominal point count is checked against the tensor read by the classifier, so a method directory cannot stand in for evidence that the intended representation was used.

The lab branch first compares four learned upsampling pipelines under the recorded full-validation detector protocols. Later diagnostics alter patch construction, point admission or voxel occupancy while retaining a corresponding baseline. The final PU-

Branch	Line	Native baseline	Upsampling
ModelNet40	A	1,024	$1,024 \rightarrow 4,096$
ModelNet40	B	256	$256 \rightarrow 1,024$
KITTI	A	N_s	$N_s \rightarrow 4N_s$
KITTI	B	$M_s = \lfloor N_s/4 \rfloor$	$M_s \rightarrow 4M_s$

Table 4.1: Executed comparison lines. ModelNet40 uses one separately trained classifier per representation; KITTI separates frozen inference and detector adaptation.

GCN study adds detector adaptation and evaluates 20 input-checkpoint arms on the complete validation split. These are separate blocks of the experiment matrix: a repaired subset pipeline does not replace a historical full-validation input retrospectively, and an adapted checkpoint does not belong in a frozen-weight comparison.

Table 4.1 fixes the point counts used by the two comparison lines.

The sequence of this chapter follows those interfaces. Data preparation and down-sampling define the visible information; method integration and strict assembly define the saved output; format conversion and model loaders define the consumed input. Evaluation, adaptation and diagnostic controls then specify how each result in Chapter 5 was obtained. The underlying network principles remain in Chapter 2.

4.2 Software and Hardware Environment

The project combines current PyTorch implementations, legacy TensorFlow 1.x research code, custom Compute Unified Device Architecture (CUDA) operators, and two independent 3D detector codebases. A single software environment was therefore not imposed across all methods. Instead, each research implementation was run in an environment in which its released checkpoint and required custom operators could be loaded reproducibly, while the interfaces between stages were fixed through explicit point-cloud files, manifests, and validation scripts.

ModelNet40 generation and PointNet++ training were executed as Slurm jobs on the Friedrich-Alexander-Universität Erlangen-Nürnberg (FAU) HPC system. The recorded PU-GCN recovery used RTX 2080 Ti nodes after failures on RTX 3080 nodes. KITTI results in this thesis come exclusively from the lab workspace. Both branches required method-specific PyTorch or legacy TensorFlow environments and custom CUDA operators; no single software or graphics processing unit (GPU) configuration

is assumed to describe every run. The archived source records identify the actual environments and recovery operations.

The lab environment audit of 2 September 2026 records separate installations for the detector stacks: PointRCNN used Python 3.9.25 with PyTorch 2.8.0+cu128, whereas CenterPoint/OpenPCDet used Python 3.10.20 with PyTorch 2.7.1+cu128 and spconv 2.3.6. The legacy PU-Net/PU-GCN installation used Python 3.6.8 and TensorFlow 1.13.1. These are the archived lab environment observations, not a common version specification for the earlier HPC jobs. Lab execution logs identify a Quadro RTX 4000 with 8 GiB GPU memory; HPC jobs used Slurm-allocated resources appropriate to their recorded environments.

Because the goal of the thesis is algorithmic evaluation rather than framework benchmarking, framework version differences are not interpreted as method advantages. Reproducibility is instead maintained by recording the method name, checkpoint, input manifest, random seed where relevant, strict target count, and downstream configuration for each final run.

The completed work also included the compatibility and data-integrity changes needed to execute the experiments. Table 4.2 summarises changes that affect reproducibility or interpretation. These were integration changes to existing methods, not new upsampling architectures. In particular, rebuilding PU-GCN’s TensorFlow/CUDA operators did not alter its graph blocks, NodeShuffle or pretrained weights.

4.3 Data Preparation

Data preparation preserves the difference between a sampled CAD surface and a partial sensor observation [1,10]. The two branches share the Line A/Line B comparison logic, but retain their own coordinate systems, identities and reference data. The following subsections contain the dataset-specific preparation and coordinate conventions needed to interpret the experiment.

4.3.1 ModelNet40

The object-level branch uses surface samples from CAD meshes rather than single-view sensor returns. Such a reference can cover surfaces hidden from a particular viewpoint; its coverage is determined by the mesh and sampling process [10,12].

Component	Executed change and significance
PointRCNN I/O	Checked short reads and point-record alignment; retrieved transient shared-filesystem reads so truncated files could not silently be treated as complete clouds.
PointRCNN sampling	Handled far-point and sparse-input edge cases; recorded safe-sampling fallback conditions. These paths belong to the specified historical configurations.
PointRCNN proposals	Handled an empty near-region proposal bucket in regional/far-only inputs. Such diagnostic configurations retain their own baselines.
OpenPCDet	Made unrelated dataset imports optional, supported the checkpoint-loading environment, and corrected sparse fill sampling.
PU-GCN	Replaced fixed CUDA paths with environment-aware compilation and the actual TensorFlow compile/link flags.
PDANS	Made point-operator tensors follow the model device. Formal sampling used float32; a half-precision trial with non-finite outputs was not used for reported results.
Output generation	Resumed missing objects or frames, then checked shape, finite values, required point counts and completion coverage before downstream evaluation.

Table 4.2: Selected recorded integration changes with an effect on execution or evidence quality. Exact code paths, environment details and recovery records are archived with the updated HPC and lab records.

ModelNet40 contains 12,311 CAD models from 40 object categories, with 9,843 training objects and 2,468 test objects in the standard split [10]. The preprocessing executed for this thesis sampled 1,024 points from each .off mesh by triangle-area-weighted surface sampling. Each sampled object was then centred and scaled to the unit sphere. The complete preprocessing produced all 12,311 object point clouds with no failed samples, and the generated files were checked for point count, shape, finite values, and normalisation range.

$$\boldsymbol{\mu} = \frac{1}{N} \sum_{i=1}^N \mathbf{x}_i, \quad s = \max_i \|\mathbf{x}_i - \boldsymbol{\mu}\|_2, \quad \mathbf{x}_i^{\text{norm}} = \frac{\mathbf{x}_i - \boldsymbol{\mu}}{s}. \quad (4.1)$$

Here N is the sample count, $\boldsymbol{\mu}$ the sample centroid, $s > 0$ its maximum centred radius, and $\mathbf{x}_i^{\text{norm}}$ the normalised coordinate. The Euclidean norm $\|\cdot\|_2$ measures distance from the centroid. Centring and division by s give zero mean and maximum radius one; the inverse is $\mathbf{x}_i = s\mathbf{x}_i^{\text{norm}} + \boldsymbol{\mu}$. This normalisation removes absolute

translation and scale, rather than adding surface information [12]. The same object identity, class index, and split are used in every experimental condition. The final Line A visible input is the 1,024-point representation. The final Line B visible input contains 256 points selected without replacement by the fixed, object-specific random sampling rule of the final preprocessing pipeline. The same prepared sparse files are reused by all five upsampling methods so that method comparisons do not depend on different random subsets.

Two additional reference datasets were generated directly from the original .off meshes for the final equal-cardinality geometric analysis. Area-weighted surface sampling produced independent 256-point and 4,096-point mesh-reference point sets for every object, followed by the same unit-sphere normalisation algorithm applied independently to each newly sampled point set. They do not reuse the centroid and radius of Original 1024. These references are evaluation-side data; they are not visible to the upsampling networks.

The earlier unequal-cardinality geometric comparison was superseded after a repeat-x4 control demonstrated a cardinality bias. The final primary geometric comparison therefore uses equal-cardinality references: 4,096-point Line A outputs are compared with 4,096-point mesh references; the 256-point Line B sparse representation can be compared with the 256-point mesh reference; and 1,024-point restored outputs are compared with the original 1,024-point object representation. Unequal-cardinality CD/HD values may remain in development logs but are not used for the final method ranking.

The initial mesh sampler used Python’s built-in hash of class, split, and shape identifier in its seed. Its output is therefore not guaranteed to be identical across fresh processes without controlling Python hash randomisation. The stored point sets are the reproducible inputs to the completed runs; the later stable-seed downsampler does not retrospectively remove this limitation of rebuilding the original clouds from meshes.

4.3.2 KITTI

The scene-level branch uses LiDAR returns from multiple objects and background surfaces. A rotating sensor measures range along discrete beam directions; visibility and surface reflectance influence which returns are recorded [1].

MV3D evaluates KITTI by dividing the labelled training data into training and validation subsets of approximately equal size [27]. The present KITTI branch uses

a fixed split of 3,712 training frames and 3,769 validation frames. The 3,769-frame validation set is used for the formal full-validation detector comparisons. The 3,712-frame training split is additionally used for the detector-adaptation experiment in Section 4.11. Pilot and diagnostic experiments use explicitly named subsets and are never presented as full-validation results.

Each LiDAR frame is stored as an $N_s \times 4$ float32 array of (x, y, z, ι) , where the first three values are the Velodyne-frame coordinates in metres and ι is reflectance. Upsampling networks that operate only on XYZ receive the coordinate channels, but the spatial coordinates remain in the Velodyne frame at the scene level. If a method requires local centring or scaling, the transformation is applied within the local patch and inverted before the generated points are merged back into the frame.

To make the scan geometry explicit, let $\rho > 0$ be range, α azimuth around the vertical axis, and β elevation above the horizontal plane. The spherical-to-Cartesian conversion is

$$\begin{aligned} x &= \rho \cos \beta \cos \alpha, \\ y &= \rho \cos \beta \sin \alpha, \\ z &= \rho \sin \beta. \end{aligned} \tag{4.2}$$

The coordinate convention in this equation takes z as vertical. It parametrises the measured beam directions. For two rays separated by a small angle $\Delta\alpha$ at the same range and elevation, the chord length is

$$d_\alpha = 2\rho \cos \beta \sin(\Delta\alpha/2) \approx \rho \cos \beta \Delta\alpha. \tag{4.3}$$

The approximation uses $\sin u \approx u$ for small u , with angles in radians. It explains why constant angular spacing produces increasing physical separation with range. It does not imply a universal return-count law: surface inclination, missing returns, nonuniform vertical beam angles and occlusion also matter in a real scan [1].

Coordinate transformations require equal care. With a rotation $\mathbf{R} \in \mathbb{R}^{3 \times 3}$ and translation $\mathbf{t} \in \mathbb{R}^3$, the same physical point expressed in a second coordinate frame is

$$\mathbf{p}' = \mathbf{R}\mathbf{p} + \mathbf{t}, \quad \mathbf{R}^\top \mathbf{R} = \mathbf{I}, \quad \det(\mathbf{R}) = 1. \tag{4.4}$$

The identity matrix $\mathbf{I}$ and determinant condition specify a proper rotation. Sensor calibration provides these transforms when measurements and annotations use different frames [1]. Normalising a patch for a neural network is a separate operation; its

translation and scale must also be inverted before combining it with metric scene data.

Calibration files and ground-truth labels are not used during upsampling candidate generation or strict point selection. They are read only by the detector and evaluation stages. The deleted points from Line B are likewise unavailable to the upsampling method; they can be used only in later diagnostic analyses as previously observed sensor locations.

4.4 Downsampling Procedure

The Line B downsampling procedures are designed as controlled information removal rather than as physical simulation of a lower-resolution sensor.

For ModelNet40, the final sparse representation contains 256 points selected without replacement from the stored 1,024-point object. The archived HPC implementation uses NumPy random choice with a stable per-object seed derived from global seed 42, the downsampling operation, split, class name, and shape identifier. The seed helper hashes this token with Message-Digest Algorithm 5 (MD5); the operation does not retain every fourth row and does not use farthest-point sampling. The resulting files were generated once, audited, and reused across all methods. The purpose is to create a common sparse object representation without changing the object’s class, pose, or normalisation.

For KITTI, each original frame with N_s points is reduced to $M_s = \lfloor N_s/4 \rfloor$ points using a deterministic per-frame random selection without replacement. A fixed seed rule ensures that the same frame produces the same sparse point indices for every upsampling method. XYZ coordinates and reflectance are retained together for selected points, so the Line B sparse baseline remains a valid KITTI XYZI point cloud.

This procedure is a controlled decimation rather than a physical low-beam LiDAR simulation. A physical change in LiDAR channel count would alter the angular scan pattern rather than remove points independently. The chosen reduction is instead a controlled stress test that provides identical visible information to every method.

4.5 Upsampling Method Integration

All learning-based upsampling networks are run in inference mode using the pretrained model available for the integrated implementation. No upsampling network is fine-tuned on the downstream validation data. Because the original repositories differ in

language version, framework, custom operators, and input representation, the integration standardises the external protocol rather than rewriting each method into an identical internal architecture.

4.5.1 Shared Input and Coordinate Interfaces

ModelNet40 objects already satisfy the object-level input assumption of the upsampling methods and therefore do not require scene decomposition. Each method receives the fixed Line A 1,024-point object or the fixed Line B 256-point object and generates the target point count for the corresponding line. Final accessible outputs were verified for all 12,311 objects in both lines for EAR, PDANS, PU-Net, PU-GCN, and PU-EdgeFormer. Line A files contain 4,096 points and Line B files contain 1,024 points.

The method outputs are not modified merely to make them contain a copy of every input point. This is important because the object-level experiment evaluates the representation produced by the upsampling method itself. Output shapes and finite values are audited before the point clouds are passed to PointNet++.

The learned methods were developed for object or local-patch inputs [4,5,6,7]. Their integrated inference paths in this project do not directly consume a complete KITTI scene as one tensor. The scene is therefore decomposed into spatially local fixed-size patches. Development experiments showed that grouping points merely by their row order or by overly coarse bins can combine physically unrelated surfaces. The final integration therefore uses spatially local neighbourhood construction and audits unique support and coverage.

For a patch $\mathcal{Q}_j = \{\mathbf{q}_i\}$, the local normalisation used by methods requiring unit-scale input is

$$\boldsymbol{\mu}_j = \frac{1}{|\mathcal{Q}_j|} \sum_{\mathbf{q} \in \mathcal{Q}_j} \mathbf{q}, \quad s_j = \max_{\mathbf{q} \in \mathcal{Q}_j} \|\mathbf{q} - \boldsymbol{\mu}_j\|_2. \quad (4.5)$$

$$\hat{\mathbf{q}} = \frac{\mathbf{q} - \boldsymbol{\mu}_j}{s_j}, \quad \mathbf{q}_{\text{back}} = s_j \hat{\mathbf{q}}_{\text{out}} + \boldsymbol{\mu}_j. \quad (4.6)$$

Here $\boldsymbol{\mu}_j$ and s_j are the centroid and maximum radius of patch $\mathcal{Q}_j$; $\hat{\mathbf{q}}$ is a normalised input, $\hat{\mathbf{q}}_{\text{out}}$ a network prediction, and $\mathbf{q}_{\text{back}}$ its restored metric coordinate. Each patch stores its own centre and scale so that the inverse transformation cannot accidentally use parameters from another patch. The reconstructed patch outputs are merged into a scene-level candidate pool before strict cardinality normalisation.

The final PU-GCN full-validation patch pipeline provides the most fully audited instance of this process. It uses 2,048 input points per patch and produces 8,192 patch-output points. The final `fps_ball_cover_knn_v3` construction uses a primary radius of 2 m in Line A and 4 m in Line B, a cover radius of 6 m, and a unique-support threshold of 2,048 points. Local patches are centred and unit-sphere normalised before PU-GCN inference and transformed back afterwards. The released PU1K model-100 checkpoint is used; the PU-GCN upsampling network itself is not retrained on KITTI.

Patch centres are selected from observations with sufficient local support. A seeded first centre is followed by XYZ farthest-point selection among eligible centres. Points around a primary centre are ordered by distance and original row index before the first 2,048 are taken. Supplemental patches address still-uncovered eligible observations; the metadata retains their source indices. Coverage is guaranteed for the cover-eligible set, not for isolated observations with fewer than 32 neighbours inside 6 m.

The final study retained a fixed PU1K `model-100` upsampler; “retraining” refers to PointRCNN and CenterPoint adaptation. Primary patch centres required 2,048 supporting points within 2 m for Line A or 4 m for Line B. Supplemental coverage considered uncovered points with at least 32 neighbours inside 6 m, then used unique nearest neighbours to reach 2,048 rows. The 6 m cover radius is therefore not a hard bound on every supplemental patch.

Figure 4.1 locates patch normalisation and reconstruction.

4.5.2 EAR

The project EAR-style geometric baseline, labelled EAR in the result tables, is fully executed on HPC in both final ModelNet40 lines. Edge-aware resampling provides its methodological background [3]; the archived project wrapper calls a migrated EAR-style implementation and does not establish an exact reproduction of the published algorithm. For KITTI, a strict fourfold full-scene wrapper was implemented and verified on five frames in Line A and five frames in Line B. The central processing unit (CPU) implementation contains global neighbourhood and principal component analysis (PCA) operations and Python-level loops; measured runtime was approximately 26-29 minutes per Line A frame and 9-10 minutes per Line B frame. Extrapolation to the full validation set made the strict scene-level run impractical, so EAR is not reported as a completed strict full-validation KITTI method.

For the HPC object branch, the fixed wrapper uses 20 neighbours, edge sensitivity 4.0, upsampling threshold 0.93, zero positional-noise parameter and at most five

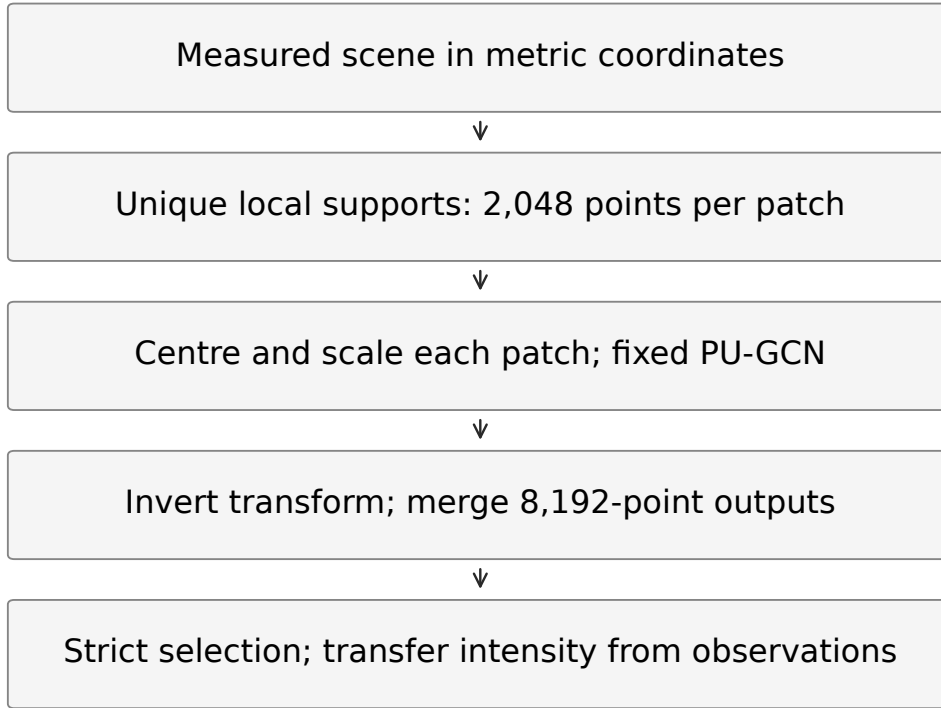


Figure 4.1: The final lab PU-GCN patch and strict-output pipeline. Centres and scales are stored per patch; metric coordinates are restored before merging.

iterations. It adds a zero-valued intensity channel only to satisfy the migrated implementation’s interface and retains XYZ for ModelNet40 classification. These settings describe the project implementation; the EAR paper supplies the methodological background [3].

EAR therefore has no entry in the strict full-validation KITTI tables. Earlier detector measurements used a different protocol and are retained only in the archived work record.

4.5.3 PU-Net

PU-Net uses hierarchical point features and learned feature expansion [4]. Its integration in this project required substantial compatibility work before stable inference. The original TensorFlow-era code was adapted for Python 3, and custom sampling, grouping, and interpolation operators were rebuilt for the available environment. The KITTI integration also required explicit verification of unit-sphere normalisation. A dedicated 2×2 diagnostic experiment later confirmed that incorrect normalisation

and non-local patch construction were both genuine causes of severe detector degradation. The final corrected pipeline therefore stores patch centres and scales, performs the inverse transformation explicitly, and audits the reconstructed coordinates.

The HPC wrapper invokes the restored PU-Net generator in test mode with upsampling ratio four and the actual line-specific input count. The input count is 1,024 in Line A and 256 in Line B; the wrapper does not substitute a fixed-size baseline cloud for a method output. Checkpoint loading, candidate generation and strict-file writing are checked separately, because successful network inference alone does not establish that the evaluated output file was written correctly.

On ModelNet40, final Line A and Line B outputs were completed for all 12,311 objects. The initial full-generation attempt encountered timeouts and one TensorFlow subprocess failure, after which a missing-only resume strategy completed and audited the full output set. On KITTI, the strict full-validation method is included in the four-method unified frozen-detector comparison under the earlier integration path, whose PU-Net normalisation error is explicitly flagged. Corrected subset experiments are reported separately.

4.5.4 PU-GCN

PU-GCN [5] uses graph-based local features and NodeShuffle feature expansion. Its TensorFlow custom graph operations were rebuilt for the available environment. The ModelNet40 final outputs were completed for both lines; a stalled Line B generation segment was recovered through an explicit missing-sample resume process, after which 12,311/12,311 outputs were available.

The HPC invocation specifies the PU-GCN model, NodeShuffle upsampler, ratio four, graph neighbourhood size 20 and inference seed 42. It sets the input point count from the chosen line. The lab patch protocol instead runs the fixed PU1K checkpoint on 2,048 $\rightarrow$ 8,192 point patches. This change in inference input size and the CUDA-operator recovery are distinct from optimising the network weights; neither is a PU-GCN retraining experiment.

For KITTI, PU-GCN is evaluated in the unified strict frozen-detector comparison and is also the method used for the later observed-first and detector-adaptation study. Its final full-validation patch configuration is given in Section 4.5.1. Candidate patch outputs are merged and then selected to the exact scene-level target using a fixed seed without replacement. If the merged candidate pool is smaller than the exact target, the sample fails rather than being completed with duplicated points.

4.5.5 PU-EdgeFormer

The published PU-EdgeFormer model uses an edge transformer [6]. The two project branches have different execution provenance. On HPC, the final ModelNet40 record verifies native EdgeTransformer inference, its epoch-100 checkpoint, complete outputs in both lines, and independent PointNet++ training. On lab, the original environment failed and the completed KITTI compatibility path reused PU-GCN-compatible operators and checkpoint inference components. The lab results are therefore labelled as compatibility-pipeline evidence, not as an unmodified reproduction of the published PU-EdgeFormer model.

The verified HPC path uses direct $1,024 \rightarrow 4,096$ inference for Line A and $256 \rightarrow 1,024$ for Line B, with inference seed 20260702. Its audit checks the loaded model and checkpoint, complete output coverage, finite float32 coordinates and the required counts. A shortage raises an error rather than invoking interpolation or duplicate padding. This preserves the distinction between a native EdgeTransformer result on HPC and the lab compatibility experiment.

4.5.6 PDANS

PDANS combines conditional diffusion and adaptive noise suppression [7]. The PyTorch implementation used here required compatibility changes in its point-operator utilities, including replacing hard-coded CUDA tensor creation with device-aware handling. After the environment and point operators were validated, final ModelNet40 Line A and Line B outputs were generated and trained with PointNet++.

The HPC invocation uses the released PU1K PDANS checkpoint, upsampling ratio four, 30 denoising diffusion implicit model (DDIM) sampling steps and a sampler blending coefficient $\gamma = 0.5$. In the released sampler, this coefficient scales the sum of the denoising update and an interpolated input [7]. Each object is processed independently and checked for finite output. The archived formal generation uses float32; the non-finite half-precision trial is an implementation failure record, not a separate result condition. Diffusion and noise suppression provide the method’s basis [7], while these numerical settings identify the executed inference path.

For KITTI, PDANS is part of the four-method strict full-validation comparison. It also serves as the method in several detector-aware diagnostic experiments because its generated candidates were used to study patch locality, generated-point dose, voxel activation, and proposal-aware selection. These experiments are described collectively in Section 4.12 rather than being presented as separate upsampling methods.

4.6 Ratio Normalisation and Point-Count Verification

The primary protocol uses fourfold upsampling. ModelNet40 Line A maps 1,024 to 4,096 points, and Line B maps 256 to 1,024 points. All five method datasets were audited for these exact shapes. For KITTI sample s , let N_s be the original point count and $M_s = \lfloor N_s/4 \rfloor$ the sparse count. The targets are

$$N_{\text{target},s}^A = 4N_s, \quad N_{\text{target},s}^B = 4M_s. \quad (4.7)$$

The restored Line B count can differ from N_s by at most three points. Both native baselines retain their original counts.

For visible input $\mathcal{X}_s$ with $n_s = |\mathcal{X}_s|$ rows and method-native candidate pool $\mathcal{C}_{s,m}$, the strict assembly is

$$\mathcal{Y}_{s,m} = S_4(\mathcal{C}_{s,m}), \quad |\mathcal{Y}_{s,m}| = 4n_s. \quad (4.8)$$

For the unified scene pipeline, S_4 selects the required number of candidate rows with a fixed seed and without replacement after patch merging. It does not consult ground truth, surface references or task predictions. A candidate shortage is a protocol failure; duplicate padding is not used to reach the target.

Observed-first KITTI input is a separate condition:

$$\mathcal{Y}_{s,m}^{\text{obs}} = \mathcal{X}_s \uplus S(\mathcal{Y}_{s,m}, 3n_s), \quad |\mathcal{Y}_{s,m}^{\text{obs}}| = 4n_s. \quad (4.9)$$

Here $\uplus$ concatenates rows and $S(\cdot, 3n_s)$ selects $3n_s$ predicted rows without replacement. This preserves every observed input row while matching the direct condition’s total cardinality. A direct strict output does not guarantee such preservation.

Every saved output is reloaded to verify its row count, dimensionality and finite coordinates. Frame-level coverage is checked before task evaluation, so a reported AP does not silently omit generation failures.

Line B sampled $M_s = \lfloor N_s/4 \rfloor$ indices without replacement using seed 20260702 + frame_id and sorted the indices to retain relative row order. Strict output sampling used the same seed base after merging patch predictions. Observed-first construction used a separately derived Secure Hash Algorithm 256-bit (SHA-256) seed with base 20260718 and drew $3N_s$ or $3M_s$ prediction rows without replacement. No confidence, curvature, surface reference, or ground-truth box guided that selection. Distinct row indices do not guarantee distinct coordinates.

4.7 KITTI Format Conversion and Intensity Transfer

The upsampling methods generate XYZ coordinates, whereas KITTI point-cloud files store four float32 values per point. Every generated XYZ point must therefore be assigned an intensity before the file can be read by the detector pipelines.

The implemented rule transfers the intensity of the nearest observed point. For a generated coordinate $\mathbf{g}$, the assigned reflectance is

$$\iota(\mathbf{g}) = \iota\left(\arg \min_{\mathbf{x}_i \in \mathcal{X}_{\text{obs}}} \|\mathbf{g} - \mathbf{x}_i\|_2\right). \quad (4.10)$$

Here $\mathcal{X}_{\text{obs}}$ is the visible measured set, $\mathbf{g}$ a generated coordinate, and ι reflectance. The rule is deterministic, local, and independent of labels or detector predictions. Original observed points retain their measured intensity values in conditions where they are explicitly preserved. The resulting XYZI arrays are stored as float32 and reloaded to verify that the file size is compatible with four channels, the shape is $Q \times 4$, where Q is the output row count, and all values are finite.

4.8 Detector Integration

The converted scene is now passed to the detector input adapter. PointRCNN selects a point tensor, whereas the implemented CenterPoint pipeline forms bounded voxel features [8,9]. Their inference constraints are described separately because equal saved point counts need not produce equal changes in their effective inputs.

Figure 4.2 compares the two detector input adapters.

4.8.1 PointRCNN Integration

The main full-validation runs pass complete frames through the detector’s filtering and fixed-budget sampling. The region-split procedure is a separate diagnostic control described in Section 4.12.2; it does not replace full-frame inference in the final comparison.

PointRCNN [8] is evaluated with a fixed pretrained checkpoint and fixed inference configuration in the primary KITTI comparison. The standard input preprocessing uses a point budget of 16,384 points per frame. When a strict upsampled frame contains more than this number of eligible points, the detector’s sampling stage determines which points are actually processed by the network. Thus, the $4\times$ file-level cardinality is intentionally reported separately from the effective PointRCNN input.

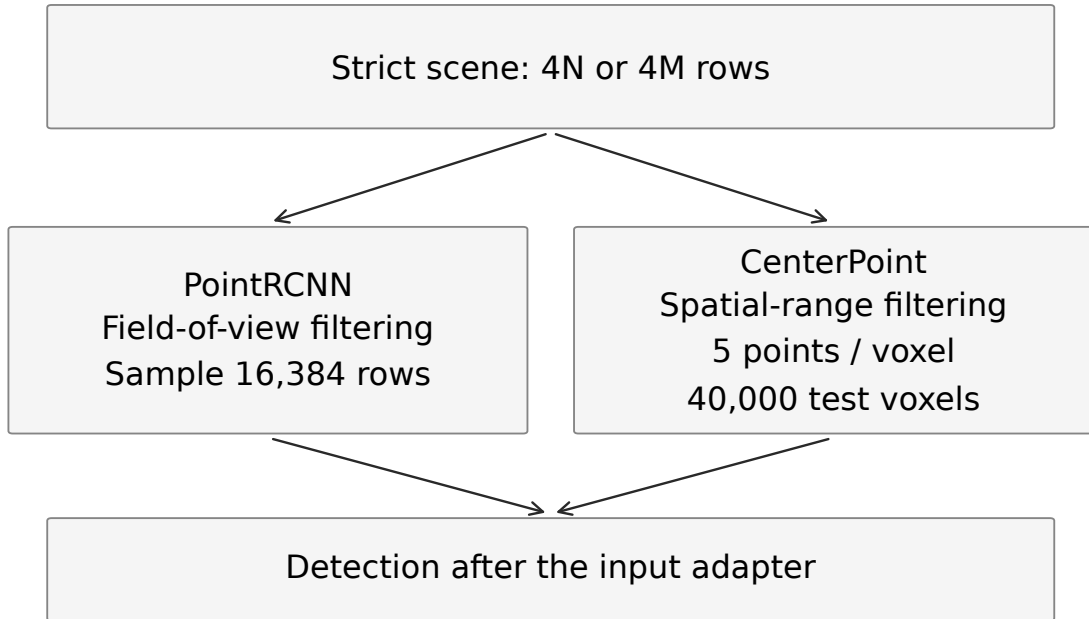


Figure 4.2: Alternative PointRCNN and CenterPoint input constraints after scene-level filtering. File cardinality does not equal model capacity.

The actual reader first applies camera-field-of-view and configured range filtering. For eligible clouds above the point budget, its usual sampling path retains the far-point pool, defined by depth of at least 40 m, and selects the remaining required rows from the near pool. For a sparse eligible cloud, repeated row selection fills the network tensor. Such loader repetition is not new spatial evidence and is separate from the prohibition on padding strict upsampling outputs. The recorded primary PointRCNN setup does not use intensity as a network feature, although it reads valid XYZI files.

During the strict full-scene experiments, a sampler-safe fallback was added to the dataset loader for frames in which the far-range pool alone already reached or exceeded the required region proposal network (RPN) point count. In that case, 16,384 points are sampled without replacement from the valid frame rather than attempting to construct a negative number of near-range samples. This change fixed a runtime failure and was applied consistently to affected variants; it is not interpreted as an accuracy enhancement.

The primary comparison keeps the checkpoint, score thresholds, non-maximum suppression settings, class mapping, and evaluation code fixed across point-cloud variants. A separate no-resample diagnostic was executed in an isolated PointRCNN copy to de-

termine whether simply removing the 16,384-point sampling cap improves upsampled inputs. Because that experiment changes the detector’s input distribution, it is treated only as a diagnostic control and is specified further in Section 4.12.

For the final PointRCNN comparison, the dataset reader reset the sampling seed for every frame to $(20260908 + \text{frame_id}) \bmod 2^{32}$. The same input consequently produced the same sampled tensor under frozen and adapted weights. Actual tensor shapes and SHA-256 hashes were retained.

4.8.2 CenterPoint Integration

CenterPoint [9] is evaluated through OpenPCDet [28] with fixed pretrained weights in the primary KITTI comparison. The implemented configuration uses voxel size (0.05, 0.05, 0.10) m. At test time, at most five points are represented per voxel and at most 40,000 voxels are retained for a frame. These limits make it necessary to distinguish raw point count from occupied-voxel count.

The full strict point cloud is retained as an auditable intermediate file. CenterPoint then applies its standard point-range filtering and voxelisation. Diagnostic scripts record the number of occupied voxels and whether the 40,000-voxel limit is reached. This allows a performance loss to be interpreted in relation to the number of generated points that activate new spatial cells rather than only the total number of new coordinates.

The same voxel size, point-cloud range, per-voxel limit, maximum voxel count, checkpoint, score threshold, and post-processing configuration are used for all comparable variants. Absolute AP values from PointRCNN and CenterPoint are not used to rank the detector architectures against each other; the relevant quantity is the change relative to each detector’s own baseline.

CenterPoint evaluation used no test-time shuffling and retained the configured range $[0, 70.4] \times [-40, 40] \times [-3, 1]$ m after field-of-view filtering. The implemented chain is MeanVFE, VoxelResBackBone8x, HeightCompression, BaseBEVBackbone and CenterHead. Thus input order can affect retained points when a voxel exceeds its five-point limit. The historical ordering-only diagnostic and the later observed-first PUGCN construction are recorded separately: the former changes the order of the same point multiset, whereas the latter constructs an observed-plus-predicted input.

4.9 Classifier Integration

The object branch uses the same idea of an explicit input interface, but its task is shape classification rather than scene detection. Each saved XYZ array is paired with its class through the fixed class map. The final loader reads files from the selected method directory; a corrected metadata link exposes only the class maps and cannot redirect a method branch to the sparse-baseline manifest.

The final ModelNet40 classifier uses the PointNet++ single-scale grouping architecture [11], implemented here as `pointnet2_cls_ssg`. Every final branch is trained independently from random initialisation for 200 epochs using Adam [29], an initial learning rate of 0.001, seed 42, and XYZ coordinates without normals. The class mapping and official ModelNet40 train/test identities are fixed across all runs.

The final primary classification matrix contains 12 branches: the 1,024-point Line A original baseline, five 4,096-point Line A upsampled datasets, the 256-point Line B sparse baseline, and five 1,024-point Line B restored datasets. In addition, two mesh-reference controls - 256 points and 4,096 points sampled independently from the CAD meshes - were trained under the same recipe.

The final data loader exposed each condition at its native point count. A later audit confirmed that the five 4,096-point Line A models genuinely received batches of 4,096 points and were independently trained rather than reusing the 1,024-point checkpoint.

For each run, the training history records epoch-level accuracy and the resulting checkpoint. Both the best observed overall accuracy and the final-epoch overall accuracy are retained. Each result table identifies its checkpoint statistic. Because only one final seed (42) was trained for each main branch, the reported values are individual-run results without a cross-seed standard deviation. A separate determinism audit showed identical parameters for repeated runs under the same seed in selected conditions, but this does not substitute for a cross-seed statistical analysis.

The HPC classifier was PointNet++ SSG with XYZ-only input, batch size 24, four loader workers, Adam, learning rate 0.001, weight decay 0.0001, StepLR period 20 and factor 0.7, 200 epochs, and seed 42. Its first abstraction layer used 512 centroids, radius 0.2 and 32 neighbours; the second used 128 centroids, radius 0.4 and 64 neighbours; the last grouped globally. Each branch retained its native point count with loader resampling disabled. Best OA selected the checkpoint on the test set, with ties retaining the latest epoch. Final OA uses epoch 200; neither statistic is a multi-seed estimate.

The loader centres and unit-sphere normalises each object when reading it, with point-count resampling disabled. Training uses random point dropout, scaling and translation, shuffles samples and drops an incomplete final training batch. Evaluation retains all test objects. The abstraction-layer feature widths are $[64, 64, 128]$ followed by $[128, 128, 256]$; the classification head maps $1024 \rightarrow 512 \rightarrow 256 \rightarrow 40$, with dropout probability 0.4 in both hidden stages and negative log-likelihood loss. These are the recorded SSG implementation settings, whose hierarchical grouping follows PointNet++ [11].

The wider HPC inventory contains 22 completed classifier jobs: the 12 canonical branches, two mesh-reference controls and eight earlier jobs. Three earlier/final pairs have identical values in all 79 saved weight tensors at seed 42. They demonstrate repeatability for those stored inputs, not eight extra independent confirmations of the final comparison. The earlier point-count protocols and their results are summarised with the controls in Section 5.1.4.

4.10 Geometric and Task Evaluation Protocol

Evaluation follows the distinction established in Section 3.2: geometric proximity, classification accuracy and detection AP measure different properties. Geometry uses the stored point sets and the declared reference; task evaluation uses the predictions of the specified input–model condition. The following protocols state point counts, evaluation units and aggregation rules so that a numerical change can be traced to the comparison that produced it.

4.10.1 ModelNet40 geometric evaluation

The final geometric analysis corrects the reference-cardinality bias identified during development. CD and HD are computed on the 2,468-object test split under equal-cardinality comparisons. For Line A, each 4,096-point upsampled output is compared with an independently sampled 4,096-point mesh reference. For Line B, the final 1,024-point restored output is compared with the original 1,024-point representation; the 256-point sparse baseline can be compared with the independent 256-point mesh reference as a matching-cardinality control.

Table 4.3 states the actual point-count controls. The HPC evaluator resolves output and reference by the same split, class and shape identifier, checks both array shapes against $(n, 3)$, and rejects a mismatched point count before computing distances. It

Evaluated condition	Points	Reference	Points
Line A method output	4,096	Mesh-ref 4,096	4,096
Line B method output	1,024	Original 1,024	1,024
Line B sparse control	256	Mesh-ref 256	256

Table 4.3: Equal-cardinality CD/HD comparisons on 2,468 test objects per condition.

uses the stored arrays without evaluation-time resampling or alignment. The archived test audit contains 14 conditions with 2,468 objects each, 34,552 rows in total, and zero failed rows. The sparse-control comparison uses a different reference and is not pooled with the restored-output ranking.

Equal cardinality does not remove every source of geometric discrepancy. Original 1024 and the mesh-reference clouds were independently sampled, centred and scaled using their own sample centroids and maximum radii. Although each comparison uses the same saved reference for all methods, the mesh-reference comparisons can include sampling-dependent translation and scale differences. Their values describe the stored normalised point sets; they are not absolute surface errors under a shared mesh-derived transform. No common-transform reevaluation was performed in this thesis.

The final CD implementation uses the non-squared Euclidean nearest-neighbour distance defined in Equation (2.48). HD follows Equation (2.49). NUC is computed with the implementation used by the project and reported as a distribution/uniformity diagnostic. Point-to-surface distance is retained as a secondary geometric analysis using the CAD surface, but it is not substituted for the equal-N primary comparison.

A repeat-x4 cardinality-bias probe was also executed. Repeating each original point four times can yield a deceptively favourable unequal-cardinality nearest-neighbour comparison despite adding no new spatial support. This probe is the reason unequal-cardinality CD/HD are not used for the final geometric ranking.

PU-Net introduced normalised uniformity coefficient (NUC) evaluation [4]. The project uses a neighbourhood-count variant, whose values are not claimed to reproduce the original NUC protocol. The HPC code used 128 query centres and radii 0.02, 0.05, and 0.10, averaging the coefficient of variation of neighbourhood counts. Query centres were not perfectly shared by object across methods; the seed also includes the method name and Python’s built-in hash. For a zero mean neighbour count, the implementation returns zero rather than an undefined coefficient. NUC is therefore a project-specific diagnostic, not an unconditional measure of surface quality. P2F independently normalised cloud and mesh and is therefore a secondary normalisation-

conditioned audit. Four reused mesh-reference training shapes limit the classifier controls but do not affect test geometry.

The secondary uniformity tables report mean absolute deviation from the reference, not a signed difference of aggregate uniformity scores:

$$D_{\text{NUC}} = \frac{1}{S} \sum_{i=1}^S |\text{NUC}(Y_i) - \text{NUC}(R_i)|. \quad (4.11)$$

Here S is the number of evaluated objects, Y_i the method output and R_i its designated reference. The absolute value is taken per object before averaging. The independently chosen query centres remain a limitation of this secondary comparison.

4.10.2 ModelNet40 classification evaluation

All 2,468 test objects are evaluated, but the saved metrics are averages of batch-level ratios rather than the population formula in Equation (2.52). The archived evaluator accumulates

$$\text{OA}_{\text{rep}} = \frac{1}{B} \sum_{b=1}^B \frac{C_b}{n_b}, \quad a_c = \frac{1}{|\mathcal{B}_c|} \sum_{b \in \mathcal{B}_c} \frac{C_{bc}}{n_{bc}}, \quad \text{mAcc}_{\text{rep}} = \frac{1}{K} \sum_{c=1}^K a_c. \quad (4.12)$$

Here B is the number of test batches, n_b and C_b are their object and correct-prediction counts, $\mathcal{B}_c$ contains batches with class c , and n_{bc} and C_{bc} are the corresponding class-specific counts. All $K = 40$ classes are present. With batch size 24 and no dropped test samples there are 102 full batches and a final batch of 20 objects. Each batch has equal weight in reported OA, including the smaller last batch; the class scores also average the ratios of batches containing that class. Multiplication by 100 gives the percentages in Chapter 5. The table labels OA and mAcc retain these recorded implementation statistics throughout; they have not been converted into object-count accuracies. The main Line A comparison is each 4,096-point upsampled condition against the 1,024-point original baseline. The main Line B comparison is each 1,024-point restored condition against the 256-point sparse baseline, with the 1,024-point original baseline retained as a reference level.

The project stores per-class accuracy summaries, but complete per-object predictions were not preserved for every final run. A McNemar comparison of correlated correctness proportions [30] would require paired object-level outcomes; aggregate class scores are insufficient. That test was not performed. Complete probability outputs were also unavailable, so calibration and probability-level analyses were not performed.

4.10.3 KITTI detection evaluation

The formal KITTI evaluation uses the fixed 3,769-frame validation split. PointRCNN is evaluated primarily for Car, consistent with the available reference setup. CenterPoint is evaluated for Car, Pedestrian, and Cyclist. Easy, Moderate, and Hard difficulty levels are reported under the class-specific KITTI rules detailed below [31]. BBox, BEV, and 3D AP are kept as separate quantities.

The final PU-GCN matrix and the earlier CenterPoint multi-method matrix use AP_{R40} . The archived PointRCNN E2/E3 runner independently confirms C++ R40 evaluation: it averages the last 40 entries of each 41-entry precision row. These earlier full-validation experiments are reported under their own input protocols. In contrast, the 64-frame adaptation screen explicitly used the repository’s legacy Python evaluator. Its AP values are retained under that label and are not pooled with R40 results.

For the central claims, 3D AP_{R40} at Moderate difficulty is emphasised, while Easy/Hard and BEV/BBox results are retained to provide context. Every formal method entry must have complete 3,769/3,769 prediction coverage before it enters the main full-validation table.

KITTI uses class-specific overlap and difficulty rules [31]. Car requires 3D IoU 0.70; Pedestrian and Cyclist require 0.50. Easy, Moderate, and Hard require image-box heights of at least 40, 25, and 25 pixels, maximum truncations of 0.15, 0.30, and 0.50, and maximum occlusion levels of 0, 1, and 2, respectively. Let TP, FP, and FN be true positives, false positives, and false negatives under the evaluator’s matching and ignore rules. Using the precision, recall, and interpolated-envelope definitions described for detection evaluation [26], precision is $P = TP/(TP + FP)$ and recall is $R = TP/(TP + FN)$. The interpolated precision envelope is $P_{\text{int}}(r) = \max_{\tilde{r} \geq r} P(\tilde{r})$. The final implementation uses the 40 positive recall-grid entries proposed for KITTI AP_{R40} [25], giving

$$AP_{R40} = \frac{1}{40} \sum_{j=1}^{40} P_{\text{int}}(j/40). \quad (4.13)$$

Here j indexes recall-grid positions and $\tilde{r}$ an achieved recall. Percent AP multiplies this fraction by 100. Historical AP_{R11} values use another sampling rule and are not interchanged with AP_{R40} .

4.10.4 Geometry-task relationship

The ModelNet40 branch provides the cleanest direct comparison because both geometry and classification are available for the same object-level outputs. Method rankings

under equal-N CD/HD/NUC are compared with their OA changes. The archived category-level analysis uses Spearman rank correlation [32]. The reported geometry intervals use bootstrap resampling [33]: objects are the resampling units for aggregate geometric errors, and categories for geometry–score associations. These analyses are descriptive. They do not identify a causal effect of a geometric metric on classification.

For KITTI, complete dense ground-truth surfaces are not available. Geometry-task interpretation therefore uses measurable scan- and detector-side diagnostics, such as duplicate behaviour, occupied voxels, generated-only voxel activation, and the relationship between point-budget changes and AP. Distances to deleted Line B observations can be interpreted as recovery of previously observed sensor locations, but they are not labelled as distance to the true complete scene surface.

The paired category summaries use the same category identities when relating geometric changes to classification scores. Their observations are category-level aggregates; they do not supply the missing object-level classification predictions. Bootstrap intervals for geometry and rank associations consequently quantify variation in those saved summaries, not variation across independently trained classifiers. The training curves are examined separately as within-run histories.

4.11 Detector Retraining with PU-GCN Inputs

The frozen-detector experiments measured substantial degradation on upsampled inputs, motivating an input-distribution mismatch hypothesis. A full-train adaptation experiment was therefore executed for PU-GCN to test whether detector retraining can recover part of the lost performance.

This KITTI study changes the detector weights while keeping the PU-GCN PU1K model-100 checkpoint fixed. Upsampling is performed offline; the detection loss does not update the upsampling network. Retraining here means continuing detector optimisation from an existing checkpoint, rather than training PU-GCN or a ModelNet40 classifier.

PointRCNN has separately adapted checkpoints for the direct strict PU-GCN inputs and the observed-first inputs. For CenterPoint, the PU-GCN-adapted checkpoints are trained on observed-first inputs; direct strict inputs are evaluated with official weights only. Both detectors also have a separately adapted sparse baseline for Line B. No adapted original-input baseline was trained for Line A.

Detector	Line	Baseline	Direct	Observed-first
PointRCNN	A	Original: F	F/A	F/A
PointRCNN	B	Sparse: F/A	F/A	F/A
CenterPoint	A	Original: F	F	F/A
CenterPoint	B	Sparse: F/A	F	F/A

Table 4.4: Initial three-epoch detector-adaptation matrix. F: frozen detector weights; A: adapted detector weights.

The training split contains 3,712 KITTI frames, and each final evaluation arm covers all 3,769 validation frames. The complete matrix contains 11 PointRCNN arms and 9 CenterPoint arms, including the corresponding official-weight comparisons.

Table 4.4 identifies the available input–weight combinations. Here F denotes the official frozen checkpoint and A a separately adapted detector checkpoint. An F/A entry therefore represents two evaluation arms, not one model evaluated twice with untracked settings.

Figure 4.3 identifies the detector weights updated during adaptation.

The training and evaluation manifest for the initial matrix is archived with its results. In that matrix, each adapted PointRCNN arm is trained with a three-epoch RPN stage followed by a three-epoch offline region-based convolutional neural network (RCNN) stage. Batch size is 1, workers are set to 0, Adam one-cycle optimisation is used with learning rate 0.0002, and seed 20260823 is fixed. Ground-truth database augmentation is disabled. Only the epoch-3 checkpoint is retained for each adapted arm of that initial matrix.

After the PointRCNN RPN stage, the epoch-3 RPN exports proposals and features for the training split. Its adapted RPN weights are combined with the official RCNN weights to initialise the offline RCNN stage. This handoff explains why the record contains separate three-epoch RPN and RCNN stages rather than one undifferentiated three-epoch PointRCNN run.

For CenterPoint in the initial matrix, adaptation starts from the official 80-epoch checkpoint; the two PU-GCN training conditions use observed-first inputs. Training uses all 3,712 training frames for three epochs with batch size 2, workers 0, Adam one-cycle, learning rate 0.0003, weight decay 0.01, and seed 666. Ground-truth sampling is disabled, while the configured world flip, rotation and scaling augmentations are retained. The adapted models are evaluated with the same AP_{R40} procedure as the frozen models.

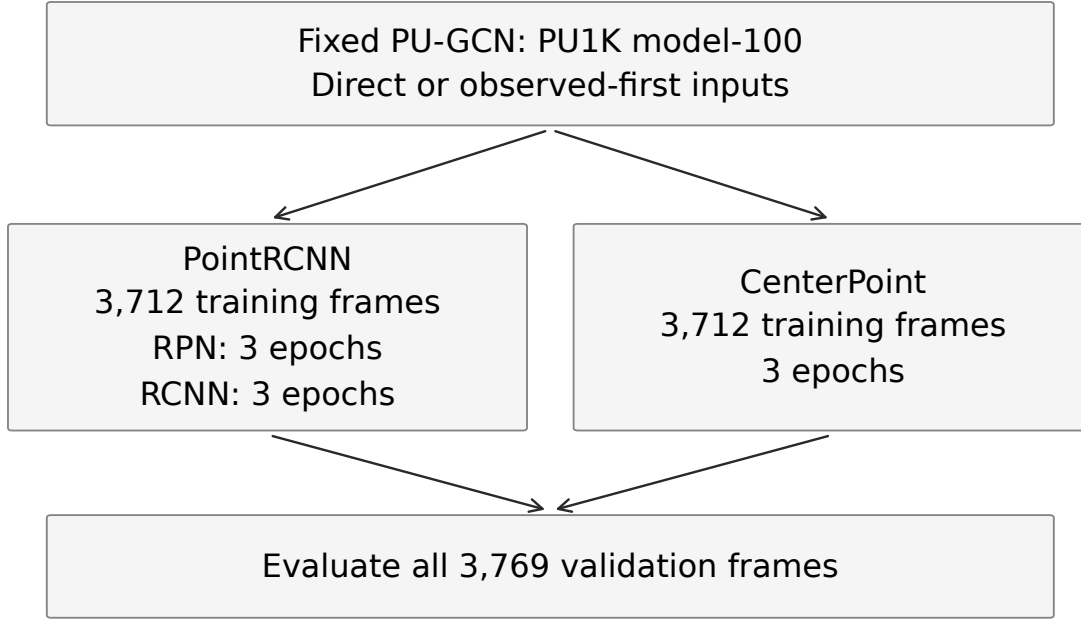


Figure 4.3: Initial three-epoch detector adaptation with a fixed PU-GCN upsampler. The 3,712 training frames denote the source split; each evaluation uses all 3,769 validation frames. PointRCNN adapts both input variants; CenterPoint adapts observed-first PU-GCN inputs only. The later extended schedules are described separately in the text.

The initial three-epoch loss decrease demonstrated completed optimisation, but that matrix did not include epoch-by-epoch validation. It is retained as the fixed-duration comparison. The subsequent observed-first experiments added the longer schedules and plateau assessment described below.

The three-epoch matrix was followed by a separate extension of the observed-first conditions in both lines. PU-GCN, the strict point budgets and the observation-preservation rule were retained. PointRCNN completed 18 RPN epochs in each line, followed by 18 RCNN epochs for Line A and a 24-epoch RCNN schedule for Line B. CenterPoint completed 12 epochs per line. Each epoch checkpoint was evaluated on the same 3,769 validation frames. The PointRCNN loader retained 3,265 effective training frames after object and range filtering; 3,712 denotes the source training split, not the number of frames remaining after that filtering.

Validation progress was assessed using Car Moderate 3D AP_{R40} . Starting with the first epoch’s score, a running reference was updated only when a subsequent score ex-

ceeded it by more than 0.2 AP points. Such an update reset the counter of consecutive epochs without a threshold-exceeding improvement; otherwise the counter increased by one. A completed schedule met the plateau criterion when this counter was at least three at its end. All epochs were inspected, including any later improvement after an intermediate plateau. The reported checkpoint was the numerical AP maximum over the schedule, which need not coincide with the last update of the threshold reference. The 0.2 threshold is an operational training rule, not a statistical significance test.

Changes in the total number of epochs were treated as new schedules. In particular, the 24-epoch Line B RCNN experiment restarted from the same RCNN initialisation with the selected RPN epoch 14 fixed. Its one-cycle learning-rate trajectory therefore differed from the earlier 18-epoch run; the two trajectories were not concatenated. Results from the earlier 12-epoch RPN and 18-epoch Line B RCNN schedules remain historical comparisons rather than additional segments of the final curves.

The extension did not retrain every baseline for the same duration. Line A retained the original- N input with official detector weights as its reference. Line B retained the sparse- M baseline after the earlier three-epoch adaptation, with the frozen sparse baseline also reported. The original- N reference is additionally shown to make the remaining deficit relative to the complete observation explicit. Checkpoint selection and final reporting used the same validation set. Consequently, these are validation-selected system comparisons with unequal training budgets, not independent test results or an isolated causal effect of point generation. The plateau criterion applies to the selected Car metric; it does not establish convergence of every class or of the optimisation parameters.

4.12 Executed Diagnostic and Mechanism Experiments

The diagnostics are grouped by the interface they test: patch construction, detector input admission, and classifier point reception. Each retains its own inputs, baseline and sample count. This organisation links the controls to the implementation described above while reserving measured performance and its interpretation for Chapter 5.

4.12.1 Patch Construction and Normalisation

Early full-scene patch extraction could group points that were adjacent in memory but not local on a physical surface. A locality audit measured the spatial extent of patches, and a revised local construction based on spatial FPS/ball/kNN neighbour-

hoods was executed. One early local version revealed a large duplicate-row fraction, which motivated a further unique-support and coverage constraint. Final local-patch comparisons were executed for PDANS, PU-GCN, PU-EdgeFormer, and corrected PU-Net on a controlled subset before the corresponding improved integrations were used in later evaluation.

The early local-patch control used a unique-neighbour floor of 256 and repeated rows to reach the 2,048-point network input. Its geometric audit covered 20 frames, with a separate source-row inspection of frame 000001 and detector evaluation on 256 frames. Recording patch extent, source coverage and distinct source rows made it possible to distinguish spatial locality from duplicate padding. This historical construction must not be confused with the later PU-GCN pipeline requiring 2,048 unique neighbours in Section 4.5.1.

A 256-frame 2×2 diagnostic crossed old versus corrected unit-sphere normalisation with old versus local patch construction for PU-Net. The purpose was to isolate whether extreme detector failure was caused by the upsampling network itself or by integration errors. The performance values belong to Chapter 5; Chapter 4 records only that all four combinations were executed under the same subset and detector evaluation.

The four PU-Net conditions combine the erroneous normalisation with old patches, corrected unit-sphere normalisation and inverse transformation with old patches, erroneous normalisation with local repeated patches, and corrected normalisation with those local patches. This crossed design compares each change while holding the other factor fixed. Both detectors use the same 256-frame subset and their corresponding unchanged baseline within the ablation. The local repeated-patch condition is retained as such; it is not renamed as the final unique-support implementation.

4.12.2 Detector Input Budgets and Generated-Point Dose

Two complementary controls were executed. First, an observed-first / enlarged-input experiment tested whether explicitly prioritising measured points reduces budget competition. Second, a separate “direct no-resample” PointRCNN copy removed the standard fixed-sample stage and supplied all valid points. Because the latter changes the detector input distribution even for the original baseline, it is treated strictly as a mechanism experiment rather than an alternative main protocol.

A region-split experiment was also executed to construct smaller spatial regions whose point counts remain under the PointRCNN cap. Shared core regions with a 3

m halo were used so that method and baseline comparisons refer to the same spatial support. These experiments were used to investigate whether the fixed full-frame sampling budget is sufficient to explain the degradation.

For a controlled set of frames, scripts measured occupied voxels, the number of newly activated voxels, and the frequency with which the 40,000-voxel cap was reached. Further detector-aware PDANS variants restricted generation to measured or selected voxel/proposal regions. These V2-V5 experiments were development diagnostics and were evaluated on explicitly limited subsets. They are not reported as new general-purpose upsampling methods.

The explicit occupancy audit uses 32 frames and records voxel counts before the test cap, cap-hit frequency and generated-only activation. Its units are detector-input cells under the configured voxel grid. They are not an additional voxel-based surface-accuracy metric. An ordering-only full-validation control separately retains the same point multiset while putting observations first; its same-multiset check covers eight method/line combinations. This control tests ordering under bounded voxel retention, while the later PU-GCN observed-first condition changes the composition of the input by copying measured rows and selecting predicted rows.

The dose study varies the fraction of generated points admitted to the PointRCNN input while holding its total budget fixed. The 256-frame fine-dose series includes 2.5%, 5%, 7.5% and 10% conditions across four methods and both lines, giving 32 completed combinations. A separate coarse-dose series includes 10%, 15%, 25%, 35%, 40% and 50%, giving 48 completed combinations on that subset. A matched real-point filling control was also evaluated for the low-dose comparison. It separates the effect of changing input composition from the effect of using method-generated coordinates.

The 10% dose was evaluated on all 3,769 validation frames. The corresponding full-validation 25% and 50% runs were not completed. Thus the higher-dose subset results are executed evidence, while full-split confirmation at those doses is absent. The subset baselines, historical PU-Net defect and compatibility implementation labels are retained in the associated result tables.

A 16-seed PointRCNN probe was executed with fixed input, checkpoint, split, and merge rules while varying only the evaluator/input sampling seed. Its purpose is to estimate how much small AP differences can arise from sampling stochasticity in the split-region pathway. This probe is especially important when interpreting apparent gains near one AP point on small or sampling-sensitive subsets.

The paired baseline and candidate are evaluated under the same seed within each repetition. The resulting spread describes the sampling-sensitive split-region pathway,

including repeated filling of sparse regions. It is not a variance estimate for the final per-frame deterministic 16,384-point comparison. The 16 evaluation seeds also do not constitute 16 independently retrained detectors.

4.12.3 ModelNet40 Controls and Input Reception

Several object-level controls were executed in addition to the main 12 branches. Independent 256- and 4,096-point mesh-reference PointNet++ models were trained. A PointNet++ first-set-abstraction reception probe measured how the network handles different input point counts. A deterministic repeat-run audit compared trained tensors under the same seed, and the repeat-x4 geometry control demonstrated why unequal-cardinality CD/HD should not be used for the final ranking.

The first-abstraction probe uses all 2,468 test objects to count available neighbours within radius 0.2 against the 32-slot grouping limit. It distinguishes occupied neighbourhood slots, repeated filling and points excluded by that limit for the 256-, 1,024- and 4,096-point controls. The 256-point input still passes through a layer requesting 512 FPS centres, so repeated centres are an architectural boundary shared by both 256-point controls. These measurements describe point reception rather than proving why a classifier prediction changes.

Training stability is examined through the saved 200-epoch histories of the 14 final and control branches and the selected same-seed checkpoint comparisons. Epoch-to-epoch variation and bit-identical repeated checkpoints answer different questions: the former describes one optimisation trajectory, and the latter checks repeatability for fixed stored data. Neither supplies cross-seed uncertainty for the main classification comparison.

4.13 Reproducibility and Validity Checks

The pipeline uses explicit checks at the data, generation, strict-output, downstream-input, and evaluation stages. A formal result is accepted only when the corresponding input list, checkpoint/configuration, outputs, and evaluation coverage can be identified.

For ModelNet40, preprocessing checks verify 12,311 expected objects, per-object shape, finite coordinates, class mapping, and split identity. Final upsampled data are checked for exact 4,096-point Line A or 1,024-point Line B shape. PointNet++

training logs are audited for the configured point count, actual batch tensor shape, 200 completed epochs, and independent checkpoints.

For KITTI, checks verify frame identity, XYZI shape, finite values, exact target cardinality, and full 3,769-frame validation coverage. Patch-based methods additionally record the transform and output integrity of local inference. PointRCNN and CenterPoint predictions are generated for every intended frame before AP is accepted into the formal matrix.

The project distinguishes smoke, pilot/diagnostic, and formal full-split evidence. A smoke run proves only that a real checkpoint can execute end to end on a small number of samples. A pilot or diagnostic run intentionally limits the sample set to isolate an implementation or mechanism. A formal result requires the fixed evaluation split and complete output coverage. This terminology prevents an attractive small-subset number from being mistaken for the final benchmark result.

A final protocol audit also separates historical and current evaluators. Earlier PointRCNN tables using a different precision-sampling convention are retained for provenance but are not combined numerically with the final AP_{R40} matrix. Likewise, early ModelNet40 $512 \rightarrow 2,048$ experiments remain genuine historical experiments but are not used as the final Line B protocol, which is $256 \rightarrow 1,024$.

The source package separates ModelNet40 HPC data from KITTI lab data. The retained HPC arrays include both lines for eight selected shapes, together with category scores, full training curves, per-object geometry, paired summaries, and reference-sensitivity audits. KITTI files include the final evaluation matrix and the earlier lab point-cloud and object-transition package. The local visualisation copies preserve their lab paths and case manifests; local rendering is not a separate source experiment.

For the historical matched-object audit, same-class oriented 3D IoU was used with thresholds 0.70 for Car and 0.50 for Pedestrian and Cyclist. Greedy matching produced maintained, confidence-degraded, localisation-degraded, lost, recovered, and missed-both events. These diagnostic events do not reproduce the complete official difficulty, DontCare, and score-integration procedure. They are kept separate from the final AP_{R40} evaluation. The three visual frames were selected in the original audit for high event counts, recovery examples, and multiple classes; they are not a random validation sample.

The ModelNet40 plates use common views and all rows in the stored arrays. KITTI overview and object figures retain all points inside explicitly defined spatial windows. PointRCNN examples use the archived effective 16,384-point inputs. CenterPoint examples reproduce the documented field-of-view, range, voxel-cap, and per-voxel reten-

tion rules and are checked against the case-manifest counts. Only saved qualifying predictions are drawn; no new detection inference is performed. Sources, transformations, output dimensions, and checks are recorded with the figures.

All figure PDFs use the thesis text width and a physical text size equal to the 12-point body. Editable vector graphics, raster previews, plotting scripts, and source data are included. File manifests retain SHA-256 hashes. The main comparisons and nonredundant class and difficulty results are reported in Chapter 5. The archived exports retain the complete class scores, secondary summaries and precision beyond the rounded manuscript values.

The completed protocols distinguish generated geometry, input assembly and downstream model adaptation. ModelNet40 supplies the controlled object comparison; KITTI adds scene integration and the targeted detector diagnostics. Their settings and evidence boundaries now define the comparisons used in Chapter 5, where the measurements and their interpretation are presented together.

Chapter 5

Results and Evaluation

The experiments show that increasing point count does not reliably improve downstream perception. The object-classification study identifies a limited recovery benefit after sparsification, whereas the scene-detection study shows large losses under frozen detectors and partial recovery after detector adaptation. This chapter establishes those task outcomes and then uses geometric measurements, genuine-sampling controls, and input diagnostics to interpret them. A visually plausible point cloud and a low geometric error are evidence about a representation, not substitutes for task evaluation.

All ModelNet40 results come from the completed HPC experiment and its exported arrays, training logs, and tables. All KITTI results come from the lab experiment. The initial KITTI adaptation matrix comprises 20 completed evaluations with 3,769 validation frames each; subsequent observed-first experiments add extended training and epoch-wise validation. Earlier multi-method studies and diagnostic subsets retain their own protocol and evaluation scope.

5.1 ModelNet40: Classification and Geometric Fidelity

OA and mAcc below are the recorded batch-aggregated statistics defined in Equation (4.12). All result values are preserved from the HPC runs. This distinction matters when interpreting small differences or comparing against externally reported accuracies.

5.1.1 Densifying the Original Observation

None of the five Line A upsampled representations exceeded the original 1,024-point baseline. Table 5.1 reports the selected-checkpoint overall accuracy (Best OA), epoch-200 accuracy (Final OA), and mean class accuracy at the selected checkpoint. PU-GCN was closest in Best OA, at 91.63% compared with 91.95%, while PDANS was nearly identical at 91.62%. The largest deficit was 1.41 percentage points (pp) for

Representation	Points	Best OA	Final OA	mAcc	Δ OA
Original baseline	1,024	91.95	91.31	88.00	0.00
PU-GCN	4,096	91.63	91.00	87.74	-0.32
PDANS	4,096	91.62	90.87	88.44	-0.33
EAR	4,096	91.48	90.85	88.19	-0.47
PU-Net	4,096	90.95	90.55	87.46	-1.00
PU-EdgeFormer	4,096	90.54	90.13	86.35	-1.41

Table 5.1: ModelNet40 Line A classification, with accuracies in percent. Δ OA is the Best-OA change from Original 1024 in percentage points.

PU-EdgeFormer. Each representation had its own classifier trained from random initialisation, so these losses occurred even when the downstream model was trained on the generated representation. Each row represents one seed-42 classifier training run over 200 epochs.

The same qualitative conclusion held at the fixed final epoch. The original representation reached 91.31%, compared with 91.00% for PU-GCN and 90.87% for PDANS. Thus the negative Line A outcome was not produced only by reporting the maximum of a noisy trajectory. The differences remain descriptive because the test set was also used for checkpoint selection and training was not repeated across independent random seeds.

Mean class accuracy and overall accuracy did not always move together. PDANS and EAR had slightly higher reported mAcc than the original baseline despite lower OA. Reported OA averages batch accuracies, whereas reported mAcc averages the class-specific batch ratios across categories. Their weights therefore differ, and a gain in one aggregate need not produce a gain in the other. Without retained object predictions, a sample-weighted recalculation or attribution to particular misclassified objects is not possible. The category scores supplied with the runs are analysed in Section 5.1.3. They are not converted into integer correct-prediction counts because per-object predictions were not retained.

5.1.2 Recovery after Fourfold Sparsification

Only PU-Net exceeded the native 256-point Line B baseline in Best OA: 91.27% versus 90.85%, a gain of 0.42 pp (Table 5.2). It remained 0.68 pp below Original 1024. Restoring the original point count therefore recovered only part of the classification

Representation	Points	Best OA	Final OA	mAcc	Δ OA
Sparse baseline	256	90.85	90.46	87.37	0.00
PU-Net	1,024	91.27	90.65	87.86	0.42
PDANS	1,024	90.34	89.46	86.20	-0.51
PU-GCN	1,024	90.06	89.63	86.41	-0.79
PU-EdgeFormer	1,024	89.32	88.58	85.21	-1.53
EAR	1,024	88.81	88.02	84.50	-2.04

Table 5.2: ModelNet40 Line B classification after 1,024 $\rightarrow$ 256 sparsification and 256 $\rightarrow$ 1,024 upsampling. The immediate baseline retains its native 256 points. Definitions match Table 5.1.

gap. The other methods performed below the sparse baseline, with deficits from 0.51 pp for PDANS to 2.04 pp for EAR.

At epoch 200, PU-Net retained a smaller advantage of 0.19 pp over the sparse baseline, 90.65% versus 90.46%. The recorded means over epochs 181–200 were 90.85% and 90.32%, respectively. This late-training record supports a persistent descriptive advantage rather than an isolated best checkpoint. Epochs from one optimisation trajectory are correlated, however, and their variation cannot substitute for training-seed uncertainty.

Figure 5.1 shows the complete test-accuracy trajectories. The later epochs show the sustained separation of EAR and PU-EdgeFormer below the sparse baseline. The best-to-final gap reached 0.88 pp for Line B PDANS and 0.79 pp for EAR. Reporting only Best OA would obscure that sensitivity. Under this training procedure, PU-Net provided the most useful recovery representation among the tested methods; the evidence does not establish a statistically significant or architecture-independent gain.

Figure 5.2 resolves the later training behaviour that is compressed in a full-epoch plot. In Line A, the final train–test gap was 6.51 pp for Original and 8.23–9.38 pp for the upsampled inputs. In Line B it was 2.52 pp for Sparse and 7.24–9.58 pp after upsampling. Higher training accuracy therefore coexisted with weaker test accuracy for most generated representations. This observation is consistent with greater fitting to the training representation, but does not isolate overfitting from representation quality or optimisation effects.

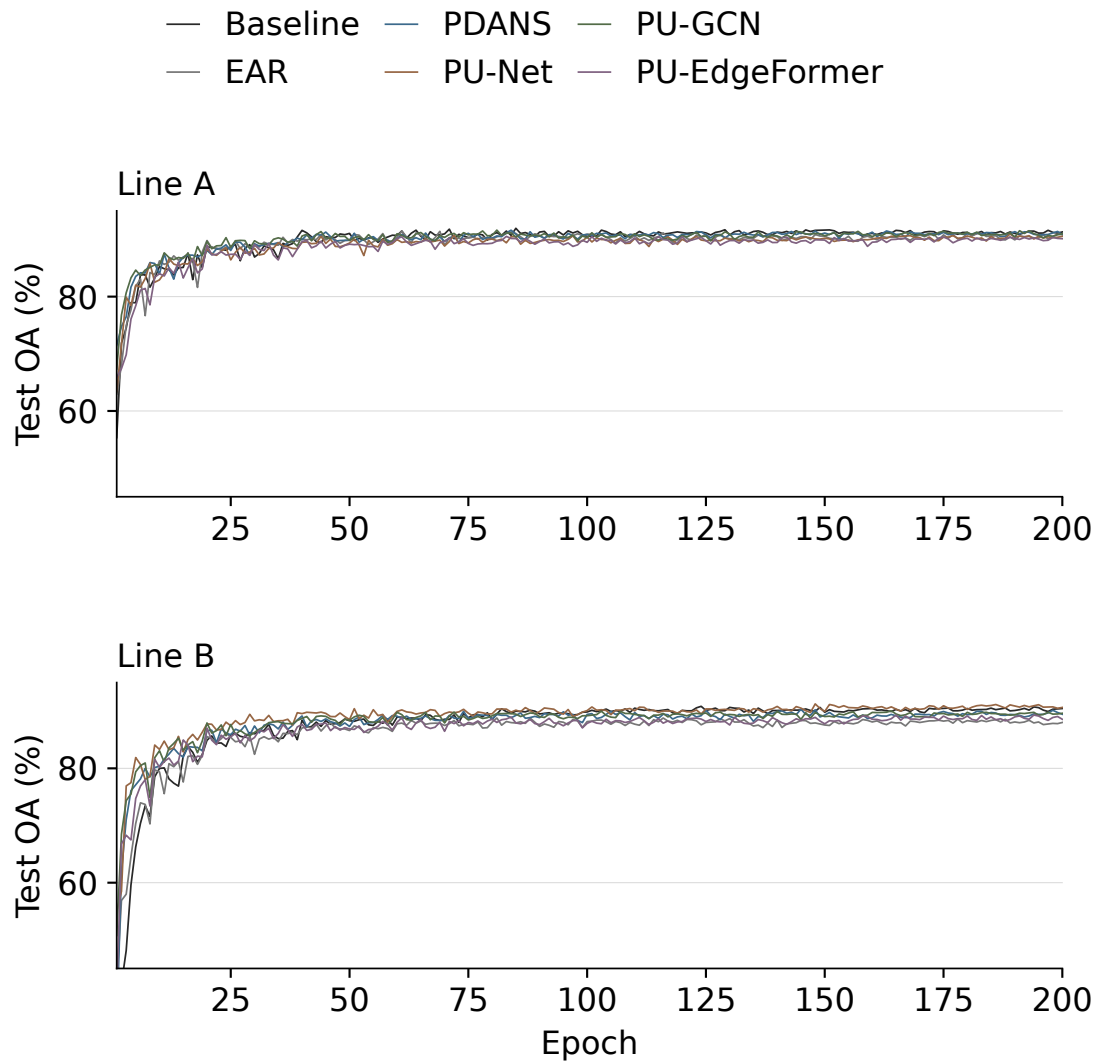


Figure 5.1: Test OA at every recorded epoch for the canonical branches. No smoothing or averaging across runs is applied. The curves describe one seed per representation.

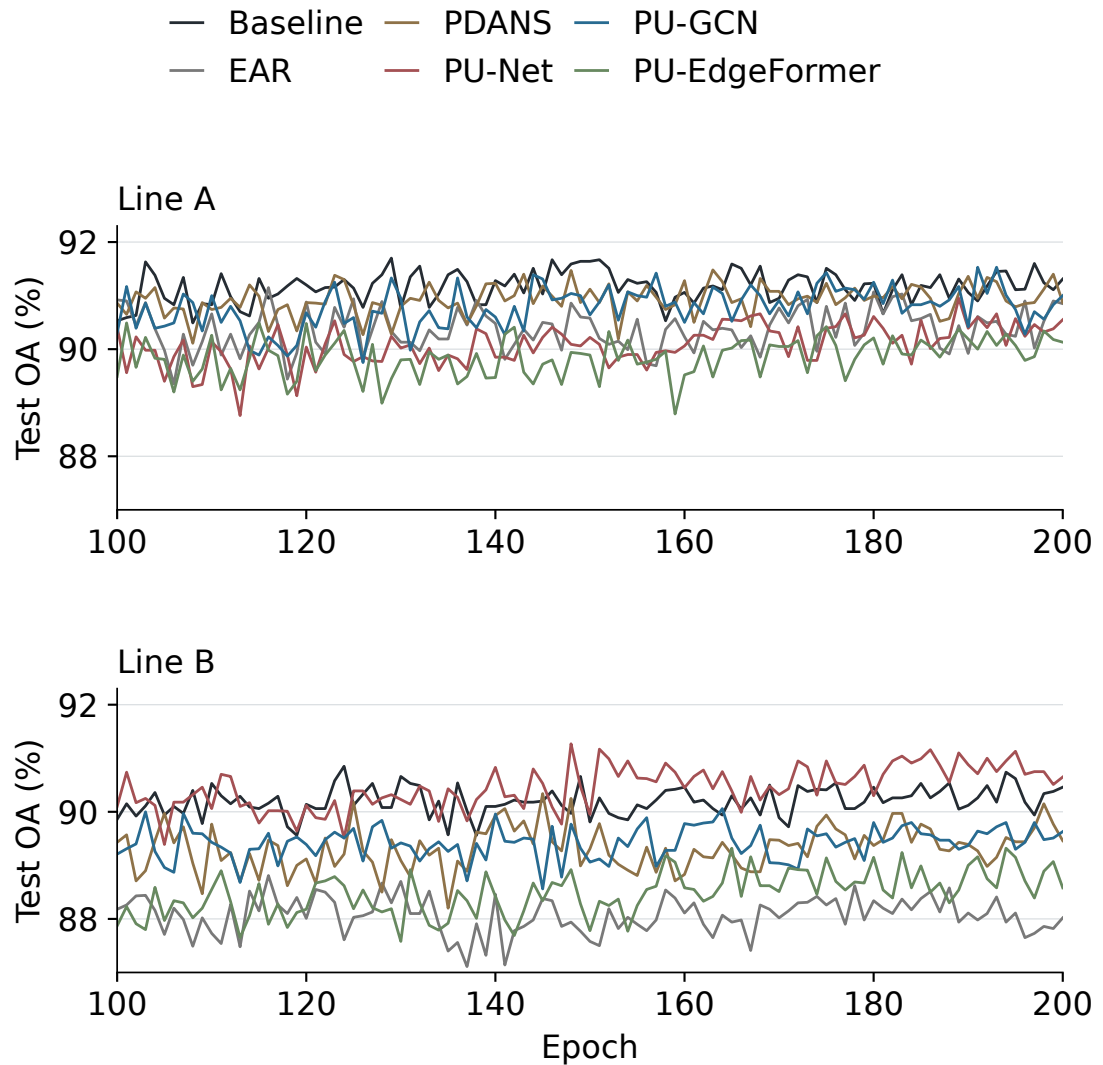


Figure 5.2: Unsmoothed test OA from epochs 100–200. This complements the complete trajectories by resolving the small late-training separations. Each curve is one training run; no seed confidence interval is implied.

Line	Method	Improved	Same	Reduced	Mean Δ (pp)
A	EAR	16	9	15	+0.19
A	PDANS	16	6	18	+0.44
A	PU-Net	15	6	19	-0.54
A	PU-GCN	18	5	17	-0.26
A	PU-EdgeFormer	17	7	16	-1.66
B	EAR	8	8	24	-2.86
B	PDANS	10	12	18	-1.17
B	PU-Net	15	15	10	+0.49
B	PU-GCN	13	8	19	-0.96
B	PU-EdgeFormer	11	9	20	-2.15

Table 5.3: Direction of reported category-score changes relative to the native baseline. Each row covers all 40 categories. The mean is an unweighted mean of archived category deltas and is not reconstructed OA.

5.1.3 Category-Dependent Classification Response

The overall results conceal different category responses. The exported Line A class scores contain both gains and losses for every method. PU-GCN improved 18 categories, left 5 unchanged, and reduced 17; PDANS improved 16 and reduced 18. Thus a method can lose overall accuracy without degrading every category. These are descriptive comparisons of the retained class aggregates, not paired counts of correctly classified objects.

Table 5.3 summarises all 40 categories in each method–line pair. In Line B, PU-Net had the most favourable balance: 15 improved, 15 unchanged, and 10 reduced. EAR reduced 24 categories, while PU-EdgeFormer reduced 20. This broader distribution of losses helps interpret their lower overall accuracy; it does not identify which individual objects changed their predictions.

Figures 5.3, 5.4, 5.5, and 5.6 retain the complete category distribution using a common colour range. Categories are split alphabetically across two panels per line, rather than selected by the size or sign of a result. For example, Line B PU-Net gained 25.00 pp on door but lost 22.62 pp on curtain. Its positive aggregate result therefore coexists with a substantial category-specific failure. Line B PU-GCN gained 11.11 pp on flower pot but lost 16.67 pp on bench. These differences motivate retaining the complete category distribution in the heatmaps; the underlying scores remain in the archived export.

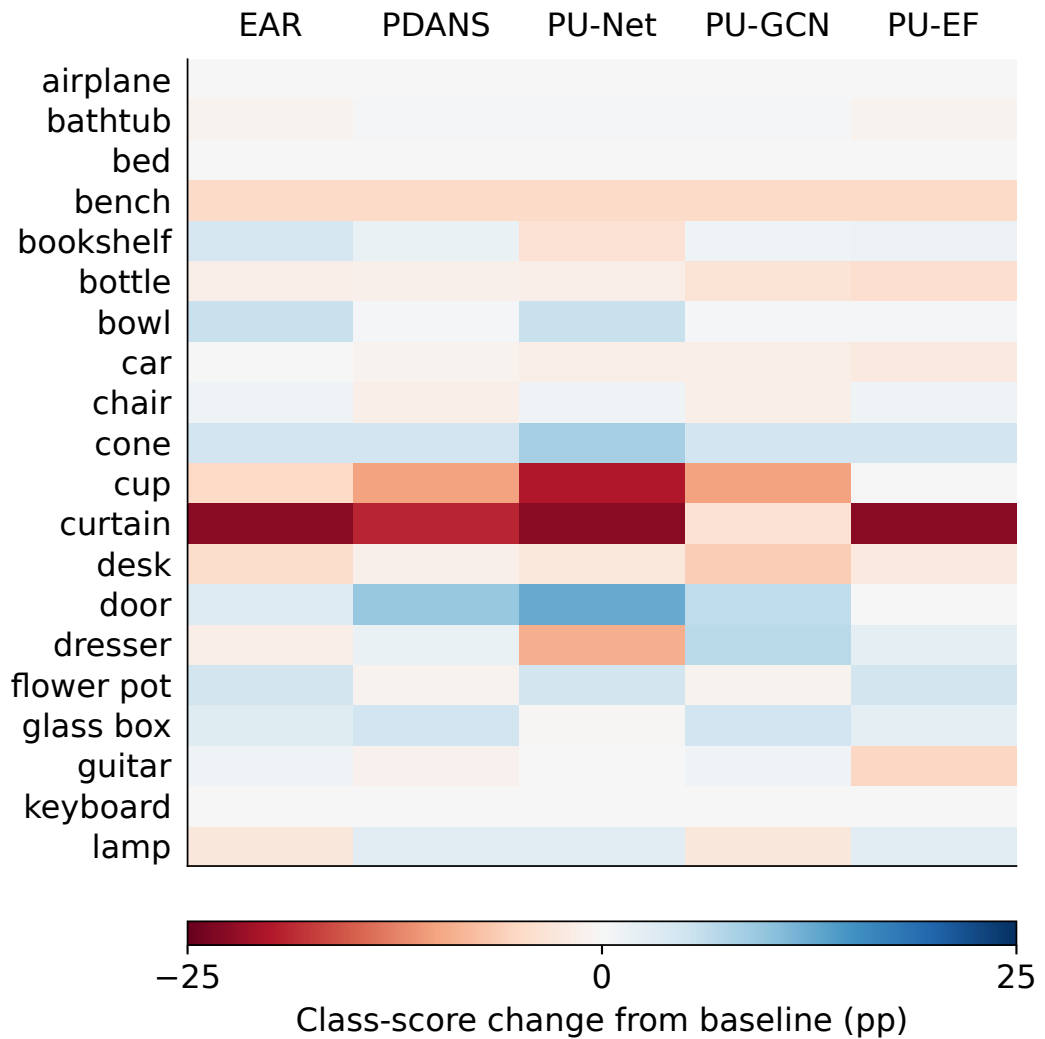


Figure 5.3: Reported class-score changes for Line A, alphabetical categories 1–20. Blue denotes improvement and red denotes reduction from the native baseline; the colour range is shared by all four maps. PU-EF denotes PU-EdgeFormer. These are archived category aggregates, with no reconstructed object-level counts.

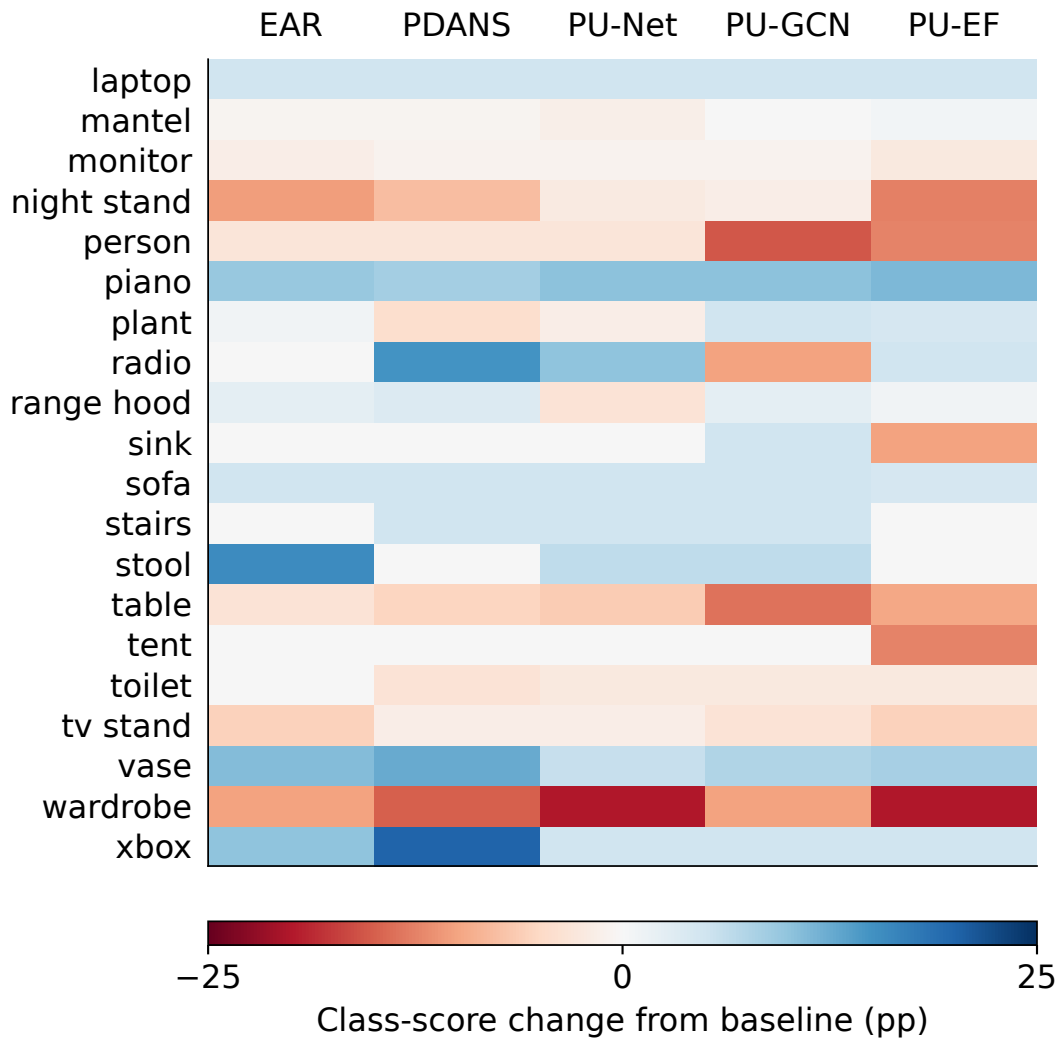


Figure 5.4: Reported class-score changes for Line A, alphabetical categories 21–40. Blue denotes improvement and red denotes reduction from the native baseline; the colour range is shared by all four maps. PU-EF denotes PU-EdgeFormer. These are archived category aggregates, with no reconstructed object-level counts.

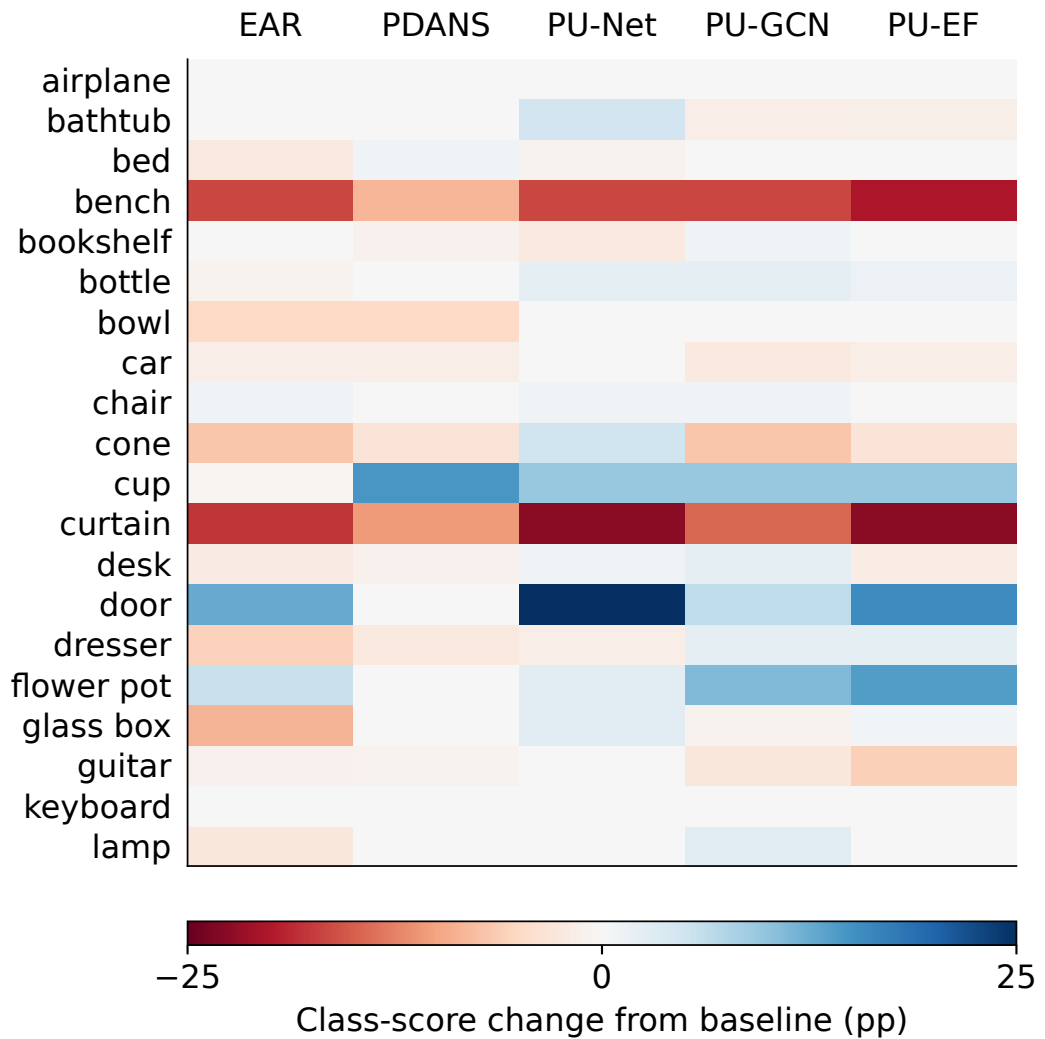


Figure 5.5: Reported class-score changes for Line B, alphabetical categories 1–20. Blue denotes improvement and red denotes reduction from the native baseline; the colour range is shared by all four maps. PU-EF denotes PU-EdgeFormer. These are archived category aggregates, with no reconstructed object-level counts.

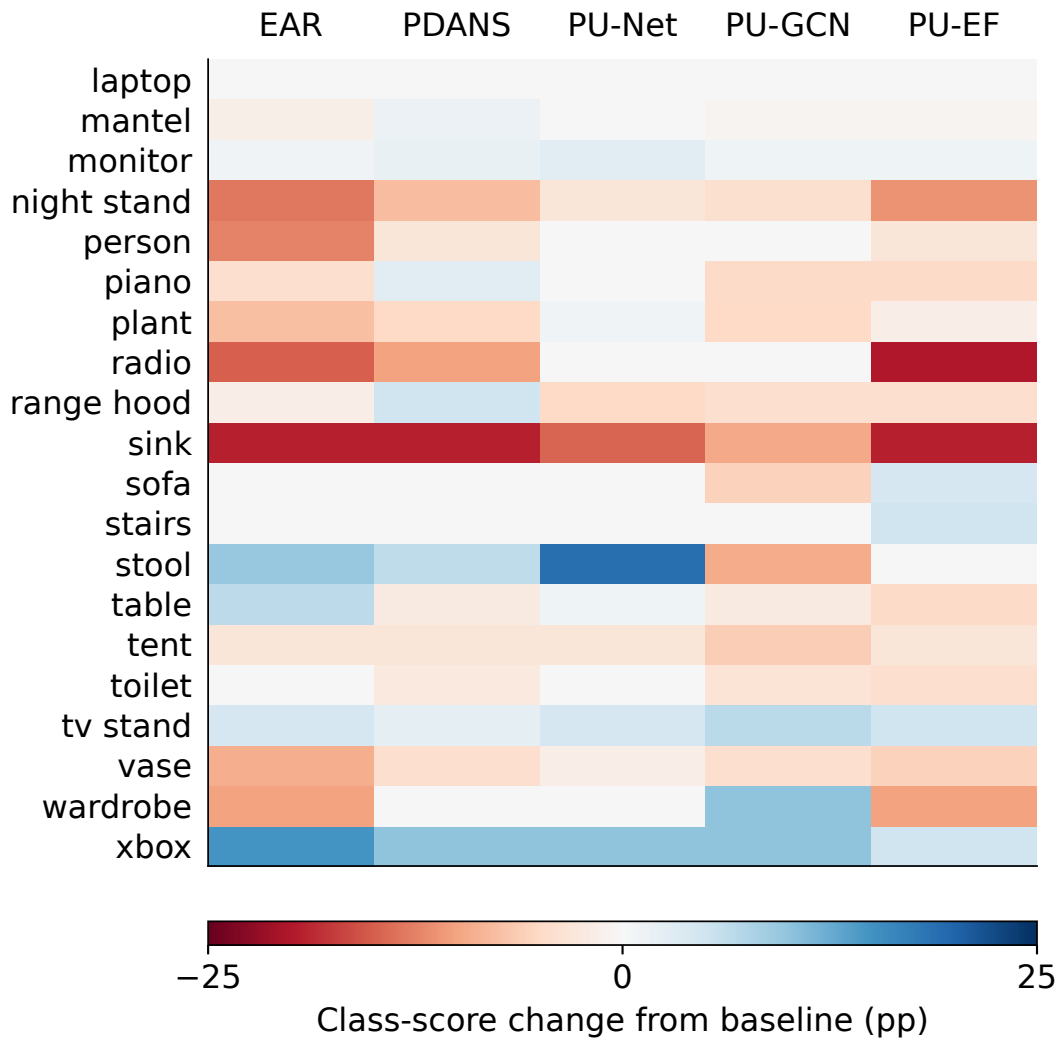


Figure 5.6: Reported class-score changes for Line B, alphabetical categories 21–40. Blue denotes improvement and red denotes reduction from the native baseline; the colour range is shared by all four maps. PU-EF denotes PU-EdgeFormer. These are archived category aggregates, with no reconstructed object-level counts.

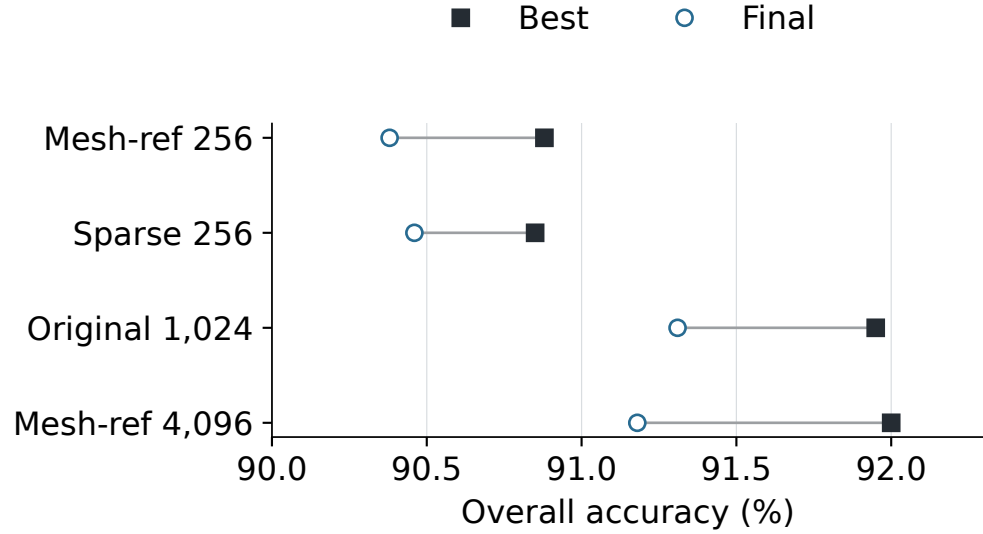


Figure 5.7: Selected and final OA for genuine mesh samples and the native sparse baseline. Four reused training shapes limit the mesh-reference training comparison; the test geometry contains no such exception.

5.1.4 Genuine-Sampling Controls and Point Reception

Increasing genuine surface samples also had a limited return above 1,024 points. Mesh-ref 256 reached 90.88% Best OA, close to the decimated baseline’s 90.85%. Mesh-ref 4096 reached 92.00%, only 0.05 pp above Original 1024. At epoch 200 it was below Original 1024, 91.18% versus 91.31% (Figure 5.7). These controls suggest diminishing benefit from additional surface samples for the configured PointNet++ SSG classifier, rather than a general upper bound on density.

Four training shapes in the mesh-reference builds were reused instead of regenerated under the final stable-seed rule. No test object was affected, so the equal-cardinality test geometry remains usable. The two trained mesh-reference controls are nevertheless provisional: their small OA differences should not support a strong causal claim without rebuilding those training shapes and repeating the control training.

The fixed-radius grouping and finite neighbourhoods of PointNet++ [11] motivate a possible explanation for the limited return. In the retained reception probe, the genuine and decimated 256-point inputs required repeated indices in 59.2% of neighbourhood slots. Original 1024 discarded 31.9% of points found within its queried neighbourhoods, while Mesh-ref 4096 discarded 78.7%. These are local grouping statistics with a fixed

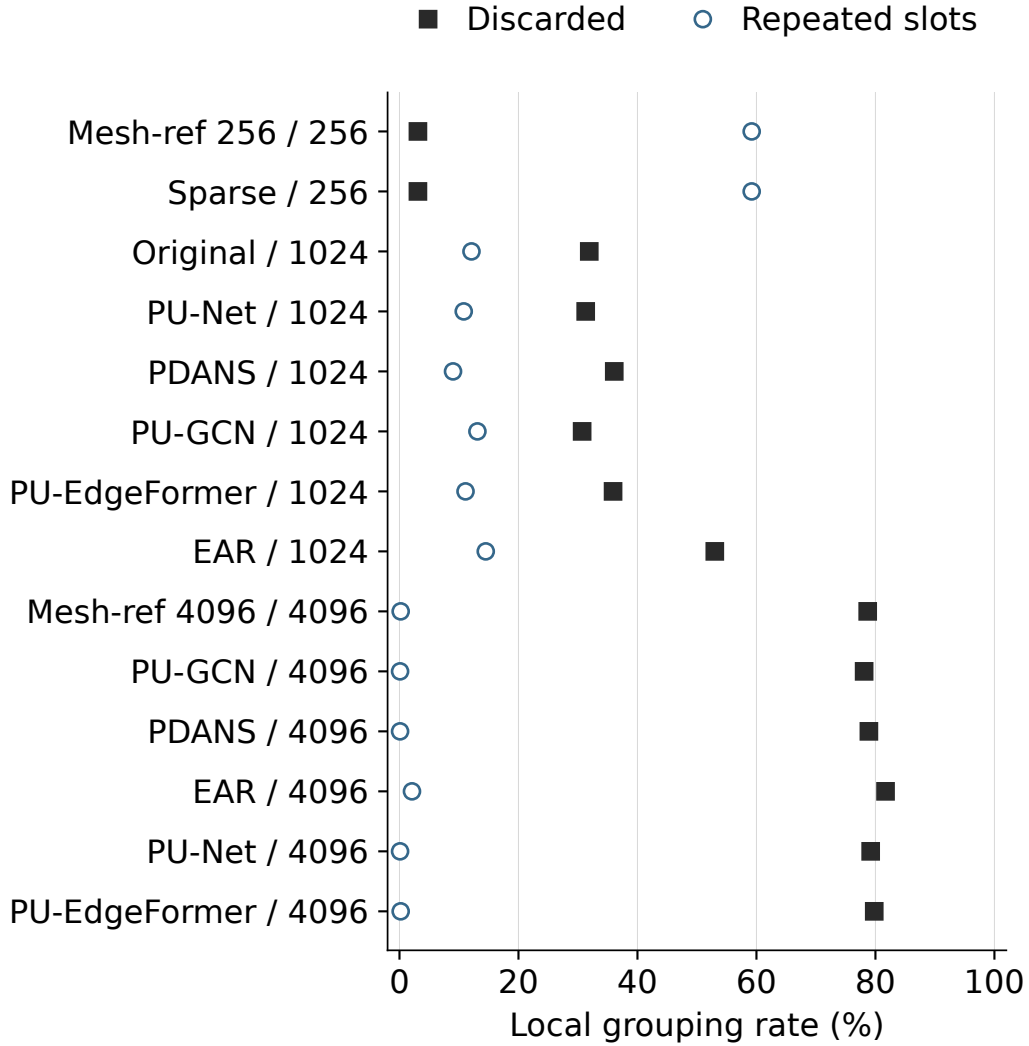


Figure 5.8: Recorded first-layer discard and repeated-slot rates for the retained representations. Rates are local grouping statistics, not complementary fractions of an entire cloud. The architecture is specified in Section 4.9.

neighbourhood budget, not percentages of globally unique points removed from the cloud.

Figure 5.8 reports repetition and discard rates. A fourfold increase in file cardinality did not produce a fourfold increase in distinct local evidence passed to the next layer. This is consistent with saturation under fixed grouping radii and capacities. Reception is not a uniquely identified cause of the accuracy differences: shape deformation, feature extraction, and optimisation can also contribute.

Method	CD	CD 95% interval	HD	$ \Delta\text{NUC} $	P2F*
EAR	0.0547	[0.0541, 0.0553]	0.1154	0.2023	0.08585
PDANS	0.0468	[0.0463, 0.0474]	0.1135	0.0725	0.08526
PU-Net	0.0497	[0.0492, 0.0502]	0.1112	0.0673	0.08413
PU-GCN	0.0423	[0.0418, 0.0428]	0.0932	0.3960	0.08539
PU-EdgeFormer	0.0467	[0.0461, 0.0472]	0.1109	0.0690	0.08638

Table 5.4: Line A geometry against the independent 4,096-point reference. CD and HD are unsquared distances in normalised coordinates. *P2F uses independently normalised clouds and meshes.

Method	CD	CD 95% interval	HD	$ \Delta\text{NUC} $	P2F*
EAR	0.0767	[0.0760, 0.0775]	0.1932	0.2294	0.08400
PDANS	0.0627	[0.0620, 0.0634]	0.1905	0.7141	0.08365
PU-Net	0.0646	[0.0641, 0.0650]	0.1083	0.1154	0.08425
PU-GCN	0.0558	[0.0552, 0.0563]	0.1514	0.4462	0.08798
PU-EdgeFormer	0.0619	[0.0613, 0.0625]	0.1776	0.1233	0.08866

Table 5.5: Line B geometry against Original 1024. Definitions match Table 5.4. The sparse 256-point baseline has a separate 256-point mesh reference and is not ranked in this table. The P2F column retains the separate cloud-to-mesh convention.

The earlier 512-point controls further show that the input protocol matters. PDANS reached 91.47% Best OA from 512 visible points against a native baseline of 91.26%, whereas the final 256-point comparison lost 0.51 pp. Both visible support and downstream input size changed; this is not an isolated estimate of sparsity. The complete earlier runs remain in the archived HPC results rather than being pooled with the final two-line comparison.

5.1.5 Equal-Cardinality Geometry

PU-GCN achieved the lowest mean unsquared Chamfer distance in both lines. The comparison used 2,468 paired test objects, a 4,096-point independent mesh reference in Line A, and Original 1024 in Line B. Tables 5.4 and 5.5 give the means and archived object-bootstrap intervals from 4,000 resamples of the 2,468 objects, using the resampling principle of Efron [33]. These intervals describe variation across test objects, not training seeds or checkpoints.

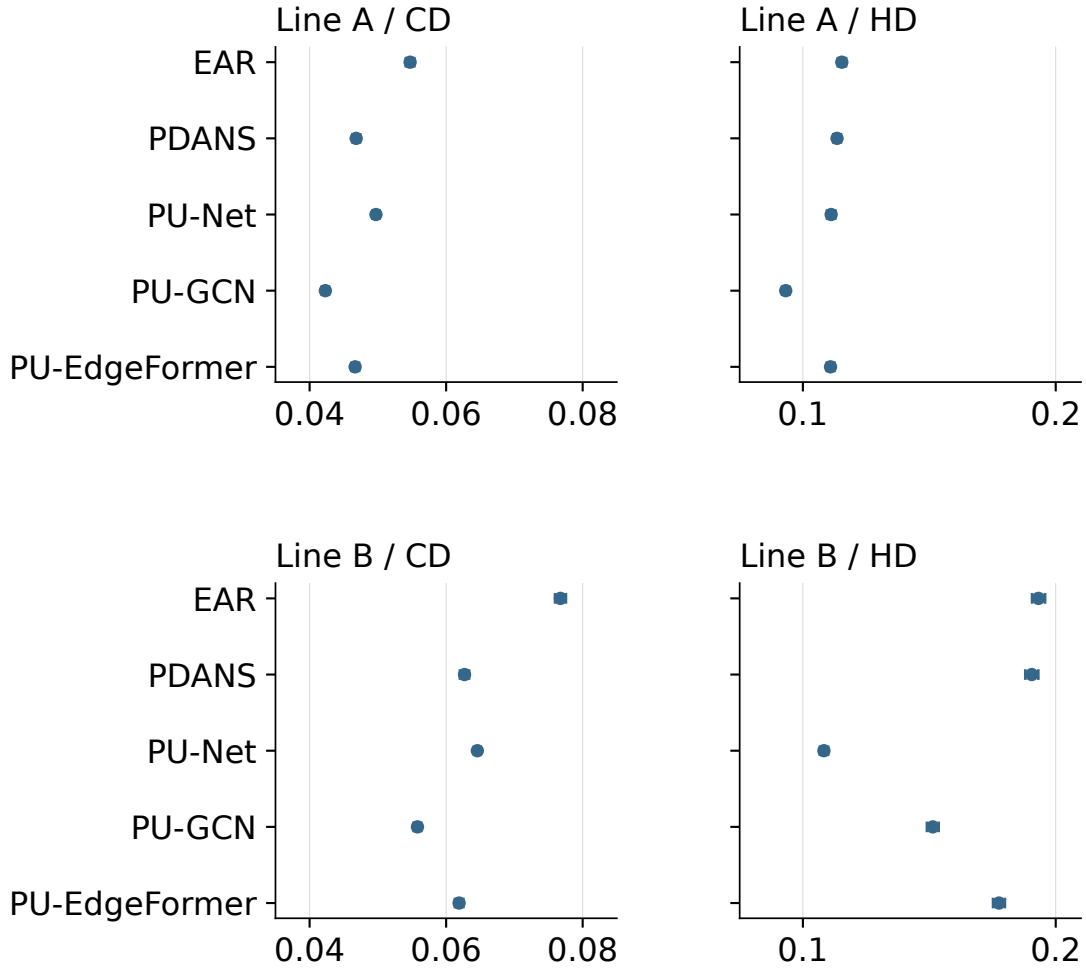


Figure 5.9: Mean CD and HD across 2,468 test objects. Error bars are archived 95% object-bootstrap intervals. The reference differs between the two lines as specified in Tables 5.4 and 5.5.

The Chamfer advantage did not imply the best result under every metric. PU-GCN also had the lowest mean Line A HD, but PU-Net had the lowest Line B HD: 0.1083 compared with 0.1514 for PU-GCN. Figure 5.9 shows the change in ordering. Chamfer distance averages nearest-neighbour residuals, whereas HD records the largest gap or outlier for each object. The rankings indicate a trade-off between average residuals and extreme local residuals under the retained normalised-reference protocol. Equal point counts control cardinality, but the independently normalised mesh reference can still contribute translation and scale differences (Section 4.10.1). These results are not presented as a common-transform surface-error ranking.

Uniformity also requires a reference. A low NUC indicates less variation in local counts, but an object surface need not have the lowest achievable NUC. The analysis therefore averages each object’s absolute deviation from its reference NUC; this differs from the absolute difference between dataset means. Query centres were not fully shared by shape ID across methods, so NUC differences include query-placement variation and remain secondary to the principal distance and task results.

The P2F columns in Tables 5.4 and 5.5 retain the completed secondary surface audit. Clouds and meshes were independently centred and scaled, so even the original mesh-sampled input had mean P2F 0.08662; the sparse control had 0.08675. PU-Net had the lowest Line A mean P2F (0.08413), and PDANS the lowest Line B value (0.08365). This ordering differs from CD and HD, but the coordinate convention prevents interpreting it as an absolute common-frame surface ranking. Co-location in the tables does not make the reference protocols interchangeable.

The mean geometric ranking is supported by the number of objects on which a method wins, rather than by a few extreme values alone. PU-GCN had the lowest CD on 2,319 of 2,468 Line A objects (93.96%) and 2,303 Line B objects (93.31%). For HD, however, its winner share changed from 72.65% in Line A to 9.60% in Line B. PU-Net had the lowest HD on 2,218 Line B objects (89.87%). Figure 5.10 therefore exposes a systematic disagreement between average nearest-neighbour fidelity and extreme-error behaviour.

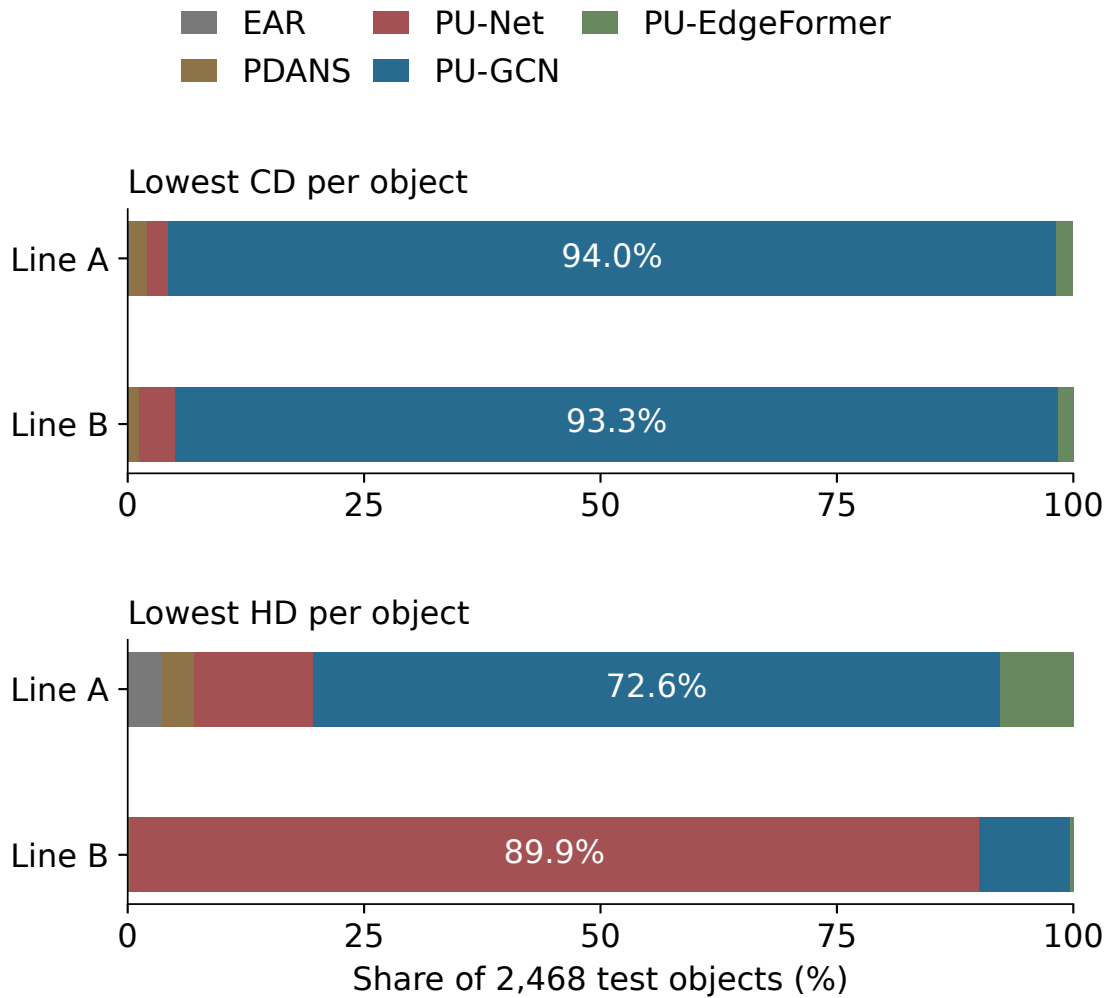


Figure 5.10: Share of 2,468 test objects on which each method has the lowest archived CD or HD. All five methods and both lines are retained. Winner counts are taken from the HPC object-wise summary; they measure geometric comparisons, not classification successes.

5.1.6 Multi-Method Point-Cloud Evidence

Figures 5.11 and 5.12 retain two of the eight archived HPC examples, one per comparison line. The airplane illustrates agreement between the lowest CD and HD; the chair illustrates their different method orderings. These examples were selected for this geometric contrast, not to estimate its frequency. No per-object classification transition is assigned because complete classifier predictions were not retained.

Each plate contains seven panels: input, reference and all five methods. Every output point in panels (c–g) is coloured by its Euclidean nearest-neighbour distance

to the same reference, with one full-range colour scale shared across methods within that plate. All stored points are shown under the same orthographic view, extent and marker size. This colour measures output-to-reference deviation only; CD also includes reference-to-output coverage, and HD takes the worst of both directions. Colour intensity is consequently not the complete CD or HD.

For `airplane_0627` in Line A, PU-GCN had the lowest CD (0.02979) and HD (0.09912), while PU-Net had 0.04231 and 0.11862, respectively. PU-EdgeFormer’s HD was close to PU-GCN’s at 0.09976 despite its higher CD of 0.03285. Figure 5.11 locates deviations around the same wings, tail and fuselage under the retained reference.

For `chair_0890` in Line B, PU-GCN had lower CD than PU-Net, 0.03281 versus 0.04330, but higher HD, 0.18622 versus 0.06956. Figure 5.12 therefore complements the dataset-level winner shares: better average proximity can coexist with a worse extreme residual for the same object. The sparse input, reference and generated outputs retain the seat, back and narrow supports in a common view. Neither their denser appearance nor their geometric ordering establishes a classification improvement.

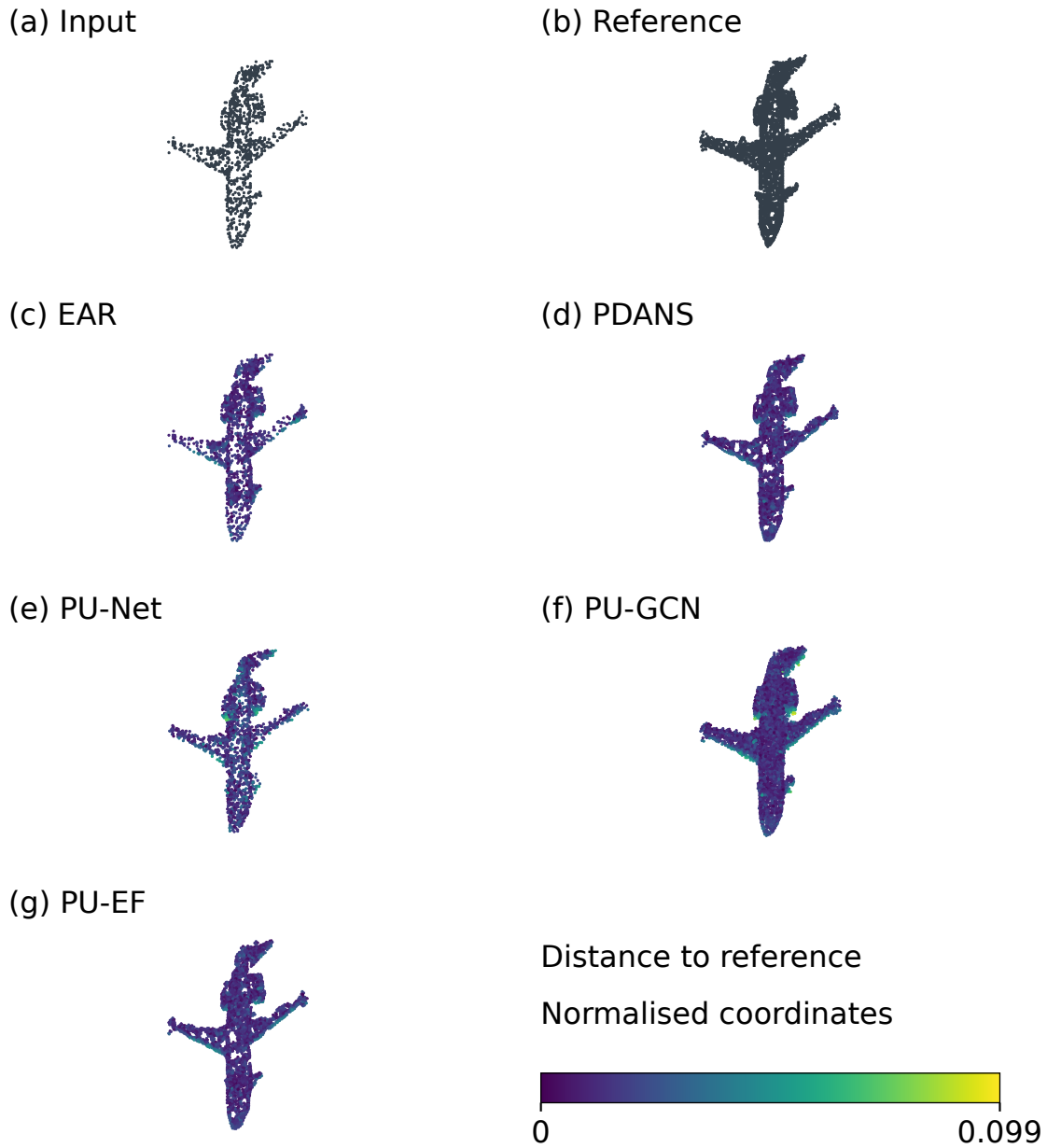


Figure 5.11: HPC airplane_0627. (a) Input, 1,024 points; (b) independent mesh reference, 4,096 points; (c–g) five 4,096-point outputs, all coloured by output-to-reference Euclidean distance in normalised coordinates. PU-EF denotes PU-EdgeFormer. Every method shares the view and colour range; the range is specific to this object.

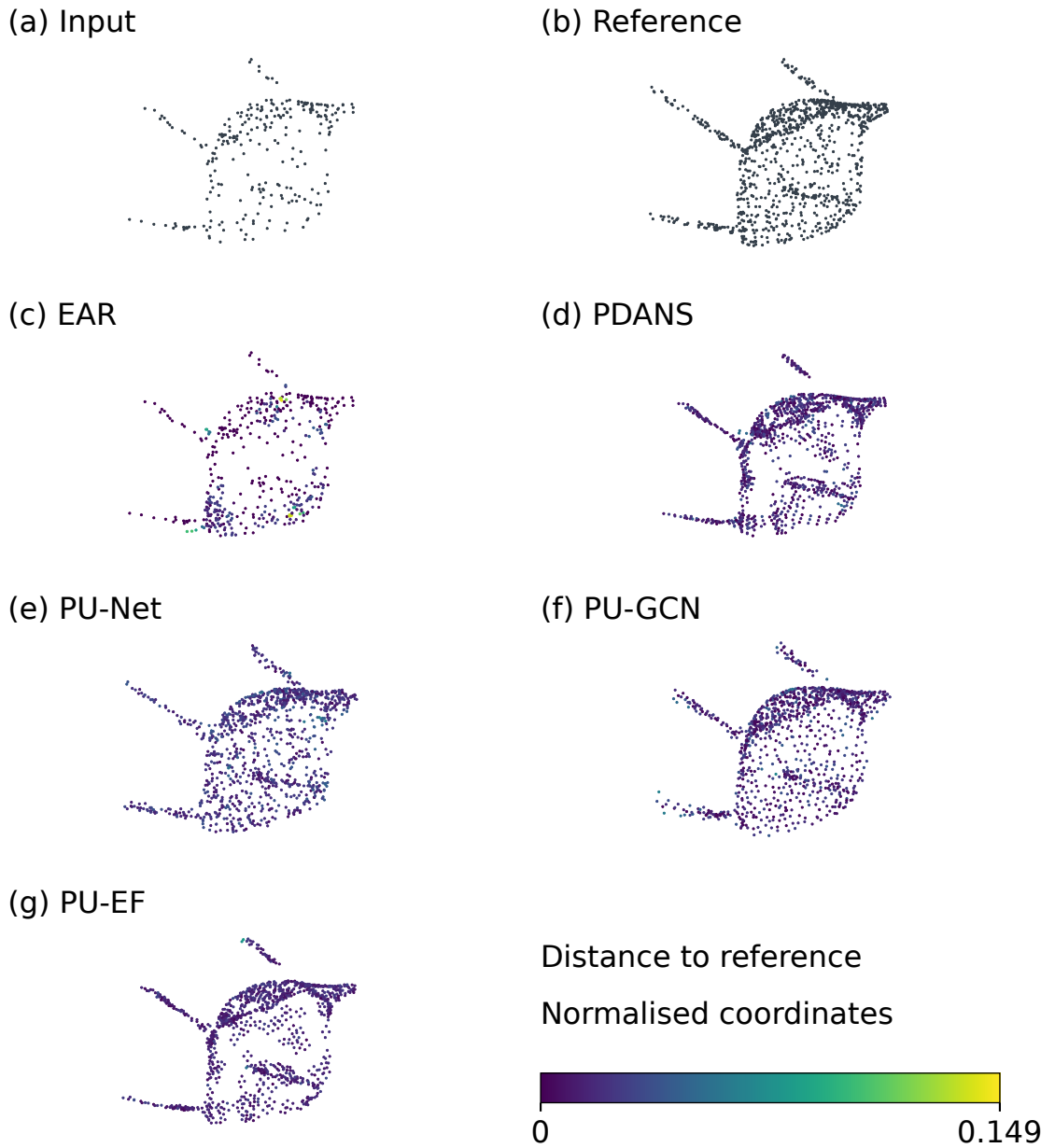


Figure 5.12: HPC chair_0890. (a) Sparse input, 256 points; (b) Original 1024 reference; (c–g) five 1,024-point outputs, all coloured by output-to-reference Euclidean distance in normalised coordinates. PU-EF denotes PU-EdgeFormer. Every method shares the view and colour range; the range is specific to this object.

5.2 KITTI: Frozen Detection and Detector Adaptation

5.2.1 Three-Epoch Full-Validation Comparison

The initial three-epoch PU-GCN study completed all 20 planned detector evaluations, each covering the same 3,769 validation frames. The matrix is intentionally asymmetric: PointRCNN includes direct-input adaptation in both lines, whereas CenterPoint adaptation was completed for observed-first inputs and the sparse baseline. An unexecuted direct-adapted CenterPoint arm is not inferred. Tables 5.6–5.9 preserve this original matrix. The later observed-first schedule extensions are reported separately in Section 5.2.5.

Throughout this section, *frozen* denotes the released detector weights, and *adapted* denotes detector weights after training on the corresponding representation. PU-GCN itself remained fixed at its released PU1K model-100 checkpoint. Direct inputs contain the strict network output; observed-first inputs concatenate all observed rows with a deterministically selected set of predicted rows. Those input constructions have the same total cardinality, but different measured-point preservation.

5.2.2 PointRCNN

For PointRCNN, frozen direct upsampling reduced Line A Car Moderate 3D AP_{R40} from 81.95 to 61.93. Preserving observations increased it to 64.27. Adaptation further raised the direct and observed-first results to 68.88 and 71.01, respectively (Table 5.6). Observation preservation helped under both weight regimes, but the best three-epoch arm remained 10.95 AP points below the original frozen baseline.

Line B revealed a larger loss under direct insertion. The frozen sparse baseline reached 65.58, compared with 30.09 for direct PU-GCN and 35.53 for observed-first PU-GCN. Training the detector on the direct representation raised AP to 46.61, and observed-first adaptation raised it to 55.21 (Table 5.7). This was substantial recovery relative to the frozen generated inputs, but it did not exceed the adapted sparse baseline at 68.33. The residual observed-first deficit was 13.12 AP points.

The sparse-baseline adaptation is an essential control. It improved PointRCNN from 65.58 to 68.33 without generating points. Comparing adapted upsampling only with a frozen sparse baseline would attribute some benefit of downstream optimisation to the point generator. The paired rows in Table 5.7 keep that control explicit. These results favour a representation mismatch explanation for part of the frozen-model loss,

Input	Weights	Easy	Moderate	Hard	Δ Mod.
Baseline	Frozen	92.43	81.95	77.85	+0.00
Direct	Frozen	84.74	61.93	57.79	-20.02
Direct	Adapted	85.17	68.88	64.67	-13.07
Observed-first	Frozen	85.04	64.27	60.09	-17.68
Observed-first	Adapted	86.71	71.01	66.79	-10.95

Table 5.6: PointRCNN Line A Car 3D AP_{R40}, 3,769 frames per row. All deltas reference the original frozen baseline. No adapted original baseline was trained, so adapted-row deltas change both input and weights.

Input	Weights	Easy	Moderate	Hard	Δ Mod.
Baseline	Frozen	85.20	65.58	61.28	+0.00
Baseline	Adapted	84.39	68.33	64.26	+0.00
Direct	Frozen	46.47	30.09	25.80	-35.48
Direct	Adapted	68.26	46.61	40.25	-21.72
Observed-first	Frozen	55.02	35.53	30.94	-30.05
Observed-first	Adapted	74.20	55.21	49.06	-13.12

Table 5.7: PointRCNN Line B Car 3D AP_{R40}. Every row evaluates 3,769 frames. Frozen rows reference the frozen sparse baseline; adapted rows reference the separately adapted sparse baseline.

while the remaining deficit shows that the three-epoch adaptation procedure did not restore baseline performance; it does not uniquely identify the remaining cause.

5.2.3 CenterPoint and Class Dependence

CenterPoint recovered more of the original performance under observed-first adaptation. Its Line A baseline was 79.28 Car Moderate 3D AP_{R40}. Frozen direct and observed-first inputs reached 60.61 and 63.21, while adapted observed-first reached 74.70 (Table 5.8). The latter remained 4.58 AP points below the original frozen detector. As for PointRCNN, there was no independently adapted original Line A baseline; this residual is a comparison between two complete systems rather than a controlled effect of geometry at fixed weights.

In Line B, frozen direct input reached 35.35 against the sparse baseline’s 64.60. Observation preservation increased it to 44.09, and adaptation raised it to 61.66. The adapted sparse baseline reached 68.05, leaving a matched-weight-family deficit of 6.39 AP points (Table 5.9). Greater recovery than PointRCNN does not mean that the

Input	Weights	Easy	Moderate	Hard	Δ Mod.
Baseline	Frozen	88.39	79.28	76.74	+0.00
Direct	Frozen	81.89	60.61	57.91	-18.67
Observed-first	Frozen	83.31	63.21	61.04	-16.07
Observed-first	Adapted	86.82	74.70	72.66	-4.58

Table 5.8: CenterPoint Line A Car 3D AP_{R40} , 3,769 frames per row. Deltas reference the frozen original baseline. The direct-adapted configuration was not part of the completed matrix.

Input	Weights	Easy	Moderate	Hard	Δ Mod.
Baseline	Frozen	81.40	64.60	59.97	+0.00
Baseline	Adapted	83.75	68.05	64.63	+0.00
Direct	Frozen	56.42	35.35	31.20	-29.24
Observed-first	Frozen	66.92	44.09	40.03	-20.50
Observed-first	Adapted	80.41	61.66	57.53	-6.39

Table 5.9: CenterPoint Line B Car 3D AP_{R40} , 3,769 frames per row. Frozen and adapted arms use their corresponding sparse-baseline control.

generated geometry was intrinsically better for all voxel detectors; the architectures, initial checkpoints, and training procedures differ.

The three-epoch CenterPoint findings were not confined to Cars. For Line A, observed-first adaptation reached Pedestrian and Cyclist Moderate AP of 48.42 and 57.70, below the original baseline’s 50.65 and 64.61. For Line B the corresponding values were 44.06 and 33.56, compared with 47.44 and 42.07 for the adapted sparse baseline. Figure 5.13 displays these class-specific comparisons. The three-epoch observed-first adaptation therefore did not exceed the appropriate reference for any of these three Moderate 3D metrics.

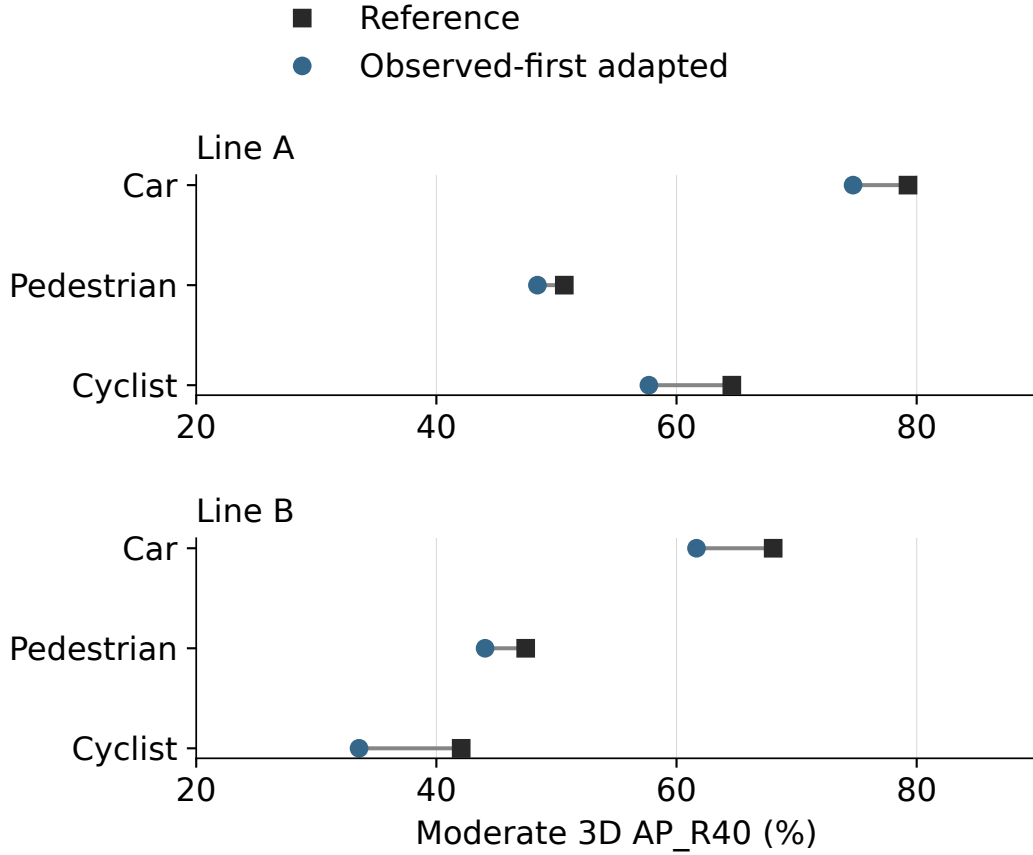


Figure 5.13: CenterPoint Moderate 3D AP_{R40} for the three-epoch observed-first adapted representation and its reference. Line A uses the frozen original baseline; Line B uses the adapted sparse baseline. All values use 3,769 frames.

5.2.4 Class, Difficulty, and Projection Results

The Car 3D results are already complete in Tables 5.6–5.9 and are not repeated here. Tables 5.10 and 5.11 give PointRCNN bird’s-eye-view (BEV) and image-box AP, while Table 5.12 supplies the remaining CenterPoint Pedestrian and Cyclist 3D results and Table 5.13 its BEV results. Values are AP_{R40} percentages over all 3,769 frames; F and A denote frozen and adapted detector weights. Direct and observed-first refer to the PU-GCN input construction. CenterPoint image-box AP was not exported.

5.2.5 PU-GCN Inputs and Detector Retraining

The training work updated detector weights while keeping PU-GCN fixed. It progressed from a small adaptation screen to the three-epoch, full-validation matrix and

Input / weights	Class	Easy	Moderate	Hard
A / Baseline / F	Car	95.87	88.86	86.43
A / Direct / F	Car	91.65	71.45	68.80
A / Direct / A	Car	92.28	80.52	76.09
A / Observed / F	Car	92.18	73.79	70.89
A / Observed / A	Car	94.00	81.03	78.32
B / Baseline / F	Car	91.53	74.92	71.90
B / Baseline / A	Car	91.75	77.97	73.67
B / Direct / F	Car	59.93	38.74	32.48
B / Direct / A	Car	82.49	58.20	51.48
B / Observed / F	Car	65.65	43.86	38.93
B / Observed / A	Car	87.48	67.30	60.73

Table 5.10: PointRCNN BEV AP_{R40} on the final validation set.

Input / weights	Class	Easy	Moderate	Hard
A / Baseline / F	Car	99.19	92.47	89.96
A / Direct / F	Car	95.52	74.57	71.80
A / Direct / A	Car	95.57	84.01	79.40
A / Observed / F	Car	95.78	77.21	74.37
A / Observed / A	Car	97.32	86.18	81.70
B / Baseline / F	Car	95.66	80.04	75.59
B / Baseline / A	Car	95.31	81.52	78.86
B / Direct / F	Car	63.38	39.83	34.99
B / Direct / A	Car	88.58	61.78	54.84
B / Observed / F	Car	69.00	46.46	39.98
B / Observed / A	Car	91.78	71.12	64.30

Table 5.11: PointRCNN Image-Box AP_{R40} on the final validation set.

then to extended observed-first training. The source training split contained 3,712 frames; the later PointRCNN loader retained 3,265 after filtering. All main comparisons below use 3,769 validation frames. The earlier 256-frame pilot remains a separate development experiment rather than part of the full-validation result.

The three-epoch full-validation experiment followed a smaller adaptation screen, which should remain visible as completed work. It used 64 selected training frames (63 effective after filtering), seed 20260823, three RPN epochs and three RCNN epochs. Six input conditions were evaluated before and after adaptation on the same 256 validation frames; all twelve evaluations completed. Table 5.14 reports their Moderate results under the explicitly recorded legacy Python evaluator. Change is adapted

Input / weights	Class	Easy	Moderate	Hard
A / Baseline / F	Pedestrian	54.04	50.65	46.25
A / Baseline / F	Cyclist	79.24	64.61	60.99
A / Direct / F	Pedestrian	49.07	45.08	41.06
A / Direct / F	Cyclist	68.40	42.83	40.32
A / Observed / F	Pedestrian	49.61	45.98	42.02
A / Observed / F	Cyclist	73.38	49.45	46.49
A / Observed / A	Pedestrian	51.96	48.42	44.17
A / Observed / A	Cyclist	77.96	57.70	54.06
B / Baseline / F	Pedestrian	27.20	24.57	21.67
B / Baseline / F	Cyclist	25.81	15.46	14.49
B / Baseline / A	Pedestrian	51.70	47.44	42.08
B / Baseline / A	Cyclist	63.17	42.07	39.46
B / Direct / F	Pedestrian	28.88	24.89	21.53
B / Direct / F	Cyclist	27.12	15.28	14.36
B / Observed / F	Pedestrian	32.55	29.03	24.99
B / Observed / F	Cyclist	34.28	20.02	18.82
B / Observed / A	Pedestrian	48.88	44.06	38.98
B / Observed / A	Cyclist	55.80	33.56	31.59

Table 5.12: CenterPoint Pedestrian and Cyclist 3D AP_{R40} on the final validation set.

minus frozen AP; Gap is adapted AP minus the matching adapted baseline, both in AP points. These values are not points on an R40 learning curve. The complete Easy/Moderate/Hard triplets are retained in the archived experiment outputs.

The screen recovered more than four AP points for both PU-GCN inputs and for sparse PDANS, but not for dense PDANS. Both adapted baselines also deteriorated. Thus a positive before/after change alone would overstate the benefit: all adapted generated inputs remained below their paired adapted baseline. The later stage expanded training to 3,712 frames, initially evaluated a 256-frame pilot, and ultimately completed the 3,769-frame, 20-arm matrix. Increasing training coverage, changing the integration configuration and changing the evaluation scope prevent these stages from being interpreted as a single controlled sample-size ablation.

The training losses decreased. For PointRCNN Line A direct input, median RPN loss changed from 1.624055 to 1.347459 and RCNN loss from 1.043027 to 0.959636 between epochs 1 and 3. CenterPoint loss changed from 2.33 to 2.11 for Line A observed-first, from 3.69 to 3.10 for Line B observed-first, and from 3.00 to 2.65 for the sparse baseline. These were successful optimisation records, but the original matrix

Input / weights	Class	Easy	Moderate	Hard
A / Baseline / F	Car	92.18	88.43	87.03
A / Baseline / F	Pedestrian	58.98	55.74	51.62
A / Baseline / F	Cyclist	82.38	68.70	65.26
A / Direct / F	Car	88.99	70.68	68.96
A / Direct / F	Pedestrian	53.09	49.38	45.80
A / Direct / F	Cyclist	72.31	46.71	44.09
A / Observed / F	Car	89.77	72.83	71.23
A / Observed / F	Pedestrian	54.17	50.82	46.94
A / Observed / F	Cyclist	76.27	52.44	49.51
A / Observed / A	Car	91.85	84.93	83.68
A / Observed / A	Pedestrian	55.58	52.61	48.34
A / Observed / A	Cyclist	81.11	60.87	57.30
B / Baseline / F	Car	89.12	77.31	73.16
B / Baseline / F	Pedestrian	34.49	31.55	28.01
B / Baseline / F	Cyclist	28.27	17.13	16.33
B / Baseline / A	Car	90.17	80.20	76.79
B / Baseline / A	Pedestrian	58.01	53.82	48.39
B / Baseline / A	Cyclist	66.05	45.01	42.69
B / Direct / F	Car	74.51	47.53	42.86
B / Direct / F	Pedestrian	33.53	29.56	25.75
B / Direct / F	Cyclist	29.98	17.00	16.42
B / Observed / F	Car	80.99	55.54	50.94
B / Observed / F	Pedestrian	37.72	33.62	29.55
B / Observed / F	Cyclist	36.27	21.81	20.86
B / Observed / A	Car	88.20	73.67	69.72
B / Observed / A	Pedestrian	54.66	49.66	44.12
B / Observed / A	Cyclist	57.93	35.62	33.93

Table 5.13: CenterPoint BEV AP_{R40} on the final validation set.

Line	Input	Frozen	Adapted	Change	Gap
A	Original	79.19	78.18	-1.01	0.00
A	PU-GCN	60.27	64.44	+4.17	-13.74
A	PDANS	68.58	67.02	-1.57	-11.17
B	Sparse	66.62	62.91	-3.70	0.00
B	PU-GCN	32.92	37.52	+4.59	-25.40
B	PDANS	43.01	47.43	+4.43	-15.48

Table 5.14: PointRCNN adaptation screen on 256 frames: Car Moderate 3D AP at IoU 0.70 under the legacy evaluator.

Line	Input / detector weights	PointRCNN	CenterPoint
A	Original N / frozen	81.95	79.28
B	Sparse M / frozen	65.58	64.60
B	Sparse M / 3 epochs	68.33	68.05
A	Observed-first / frozen	64.27	63.21
A	Observed-first / 3 epochs	71.01	74.70
A	Observed-first / extended	71.59	75.97
B	Observed-first / frozen	35.53	44.09
B	Observed-first / 3 epochs	55.21	61.66
B	Observed-first / extended	57.05	62.90

Table 5.15: Detector-adaptation comparison: Car Moderate 3D AP_{R40} on 3,769 validation frames. Bold marks extended-schedule results, not the highest scores across input controls.

retained only epoch 3 checkpoints and recorded no epoch 1/2 validation AP. That fixed-duration study alone did not establish a validation plateau.

The extended observed-first experiments improved all four selected validation scores beyond their three-epoch counterparts, but none reached its line-specific reference (Table 5.15). The comparison separates detector adaptation on an unchanged upsampled input from comparison with the original observation. Thus the frozen observed-first rows measure the starting point for adaptation, whereas the original- N and sparse- M rows are distinct input controls. The original- N row also serves as the complete-observation reference for Line B.

Frozen denotes official detector weights. Extended denotes the validation-selected best checkpoint from the schedules in Table 5.16, with the PU-GCN checkpoint unchanged. Baseline training budgets were not extended.

Relative to the same observed-first input with frozen detector weights, the extended PointRCNN results gained 7.32 AP points in Line A and 21.52 in Line B. The corresponding CenterPoint gains were 12.76 and 18.81 points. The additional gains over the earlier three-epoch adaptation were smaller: 0.58 and 1.83 for PointRCNN, and 1.26 and 1.24 for CenterPoint. Figure 5.14 separates these recovery gains from the remaining baseline deficits, so a large improvement from a degraded starting point is not mistaken for an advantage over the original input.

For Line A, the remaining original-baseline deficits were 10.36 AP points for PointRCNN and 3.31 for CenterPoint. In Line B, the extended results remained 11.28 and 5.15 points below the adapted sparse controls. They were also below the *unadapted*

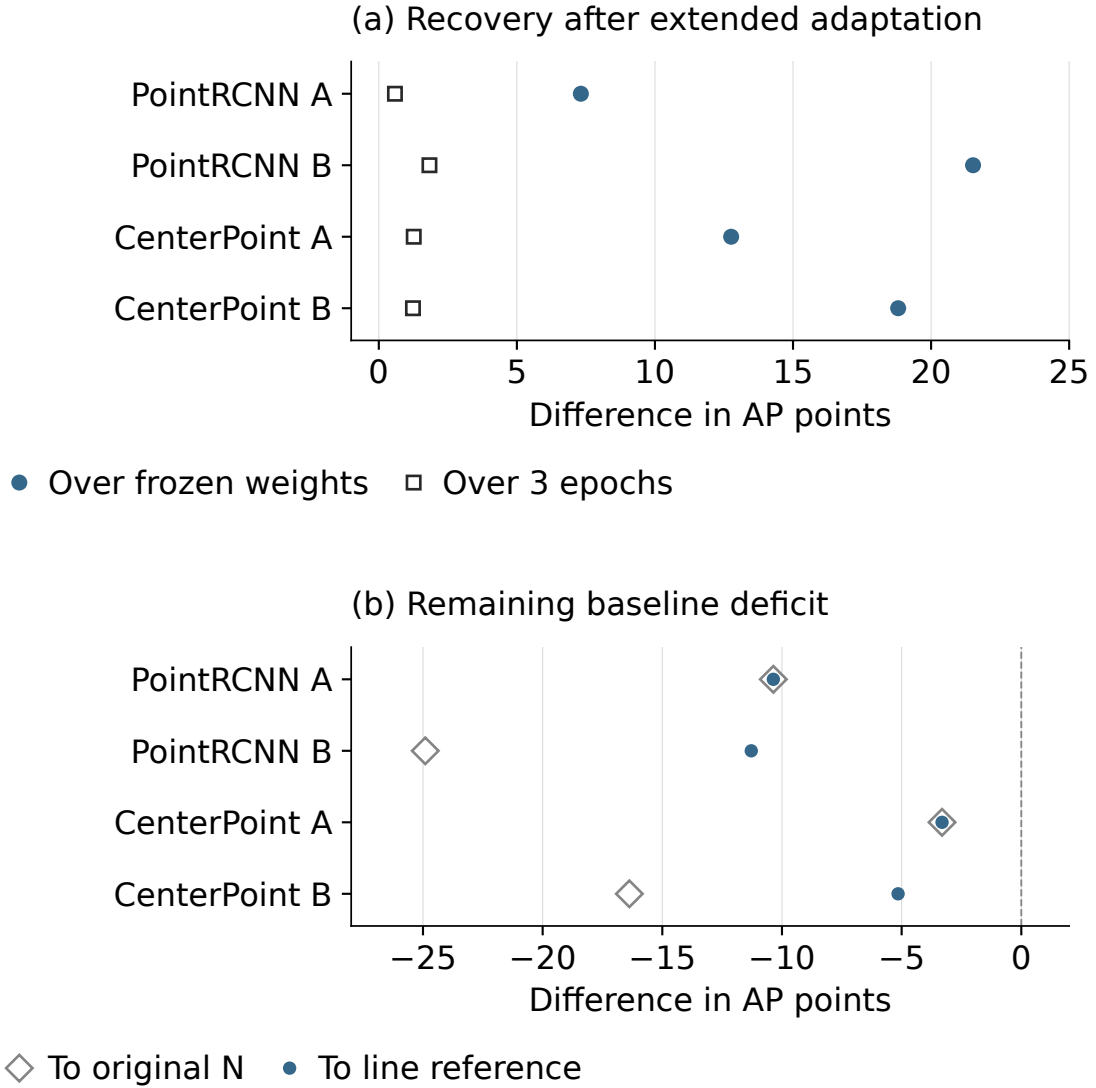


Figure 5.14: Car Moderate 3D AP_{R40} differences on 3,769 validation frames. (a) Extended-best minus frozen observed-first and minus three-epoch observed-first results. (b) Extended-best minus the original- N frozen baseline and minus the line-specific reference. The latter is the same original baseline in Line A and the three-epoch adapted sparse- M baseline in Line B; the two markers therefore coincide in Line A. Each point comes from one completed schedule and its recorded comparison, not an average across independent seeds.

sparse controls, by 8.53 and 1.70 points, respectively. Compared with the complete original observation, the Line B deficits were larger still, at 24.90 and 16.38 points.

Detector	Line	Stage	Total epochs	Best epoch	Best AP	Final count
PointRCNN	A	RPN	18	14	70.97	4
PointRCNN	A	RCNN	18	2	71.59	17
PointRCNN	B	RPN	18	14	52.76	4
PointRCNN	B	RCNN	24	19	57.05	5
CenterPoint	A	Detector	12	12	75.97	4
CenterPoint	B	Detector	12	12	62.90	3

Table 5.16: Completed schedules and selected checkpoints. Final count denotes consecutive terminal epochs without improvement exceeding the running reference by 0.2 AP points (Section 4.11).

The recovery therefore does not depend on choosing only one unfavourable starting comparator.

The stage summaries in Table 5.16 establish the scope of the plateau assessment. PointRCNN selected RPN epoch 14 in both lines; the associated RCNN selections were epoch 2 in Line A and epoch 19 in the 24-epoch Line B schedule. The four final PointRCNN trajectories contain 78 epoch evaluations in total. CenterPoint selected epoch 12 in each of its two 12-epoch schedules, adding 24 evaluations. These 102 checkpoint evaluations are repeated measurements of the completed training trajectories, not 102 independent training runs. PointRCNN RPN-stage AP belongs to its stage-specific detection evaluation and is not an additional final RCNN result.

All six terminal counts in Table 5.16 meet the threshold of three. A numerical maximum at the last epoch does not contradict this threshold-based plateau rule. CenterPoint Line A improved from the last threshold-reference score of 75.7998 at epoch 8 to 75.9661 at epoch 12; Line B improved from 62.8977 at epoch 9 to 62.9007 at epoch 12. These increases of 0.1663 and 0.0030 AP points did not reset the counter. For PointRCNN Line B, Figure 5.15 shows the six recorded terminal RCNN evaluations. The selected epoch 19 scored 57.0485, whereas epoch 24 scored 55.5902 and all five intervening or final evaluations remained below the selected value. The last checkpoint was therefore not the selected checkpoint.

This extension supplies the validation-progress evidence missing from the initial three-epoch study. It narrows the explanation for the remaining deficit: stopping after three epochs alone does not account for all of the loss under the tested schedules. It does not establish a global optimum, exclude improvement with another optimiser or input design, or remove the unequal-budget and validation-selection limits. The

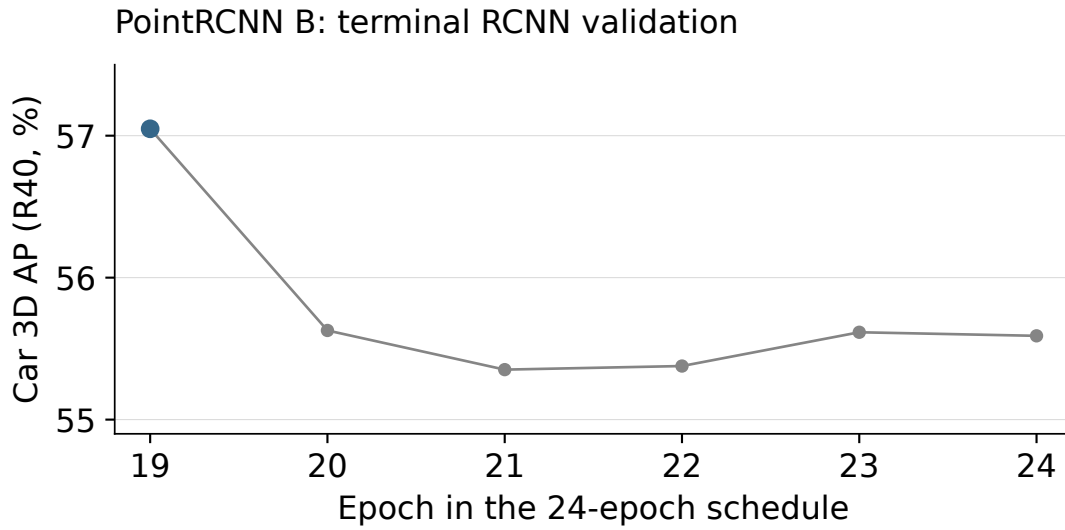


Figure 5.15: PointRCNN Line B Car Moderate 3D AP_{R40} for epochs 19–24 of the completed 24-epoch RCNN schedule, with RPN epoch 14 fixed. Blue marks the selected RCNN epoch; grey marks subsequent epochs. The panel shows the complete terminal six-epoch window tabulated in the experiment record, rather than the full training trajectory. Every point evaluates all 3,769 validation frames; the connecting segments join adjacent recorded epochs and do not estimate missing earlier values.

earlier all-class tables retain their original three-epoch checkpoints; their Pedestrian and Cyclist values are not reassigned to these Car-selected models.

5.3 KITTI Integration and Mechanism Diagnostics

5.3.1 Earlier Multi-Method Evidence

The preceding multi-method experiment is informative about integration, but it is a distinct experimental generation. Its CenterPoint evaluations covered all 3,769 validation frames under AP_{R40} . Figure 5.16 retains all three Moderate class results. PDANS gave the highest Car AP among the four generated-input pipelines in both lines, yet remained below the corresponding Car baseline. The class-specific exception matters: on Line B, PDANS Pedestrian AP was 28.18 against the sparse baseline’s 24.57, and its Cyclist AP was 15.52 against 15.46. These results rule out the stronger claim that every class always deteriorated.

Two implementation qualifications constrain that matrix. The old PU-Net adapter had a confirmed normalisation error, so its very low AP cannot be attributed to the PU-Net architecture. The lab PU-EdgeFormer path reused compatible PU-GCN operators and checkpoint inference components; it is not the independently verified native EdgeTransformer execution used for the HPC ModelNet40 branch. The historical PointRCNN E2/E3 results can be identified more precisely: their archived runner uses C++ AP_{R40} . They are therefore included below with their own paired baselines, while the legacy adaptation screen retains its different metric label.

PointRCNN E2 and E3 Full-Validation Controls

The earlier E2 protocol evaluated a deterministic 16,384-point input constructed after field-of-view and range filtering, using 0.1 m voxel representatives and depth-stratified quotas. Its four depth intervals were 0–20, 20–40, 40–60 and 60–70.4 m; largest-remainder allocation assigned quotas from candidate counts without labels or AP-based selection. E3 tested a 32,768-point, observed-first input. Both completed ten conditions on all 3,769 validation frames. The primary CSV and runner identify these as C++ AP_{R40} results.

Figure 5.17 and Tables 5.17–5.20 retain this independent comparison. E2’s original and sparse Car Moderate baselines were 82.26 and 65.75. Among generated inputs, PDANS was highest in both lines at 67.22 and 44.08. These figures describe the integrated pipelines, including the old PU-Net error and PU-EdgeFormer compatibility path; they do not rank faithful original implementations.

E3 raised dense-input PDANS from 67.22 to 70.06 and PU-GCN from 58.28 to 64.95, while the original baseline fell from 82.26 to 80.38. The stronger generated-

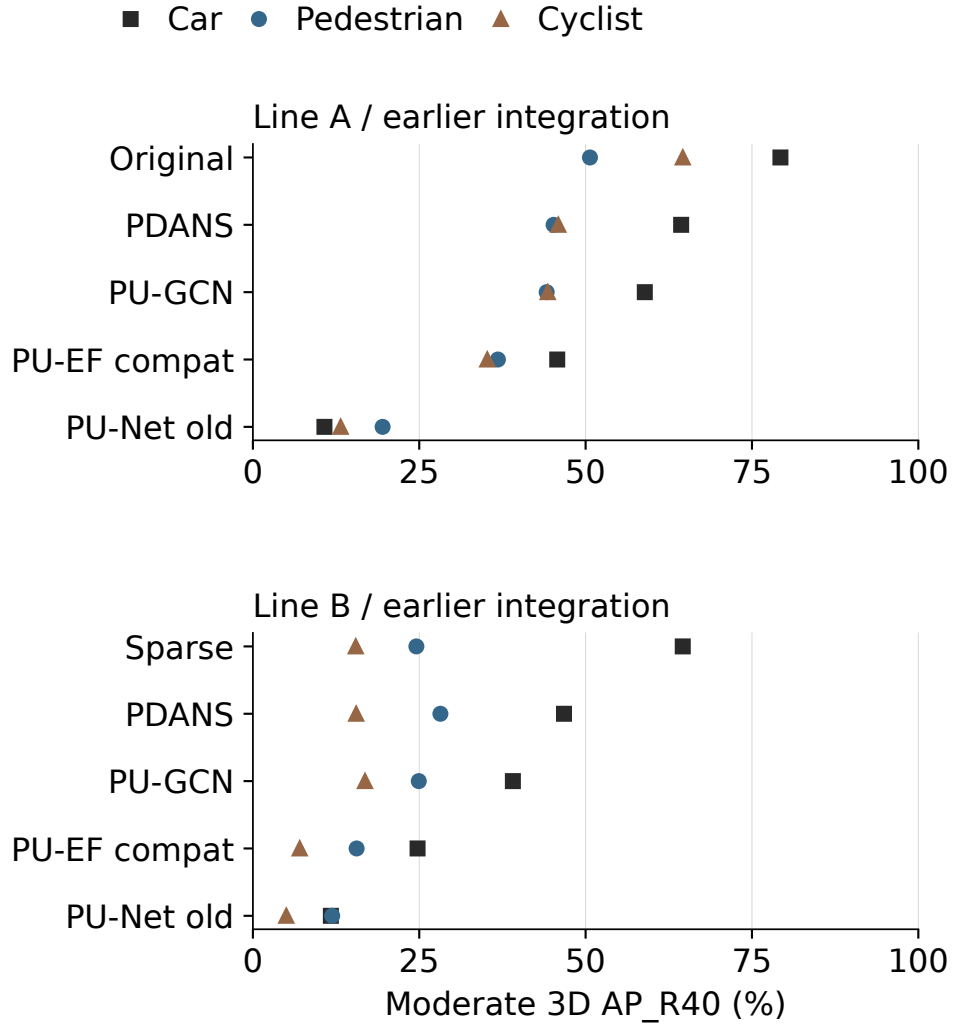


Figure 5.16: Recorded full-validation Moderate 3D AP_{R40} from the earlier integration generation. PU-Net old contains a known normalisation error; PU-EdgeFormer compat uses the lab compatibility path. These labels identify implemented pipelines and do not establish a fair ranking of the original architectures.

input recovery therefore coexisted with a changed baseline, and all Line A generated inputs remained below it. For Line B, PDANS was approximately unchanged (44.08 to 44.04), PU-GCN declined slightly (28.68 to 28.28), and the sparse baseline fell from 65.75 to 64.14. Enlarging the budget with observed-first selection did not remove the downstream deficit. Because both budget and point selection changed, these results establish sensitivity of the complete input adapter rather than a causal effect of cardinality alone.

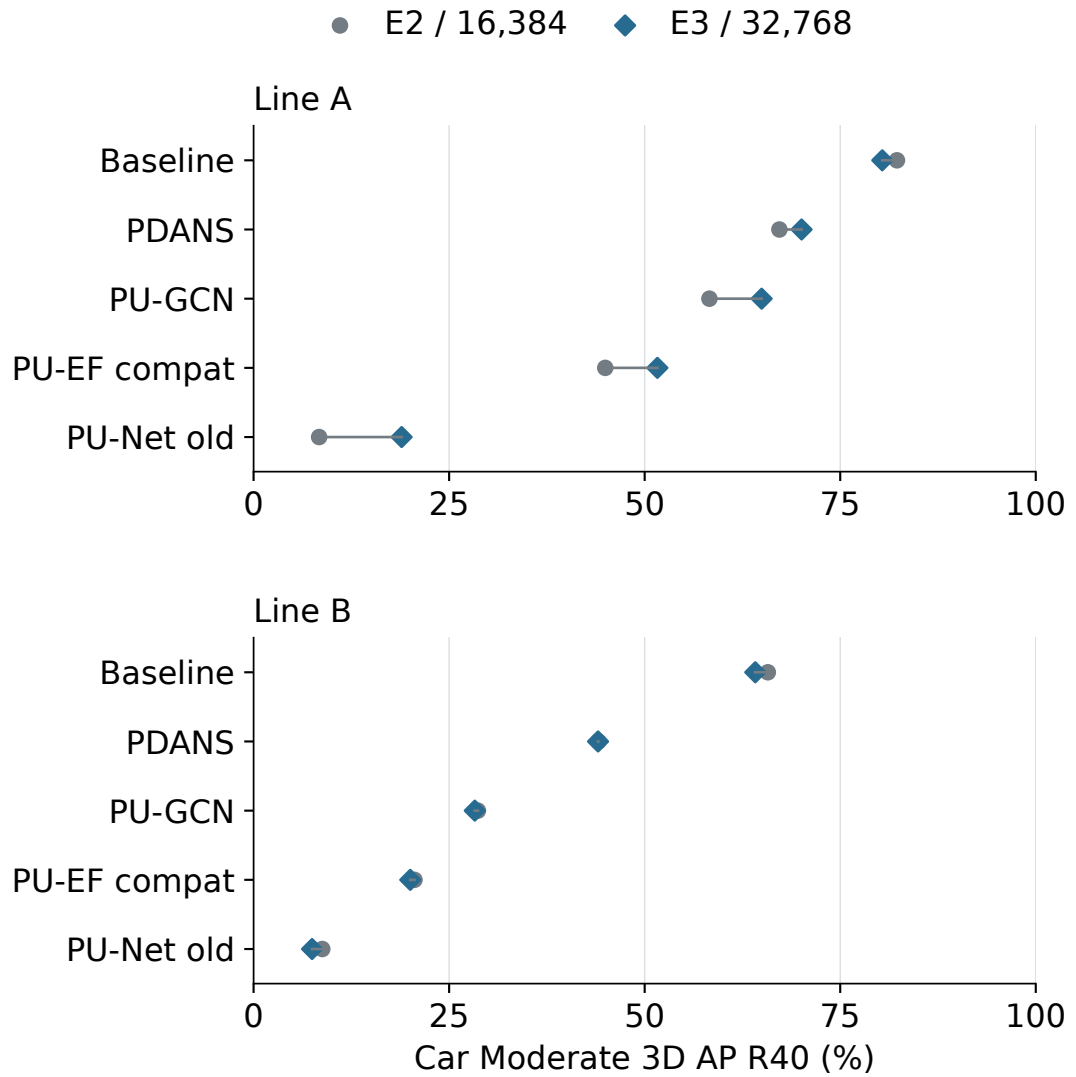


Figure 5.17: Car Moderate 3D AP_{R40} for the historical E2 (16,384-point) and E3 (32,768-point, observed-first) inputs, each evaluated on 3,769 frames. Markers share a 0–100 axis. The change combines input budget and selection policy; it is not a pure point-count effect. Baselines belong to these protocols, not the later PU-GCN adaptation matrix.

Line	Input	Easy	Moderate	Hard
A	Baseline	92.27	82.26	77.95
A	PDANS	84.61	67.22	62.40
A	PU-GCN	80.17	58.28	53.47
A	PU-EF compat	65.68	44.96	40.19
A	PU-Net old	11.76	8.38	7.44
B	Baseline	85.18	65.75	61.38
B	PDANS	65.01	44.08	39.17
B	PU-GCN	45.93	28.68	24.13
B	PU-EF compat	33.76	20.57	17.62
B	PU-Net old	12.41	8.78	7.76

Table 5.17: Historical PointRCNN E2 3D AP_{R40} for Car. Each row covers all 3,769 frames; the integrated input protocol is shared within E2. Values are percentages.

Line	Input	Easy	Moderate	Hard
A	Baseline	95.98	88.98	86.59
A	PDANS	90.00	78.02	73.21
A	PU-GCN	86.93	66.79	62.02
A	PU-EF compat	75.97	53.83	48.94
A	PU-Net old	28.95	18.74	16.85
B	Baseline	91.04	76.40	72.03
B	PDANS	76.63	54.57	48.14
B	PU-GCN	57.25	36.19	31.43
B	PU-EF compat	43.65	27.74	23.38
B	PU-Net old	23.77	15.52	13.62

Table 5.18: Historical PointRCNN E2 BEV AP_{R40} for Car. Each row covers all 3,769 frames; the integrated input protocol is shared within E2. Values are percentages.

Line	Input	Easy	Moderate	Hard
A	Baseline	99.32	94.08	90.10
A	PDANS	93.32	81.75	76.85
A	PU-GCN	92.44	71.77	67.07
A	PU-EF compat	81.95	58.80	52.30
A	PU-Net old	38.46	23.89	20.69
B	Baseline	95.24	79.95	75.58
B	PDANS	83.75	60.33	53.79
B	PU-GCN	60.61	38.61	32.40
B	PU-EF compat	52.75	33.49	28.71
B	PU-Net old	33.62	20.51	18.40

Table 5.19: Historical PointRCNN E2 image-box AP_{R40} for Car. Each row covers all 3,769 frames; the integrated input protocol is shared within E2. Values are percentages.

Line	Input	Easy	Moderate	Hard	Δ Mod.
A	Baseline	92.03	80.38	77.52	-1.88
A	PDANS	86.27	70.06	65.27	+2.84
A	PU-GCN	85.19	64.95	58.66	+6.68
A	PU-EF compat	71.54	51.63	45.65	+6.67
A	PU-Net old	27.92	18.93	17.88	+10.55
B	Baseline	84.59	64.14	57.50	-1.61
B	PDANS	64.92	44.04	39.16	-0.04
B	PU-GCN	45.09	28.28	23.83	-0.40
B	PU-EF compat	32.50	20.03	16.20	-0.54
B	PU-Net old	10.99	7.48	7.19	-1.31

Table 5.20: Historical PointRCNN E3 Car 3D AP_{R40} . Δ Mod. is E3 minus E2 for the same input name, in AP points. All ten rows have 3,769 prediction files.

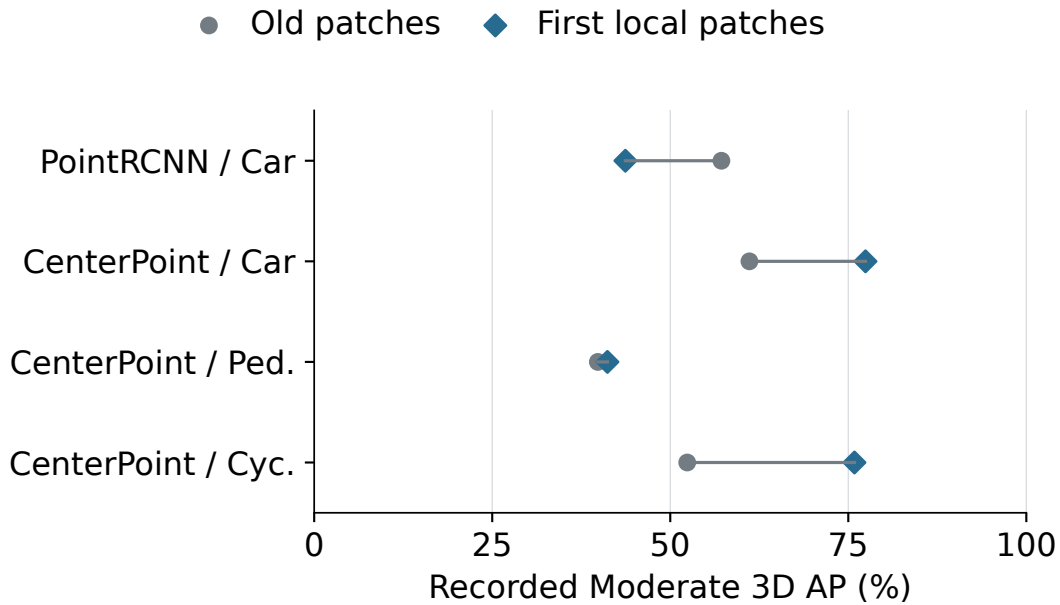


Figure 5.18: Paired Moderate 3D AP from the archived 256-frame PDANS patch experiment. The source report labels these values AP_{R40} ; they retain that report’s protocol and are not inserted into the final adaptation matrix. Each segment connects the old and first local patch result for one detector and class. The CenterPoint values use the original evaluation logs named in the paired report, which differ slightly from an older summary CSV.

5.3.2 Patch Locality and Normalisation

The original patch extractor could combine distant structures because it partitioned coarse spatial bins by row order. In the audited Line B patches, the median XY diagonal was 30.46 m and its 90th percentile was 124.10 m. Such supports do not resemble the local surface patches used to train object upsamplers [4, 5]. A first local replacement reduced the spatial extent but introduced 59.4% duplicate rows. Locality and unique support therefore had to be checked separately.

The first local PDANS replacement produced opposite detector responses (Figure 5.18). On the same 256 frames, PointRCNN Car Moderate 3D AP fell from 57.18 to 43.70, while CenterPoint Car rose from 61.12 to 77.42. The saved paired report also records CenterPoint Pedestrian and Cyclist increases. Its locality rule guaranteed only 256 unique supporting points for a 2,048-row patch; repeated filling and overlapping patches concentrated the generated output. This was a development configuration, distinct from the later unique-2,048-support PU-GCN pipeline.

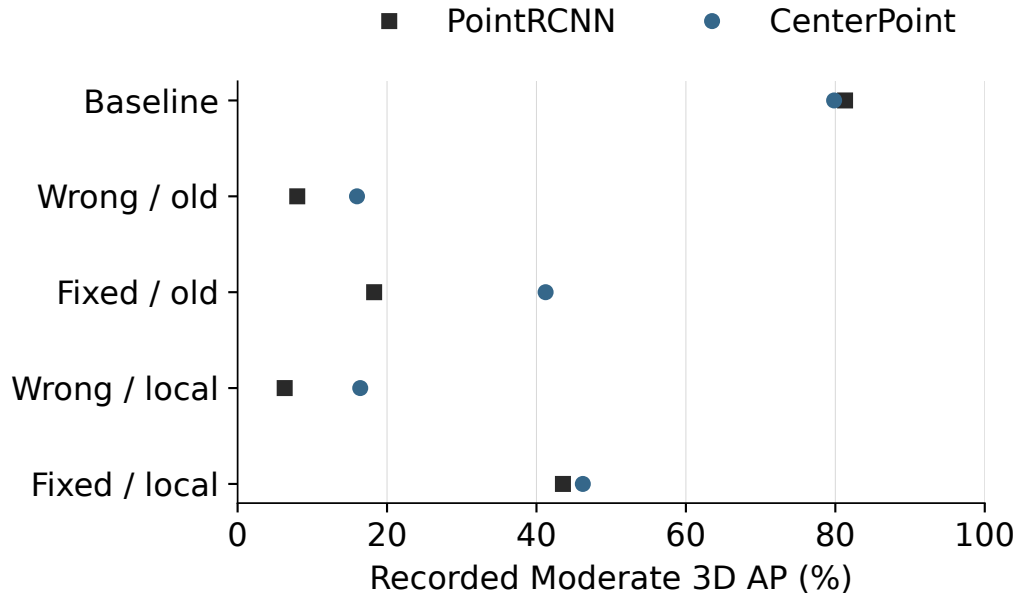


Figure 5.19: Recorded 256-frame PU-Net diagnostic: wrong or fixed normalisation crossed with old or local patches. Values are compared only within this experiment. The historical PointRCNN metric is not promoted to the final AP_{R40} series.

The retained input audit supports a concentration explanation. Across 20 frames, the median fraction of PointRCNN input coordinates unique at 1 mm fell from 99.90% to 57.70%; occupied diagnostic voxels fell from 11,584.5 to 2,124.5, and the share of points in the densest 10% of voxels rose from 27.23% to 87.85%. CenterPoint preserved the original measured points before voxelisation, whereas PointRCNN sampled 16,384 input rows. The Car matching audit changed from 539 to 402 matches for PointRCNN and from 621 to 748 for CenterPoint. These counts corroborate the opposite observed changes, but do not replace AP or isolate a single causal operation: locality, duplication and effective support changed together.

A 256-frame PU-Net ablation isolated normalisation and patch construction as two factors (Figure 5.19). Fixing normalisation with the old patch improved PointRCNN Moderate AP from 7.98 to 18.27 and CenterPoint from 15.98 to 41.22. Fixing both factors raised the values to 43.54 and 46.20. The paired baselines were 81.30 and 79.84. Both faults affected performance, but correcting them was insufficient to recover the baselines. Because this was a diagnostic subset and an earlier evaluator configuration, its AP values are not inserted into the final full-validation ranking.

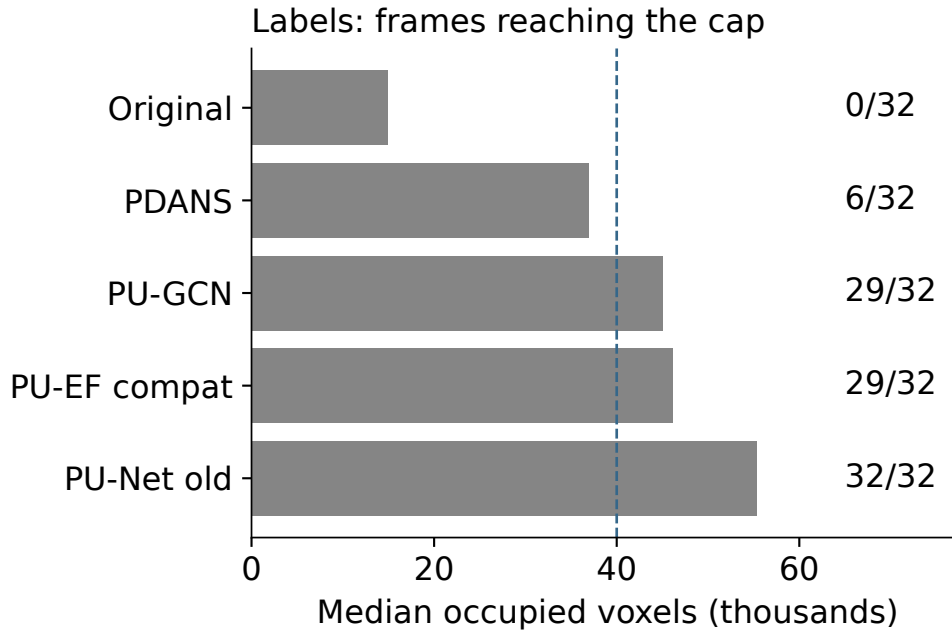


Figure 5.20: Median occupied voxels and numbers of cap-triggering frames in the archived 32-frame Line A audit. The dashed line is the 40,000-voxel evaluation cap. These are diagnostic counts from the earlier input generation, not statistics of the final 3,769-frame adaptation matrix.

5.3.3 The Effective Detector Input

The input-budget audits show why exact file cardinality is insufficient to describe the detector’s observation. PointRCNN samples a fixed 16,384 points at its entrance. CenterPoint limits points per voxel and the number of retained voxels. In a 32-frame Line A audit, median occupied-voxel counts were 14,944 for Original, 36,913 for PDANS, 45,108 for PU-GCN, 46,182 for the PU-EdgeFormer compatibility path, and 55,384 for old PU-Net. The 40,000-voxel cap was reached in 0, 6, 29, 29, and 32 frames, respectively (Figure 5.20).

Line B did not trigger that cap in the audit, yet its detection losses were large. Voxel truncation is therefore a relevant Line A mechanism, but cannot be the only explanation. Similarly, a separate full-input PointRCNN experiment that removed fixed entrance resampling reduced even the original-input Moderate result from approximately 81.95 to 74.91 within its own protocol. Simply presenting more points changed the input distribution expected by the pretrained detector.

The region-split pilot tested a further alternative. It processed 2,067 shared core regions with a 3 m halo, preserved all core points, and kept individual detector inputs within 16,384 points. Under that three-class PointRCNN configuration, baseline Car/Pedestrian/Cyclist Moderate AP_{R40} was 77.82/58.21/78.29; the closest tested PDANS branch reached 68.87/58.07/71.58. Avoiding core-point truncation did not produce a consistent gain. This diagnostic uses a separate detector implementation and inference protocol, so it cannot quantify the effect of truncation in the final native PointRCNN experiment.

5.3.4 Selection, Sampling Variance, and Negative Controls

A retained 256-frame PDANS experiment tested a small generated-point dose of 2.5% against a matched observed-point fill control. Car Moderate 3D AP was 82.6611 for the baseline, 84.4160 for the observed-fill control, and 85.0953 for PDANS. The apparent 2.4342-point gain over baseline therefore consisted of a 1.7548-point control improvement and only a 0.6793-point additional difference from the generated representation. This attribution prevents the whole gain from being assigned to generated geometry.

Table 5.21 retains the other metrics from the same comparison. The sign was not uniform across difficulty or projection: the PDANS variant reduced Hard 3D AP and Moderate image-box AP. This small-dose result was a diagnostic candidate rather than evidence of an established full-validation gain. Values retain the historical evaluator’s metric definitions and are not pooled with the final adaptation matrix. AOS denotes KITTI’s average orientation similarity [2].

The executed detector-aware PDANS sequence tested whether generated points could be restricted to more useful regions. The V4 measured-voxel anchor reduced the median number of newly activated voxels to zero and gave a 0.31-point Car gain in its PointRCNN pilot. CenterPoint Car was approximately unchanged, while Pedestrian and Cyclist still declined. The V5 class-adaptive gate nearly preserved all CenterPoint pilot metrics, but its independent 64-frame holdout changed Car Moderate AP from 74.1881 to 74.0512. These tests support controlled occupancy as an integration factor; they did not establish a repeatable cross-detector gain.

A 16-seed evaluator probe further qualified a positive single-run result. With input, checkpoint, split, and merge rules fixed, changing only the sampling seed gave Car Moderate scores (mean $\pm$ standard deviation across seeds) of 80.998 ± 1.198 for the baseline and 80.560 ± 1.086 for the V1 full variant. The paired difference was -0.438 ± 1.769 AP points, and 9 of 16 differences were negative. The previously reported 1.2227-

Metric	Difficulty	Baseline	Obs. fill	PDANS	Δ vs fill
BBox	Easy	99.37	99.49	99.22	-0.27
BBox	Moderate	94.12	93.93	92.31	-1.62
BBox	Hard	89.59	89.48	89.53	+0.05
BEV	Easy	96.45	96.53	96.43	-0.10
BEV	Moderate	90.91	90.83	91.08	+0.25
BEV	Hard	88.44	86.36	86.37	+0.00
3D	Easy	95.09	94.80	95.44	+0.64
3D	Moderate	82.66	84.42	85.10	+0.68
3D	Hard	79.93	79.82	78.63	-1.20
AOS	Easy	99.36	99.48	99.22	-0.27
AOS	Moderate	94.01	93.80	92.19	-1.61
AOS	Hard	89.44	89.33	89.34	+0.02

Table 5.21: Low-dose comparison on 256 frames. The last column is PDANS minus observed-fill control, in percentage points.

point gain was the variant’s most favourable seed in that probe. These are evaluator-seed statistics for the diagnostic path, not training-seed uncertainty and not an error bar applicable to the final full-validation matrix.

Frame-Subset Size and AP Stability

A separate completed probe varied which validation frames were scored, while keeping saved detector predictions fixed. It drew 200 subsets without replacement at each size of 20, 50, 100, 256 and 512 frames, using seed 0 and paired frame identities for the original, sparse and sparse-PDANS inputs. Car Moderate 3D AP_{R40} was recomputed from the saved predictions. This is variation from frame selection, distinct from the preceding 16-seed detector-input sampling probe and from retraining variability.

Figure 5.21 and Table 5.22 show the standard deviation of each paired AP contrast. For the sparse minus original contrast, it fell from 6.02 points at 20 frames to 1.26 at 512 frames. For PDANS minus sparse, the standard deviation was 4.78 at 50 frames and 2.27 at 256 frames. Quoting the former contrast’s variability as uncertainty of the latter would misstate the experiment.

At 20 frames, the original-input AP’s empirical 5th–95th percentile range was 51.96–89.95, despite fixed model predictions. The mean was 75.63 compared with 81.99 on the complete source set. Small-subset AP therefore changed both variability and average value, reflecting the composition and recall support of the selected frames. This is a reason to distinguish case diagnosis from full-validation evaluation. The archived probe

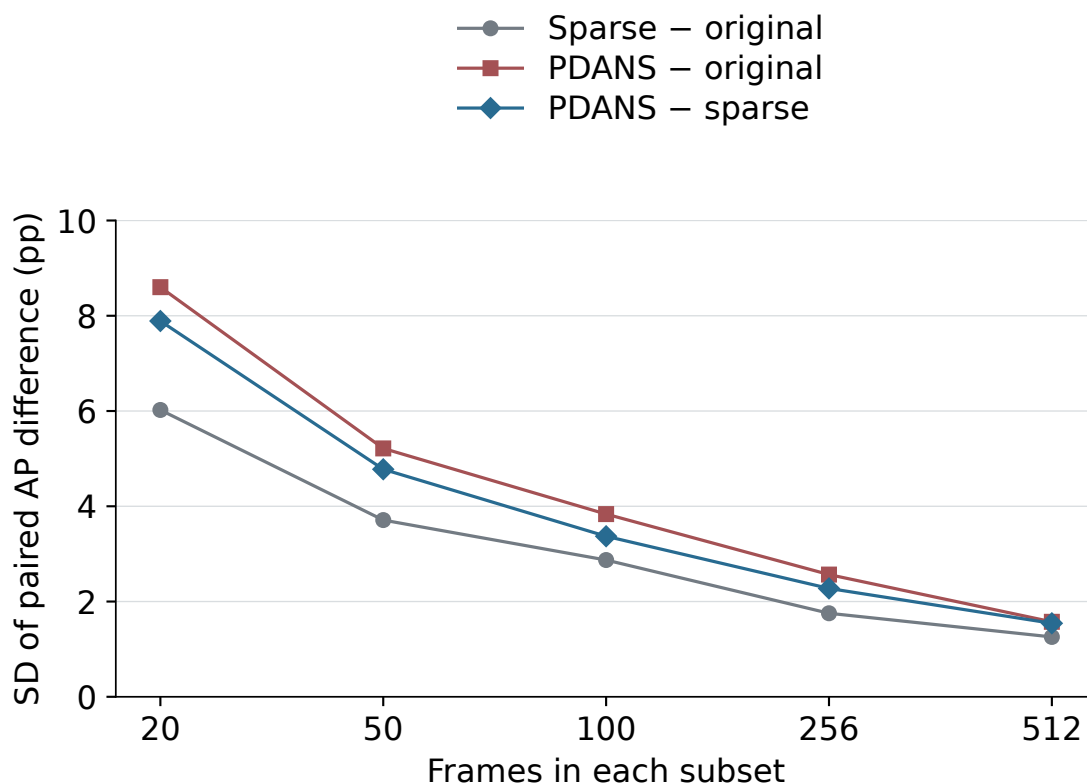


Figure 5.21: Standard deviation across 200 random frame subsets per size, calculated from fixed historical PointRCNN predictions. Each curve is a different paired Car Moderate 3D AP_{R40} contrast. These are observed subsampling standard deviations, not confidence intervals for the final matrix and not results of additional detector training.

Frames	Sparse – original	PDANS – original	PDANS – sparse
20	6.02	8.60	7.89
50	3.71	5.21	4.78
100	2.87	3.84	3.37
256	1.75	2.57	2.27
512	1.26	1.57	1.54

Table 5.22: Sample standard deviations of paired Car Moderate AP contrasts in AP points. All three contrasts use the same sampled frames within a draw.

also multiplies standard deviation by 1.96 under a sensitivity label. That number is only a descriptive scale here; it is not a formal minimum detectable effect established by a power analysis. No repeated-training uncertainty or extrapolated full-set confidence interval is inferred from this probe.

Input	Easy	Moderate	Hard
Original / standard	92.26	81.95	77.75
Original / all-valid	79.83	74.91	72.73
PDANS / all-valid	83.59	63.49	54.34
PU-GCN / all-valid	78.21	54.26	47.11
PU-EF / all-valid	67.11	43.85	37.10
PU-Net old / all-valid	12.23	8.82	8.17

Table 5.23: Recorded full-validation all-valid-input experiment. Values are Car 3D AP from that historical protocol. PU-EF is the compatibility path and PU-Net old retains the faulty adapter.

5.3.5 Completed Input and Selection Interventions

The completed interventions changed input capacity, patch support, generated-point dose, and selection rules. Their separate baselines and evaluation scopes are retained below. Differences between tables are not interpreted as isolated method effects when the pipelines differ.

Removing Entrance Resampling

The all-valid PointRCNN experiment evaluated the original cloud and four generated Line A inputs without standard entrance resampling. Table 5.23 shows that the original-input Moderate result itself fell from about 81.95 to 74.91. PDANS was the strongest generated input at 63.49, but still trailed the all-valid original. Removing a point cap changed the representation expected by the detector rather than simply releasing a performance bottleneck.

Correcting Local Patch Support across Methods

Table 5.24 retains all four paired surface-patch pilots and their two batch baselines. Every method improved with surface-c32. Fixed-normalisation PU-Net additionally reached 44.0985 with cover-kNN, against its batch baseline of 81.7441. Local support therefore affected the recorded performance, but did not close the baseline gap.

Region Splitting with Complete Core-Point Retention

The region-split pilot retained every core point across 2,067 shared regions with a 3 m halo and at most 16,384 points per detector input. Table 5.25 reports every method

Batch	Method	Baseline	Old patch	Surface
Batch 1	PDANS	81.56	57.99	68.47
Batch 1	PU-GCN	81.56	55.81	63.45
Batch 2	PU-EF	81.74	41.74	58.02
Batch 2	PU-Net fixed	81.74	16.81	22.93

Table 5.24: PointRCNN 256-frame paired patch pilots, Car Moderate 3D AP. Surface denotes surface-c32; the two pilot batches retain separate baselines.

Input	Car	Pedestrian	Cyclist
Baseline	77.82	58.21	78.29
PDANS	68.87	58.07	71.58
PU-GCN	61.51	55.47	57.48
PU-EF	61.28	56.14	56.18
PU-Net fixed	26.18	25.02	28.92

Table 5.25: Region-split pilot on 256 frames: Moderate 3D AP_{R40}. Generated rows use surface-c32; PU-Net normalisation is fixed. This is a separate three-class PointRCNN implementation and inference protocol.

and class. PDANS was closest to the baseline, but no generated input improved the three principal class outcomes together.

An independent 64-frame region-split V4 holdout improved Car/Pedestrian/Cyclist from 75.7267/38.1070/15.0000 to 77.4777/45.8565/17.2222. These positive results remain part of the completed evidence, bounded by the small holdout and its region-split protocol. In native whole-frame OpenPCDet PointRCNN, V4 changed Car by +4.0651 under deterministic workers=0 but −2.2718 under workers=2, with other class effects also changing sign. The records therefore do not establish improvement stable across those sampling implementations.

Dose and Selector Components

The 256-frame PDANS Line A dose sequence used 2.5%, 5%, 7.5%, and 10% generated points. Its reported Moderate results were approximately 85.10, 82.37, 82.25, and 82.42, against 82.66. All tested Line B doses remained below the sparse reference. Full-validation g10 runs completed for all four methods: Line A PDANS/PU-GCN/PU-EF/PU-Net old reached approximately 79.951/79.478/76.433/72.371, and Line B reached 59.048/55.954/52.415/45.700. The g25 and g50 runs did not complete and are not treated as evaluated results.

Configuration	Moderate
Baseline	81.18
Density-only	79.86
Fixed 2.5% PDANS	81.44
Full	82.40
Matched-real dynamic	81.80
NN-only	81.59
No-adaptive	80.15
No-confidence	81.64
No-sparse	80.94

Table 5.26: PointRCNN V1 selector/component pilot on 256 frames, Car Moderate 3D AP_{R40} . The separate 16-seed probe qualifies these single-evaluation scores.

Version	PR Car	CP Car	CP Ped.	CP Cyc.
Baseline	81.18	79.84	43.51	76.11
V2 full	79.23	76.77	48.56	72.28
V3 full	79.29	76.74	48.59	72.29
V4	81.49	79.82	43.07	75.77

Table 5.27: Completed 256-frame PDANS versions: PointRCNN Car and CenterPoint Car/Pedestrian/Cyclist Moderate 3D AP. CenterPoint Pedestrian/Cyclist baselines are obtained from the recorded scores and baseline deltas.

The V1 component experiment compared density, fixed dose, matched-real, nearest-neighbour, and disabled-component variants (Table 5.26). The full variant had the highest single-evaluation score, but the matched-real control also improved. The later evaluator-seed probe described above shows why this one favourable score did not establish a stable gain.

Detector-Aware Selection Versions

V2 used final-box guidance, V3 used internal pre-RCNN or pre-NMS proposals, V4 anchored measured voxels, and V5 introduced class-adaptive gating. PointRCNN V2 voxel-only and proposal-only scores were 79.3805 and 75.7533; their combined score was 79.2269. Internal proposals in V3 yielded 79.2933, so that replacement did not remove the loss. Table 5.27 retains the detector and class comparisons.

V4 reduced the median number of newly activated voxels to zero, yet did not improve all classes. V5 changed development-set CenterPoint Car/Pedestrian/Cyclist by $-0.0184/ + 0.0004/ + 0.0000$ and reused the PointRCNN V4 result. The completed

Protocol	Base	PDANS	PU-GCN	PU-EF	PU-Net
PointRCNN c2048-r4	64.04	36.62	24.96	21.24	2.40
PointRCNN E1	64.04	44.05	32.84	28.26	10.58
PointRCNN consensus	63.42	39.25	33.16	22.44	18.88
CenterPoint random	61.66	47.74	41.38	35.43	15.15
CenterPoint consensus	61.66	43.61	37.31	27.33	19.80

Table 5.28: Line B 256-frame selection pilots, Car Moderate 3D AP. PU-EF is the compatibility implementation and PU-Net uses fixed normalisation. Protocol-specific baselines are not pooled.

independent 64-frame holdout did not retain a positive Car effect: 74.1881 became 74.0512, while BEV fell from 86.3893 to 85.2910. Input control recovered performance during development; the holdout limits the claim of a general benefit.

Line B Selection and Small Negative Probes

Table 5.28 compares the completed c2048-r4, E1, consensus, and random pilots. For PointRCNN, PDANS favoured E1, whereas PU-GCN and fixed PU-Net had their highest listed score with consensus. For CenterPoint, consensus reduced PDANS, PU-GCN, and PU-EF relative to random selection but improved PU-Net. No listed generated input reached its corresponding baseline.

A quality-selection probe used three frames without a voxel quota and five frames with quota 6. Without a quota, reported quality precision was 0.6821 versus 0.8155 for random selection, and unsupported fraction was 0.5825 versus 0.4940. With quota 6, the comparisons were 0.5965 versus 0.8024 and 0.6329 versus 0.5337. These small geometric probes had no detector AP and were not expanded. Their negative outcomes qualify the selection hypothesis, while saying nothing directly about detector performance.

Together, these interventions tested capacity, local support, dose, proposal guidance, measured-voxel anchoring, and selector components. Several recovered performance within their own protocols, but no consistent cross-protocol gain survived the available controls. They motivated the later PU-GCN input construction and detector adaptation, whose final full-validation results remain separate.

5.3.6 Object-Level Losses and Recoveries across the Validation Set

The historical lab audit retained individual object matches across the 3,769-frame validation split. This evidence connects the aggregate degradation to concrete losses and recoveries. It belongs to the earlier input-preserving integration generation, with the old PU-Net normalisation defect and the PU-EdgeFormer compatibility path. It is therefore analysed within that generation, separately from the final PU-GCN adaptation matrix.

Each object was greedily matched to a same-class prediction by oriented 3D IoU. Car required 0.70, while Pedestrian and Cyclist required 0.50. A lost object had a qualifying baseline match and none after upsampling; a recovered object had the reverse transition. Maintained, confidence-degraded, and localisation-degraded objects remained matched. The audit does not reproduce the official difficulty filters, DontCare treatment, or score-threshold integration. Its rates describe these retained matches and are not AP or official difficulty-specific recall.

For PointRCNN Line A, PU-GCN lost 3,597 baseline-matched Cars and recovered 210, giving a net change of $-3,387$ matched objects. The loss rate was 32.65% of 11,016 baseline matches. For Line B, it lost 4,829 and recovered 115: 55.37% of the 8,721 baseline matches were lost. PDANS had a smaller loss in both lines, but its recoveries also failed to offset its losses. Figure 5.22 makes both directions visible, so occasional improvements are not concealed by a negative aggregate.

Figure 5.23 shows the corresponding CenterPoint counts. For Line A, PU-GCN lost 2,932 Cars and recovered 340; Line B lost 3,505 and recovered 333. Its loss rates of 25.88% and 37.43% were lower than PointRCNN's in the corresponding historical comparison, but both remained negative in net matches. The two detectors thus share a qualitative failure pattern while differing in its magnitude. This does not isolate architecture as the cause: their checkpoints, preprocessing, and native inputs also differ.

Table 5.29 retains every class and method row from this audit. PointRCNN is Car-only in this configuration; CenterPoint also supplies Pedestrian and Cyclist. Different class thresholds and baseline match counts are kept explicit. A recovery count is not normalised by baseline TP: its natural denominator is baseline FN. Loss and recovery percentages therefore cannot be subtracted directly. The net count equals recovered minus lost.

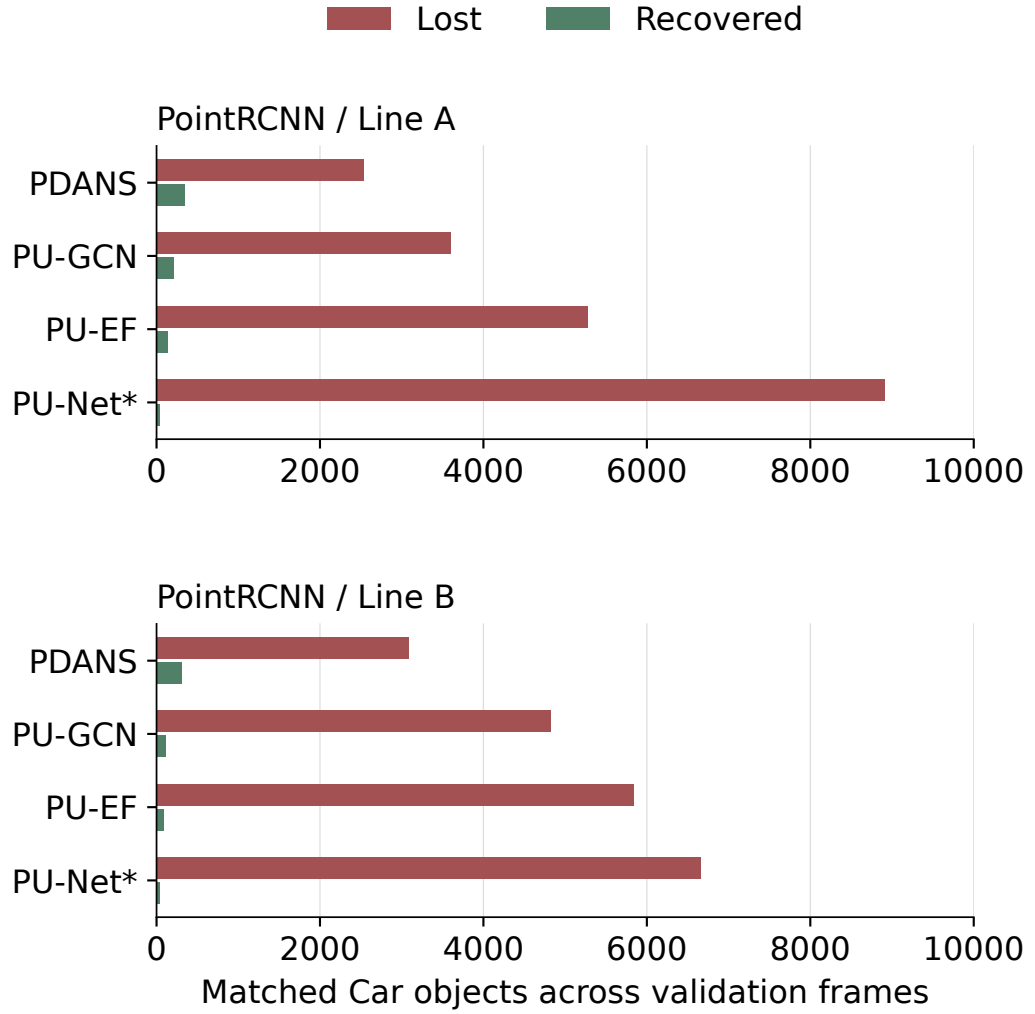


Figure 5.22: Historical full-validation Car matching audit. Lost and recovered counts use the same 14,385 Car opportunities within each comparison. PU-Net* denotes the historical faulty adapter; PU-EF denotes the lab compatibility path. These are object-event counts, not AP.

Table 5.29: Complete retained transition summary. B is baseline matched count, L is lost, R is recovered, and $\Delta = R - L$. PU-EF is the compatibility path and PU-Net* the faulty historical adapter.

Line	Method	Class	B	L	R	Δ
<i>PointRCNN</i>						
A	PDANS	Car	11,016	2,544	353	-2,191
A	PU-EF	Car	11,016	5,279	143	-5,136
A	PU-GCN	Car	11,016	3,597	210	-3,387
A	PU-Net*	Car	11,016	8,909	39	-8,870

Line	Method	Class	B	L	R	Δ
B	PDANS	Car	8,721	3,091	317	-2,774
B	PU-EF	Car	8,721	5,845	94	-5,751
B	PU-GCN	Car	8,721	4,829	115	-4,714
B	PU-Net*	Car	8,721	6,668	45	-6,623
<i>CenterPoint</i>						
A	PDANS	Car	11,329	2,330	424	-1,906
A	PDANS	Cyclist	595	113	58	-55
A	PDANS	Pedestrian	1,568	277	117	-160
A	PU-EF	Car	11,329	4,447	255	-4,192
A	PU-EF	Cyclist	595	121	67	-54
A	PU-EF	Pedestrian	1,568	538	112	-426
A	PU-GCN	Car	11,329	2,932	340	-2,592
A	PU-GCN	Cyclist	595	120	75	-45
A	PU-GCN	Pedestrian	1,568	340	113	-227
A	PU-Net*	Car	11,329	8,016	98	-7,918
A	PU-Net*	Cyclist	595	268	34	-234
A	PU-Net*	Pedestrian	1,568	755	107	-648
B	PDANS	Car	9,364	2,587	493	-2,094
B	PDANS	Cyclist	272	100	102	+2
B	PDANS	Pedestrian	1,060	302	268	-34
B	PU-EF	Car	9,364	5,175	227	-4,948
B	PU-EF	Cyclist	272	123	77	-46
B	PU-EF	Pedestrian	1,060	575	172	-403
B	PU-GCN	Car	9,364	3,505	333	-3,172
B	PU-GCN	Cyclist	272	81	94	+13
B	PU-GCN	Pedestrian	1,060	372	203	-169
B	PU-Net*	Car	9,364	6,274	148	-6,126
B	PU-Net*	Cyclist	272	154	40	-114
B	PU-Net*	Pedestrian	1,060	608	148	-460

5.3.7 Where Matched Objects Are Lost

Distance stratification asks whether the failure is concentrated in sparser, more distant observations. The archived audit partitions objects into planar ranges of 0–20, 20–40, and at least 40 m. Figures 5.24 and 5.25 retain all three bins and all four pipelines. Each point is lost baseline matches divided by baseline matches in that same bin, not a fraction of all objects or all detections.

For PointRCNN Line A, PU-GCN lost 9.5%, 44.6%, and 81.6% of baseline-matched Cars in these three bins, respectively. For PointRCNN Line B, PU-GCN lost 31.2%, 86.8%, and 95.4% of baseline-matched Cars in these three bins, respectively. For

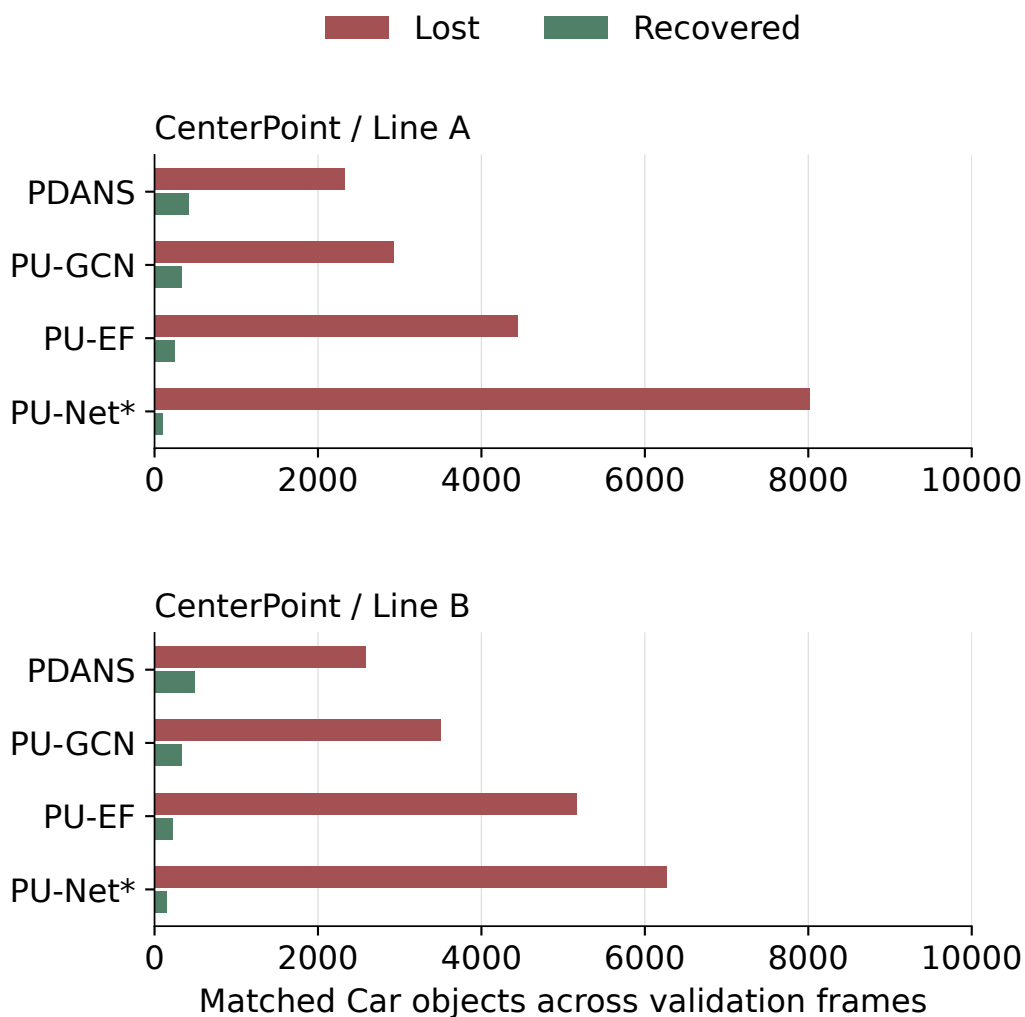


Figure 5.23: Historical full-validation Car matching counts. The comparison retains both loss and recovery events for every implemented pipeline. Definitions and implementation qualifications match Figure 5.22.

CenterPoint Line A, PU-GCN lost 7.2%, 33.0%, and 69.2% of baseline-matched Cars in these three bins, respectively. For CenterPoint Line B, PU-GCN lost 15.3%, 58.8%, and 84.3% of baseline-matched Cars in these three bins, respectively.

The distance curves locate the degradation but do not establish distance as its sole cause. Occlusion, object size, pose, visible surfaces, and detector confidence also vary across range bins. The conditional denominator additionally excludes objects already missed by the baseline.

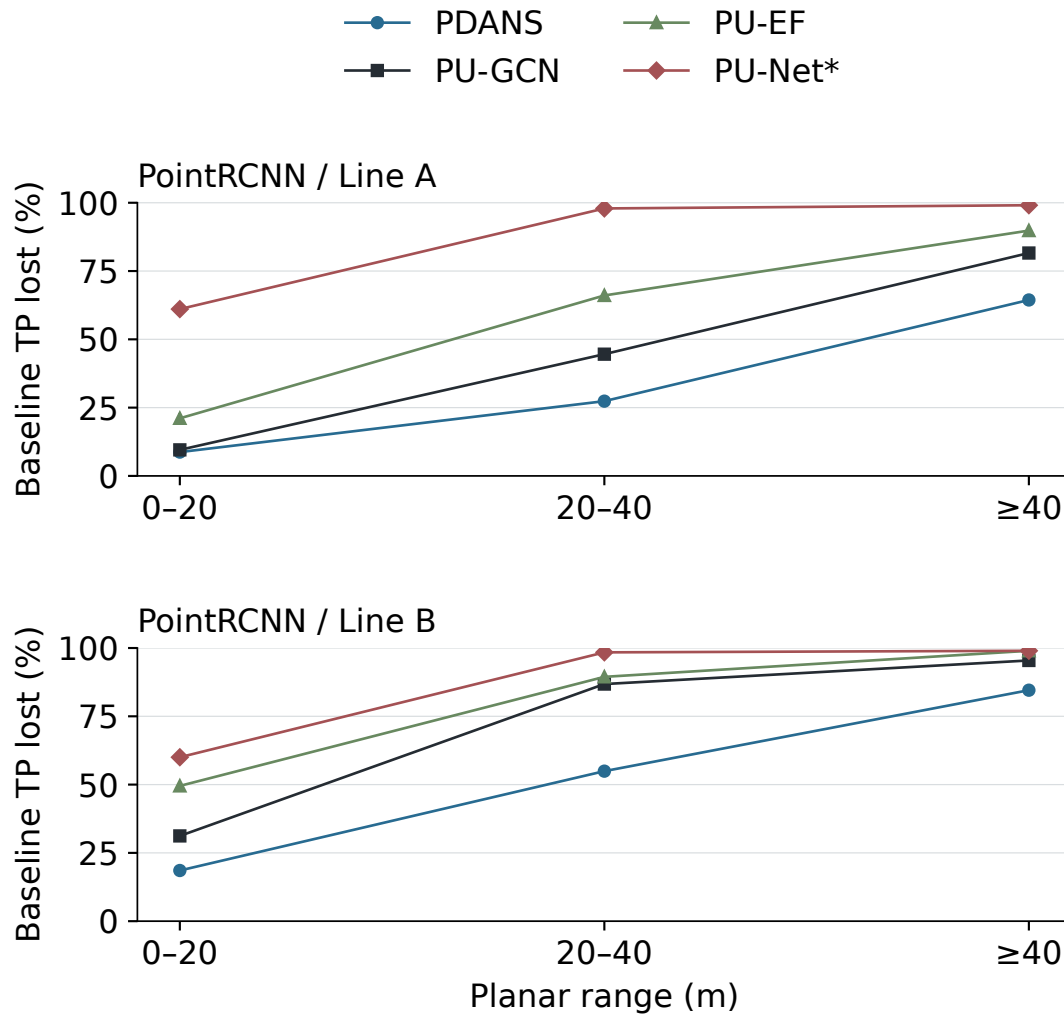


Figure 5.24: Historical Car loss rates conditional on a qualifying baseline match within each planar-distance bin. Lines connect the three archived strata; they do not interpolate unmeasured performance. PU-Net* and PU-EF retain the historical implementation qualifications.

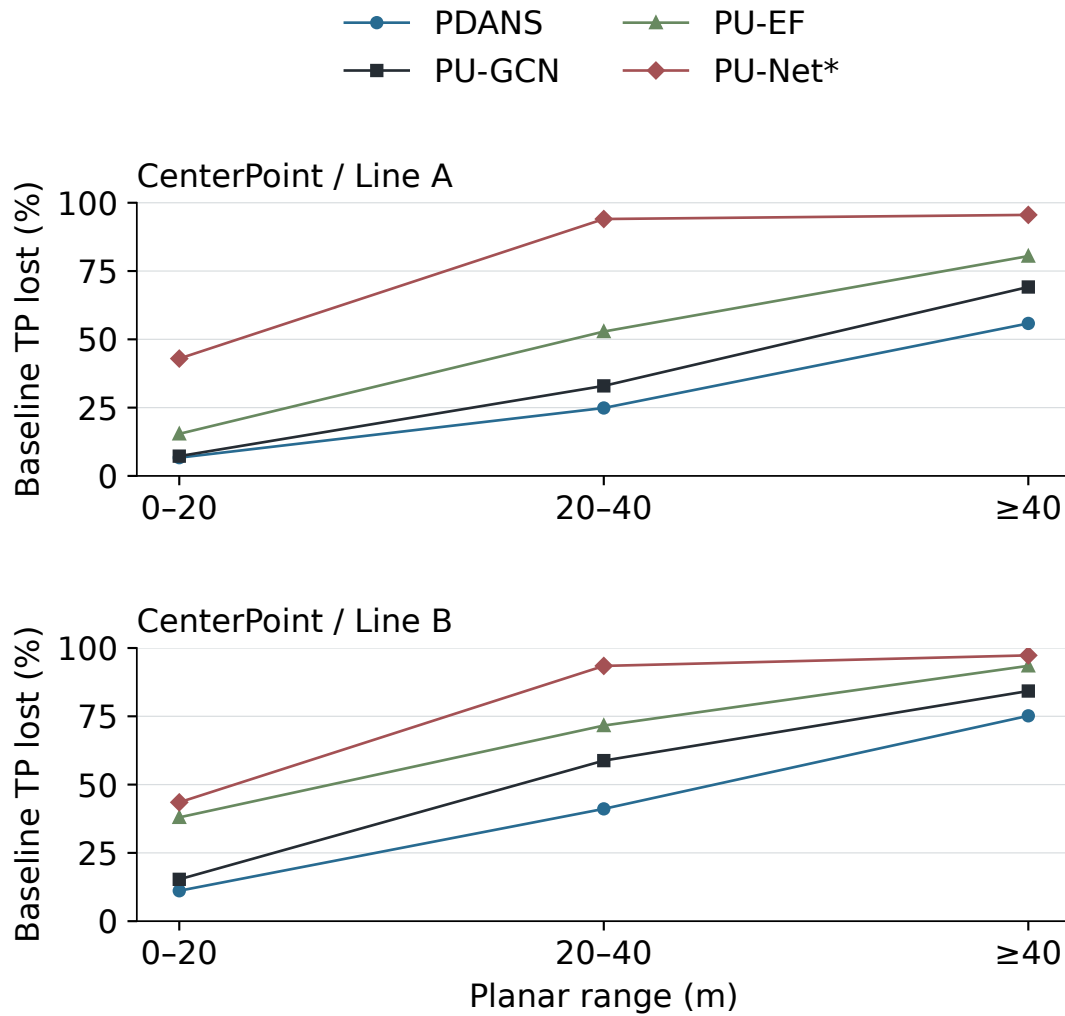


Figure 5.25: CenterPoint Car loss rates from the same historical audit. The denominator is the baseline-matched count within each bin. These descriptive strata are separate from KITTI Easy, Moderate, and Hard difficulty definitions.

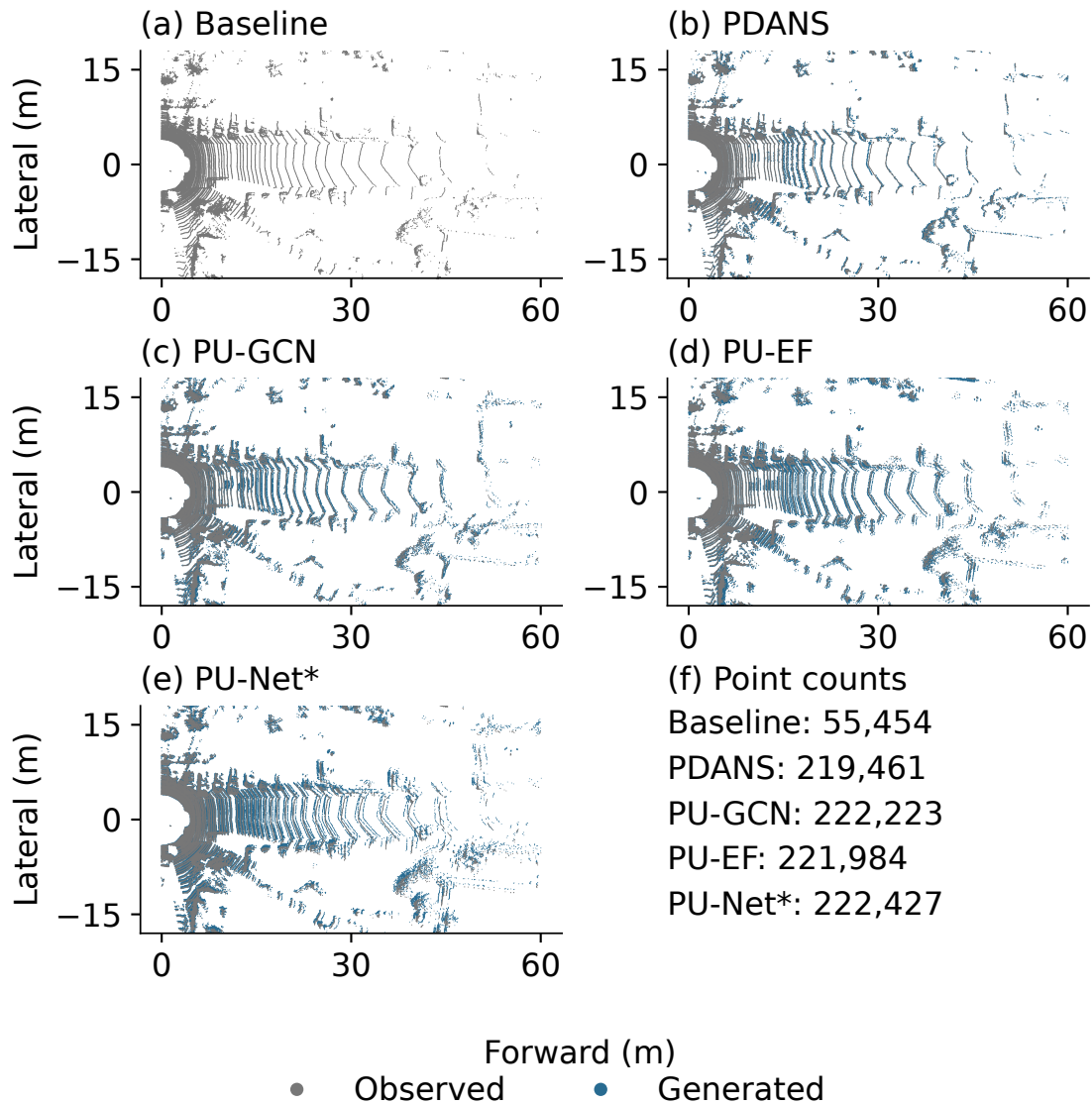


Figure 5.26: Lab frame 000590, historical Line A E1 inputs. Gray points are retained observed rows and blue points are generated rows. The shared crop is 0–60 m forward and –18–18 m lateral, all heights, with no display subsampling. PU-Net* is the old faulty adapter; PU-EF is the lab compatibility path.

5.3.8 Whole-Scene Multi-Method Comparisons

Figures 5.26 and 5.27 re-render the local lab visualisation arrays for frame 000590, including PDANS, PU-GCN, PU-EdgeFormer, and PU-Net in both lines. The display window is fixed to 0–60 m forward and –18–18 m lateral, with no height filter. All points inside this window are drawn. Measured rows are identified by exact float32-row

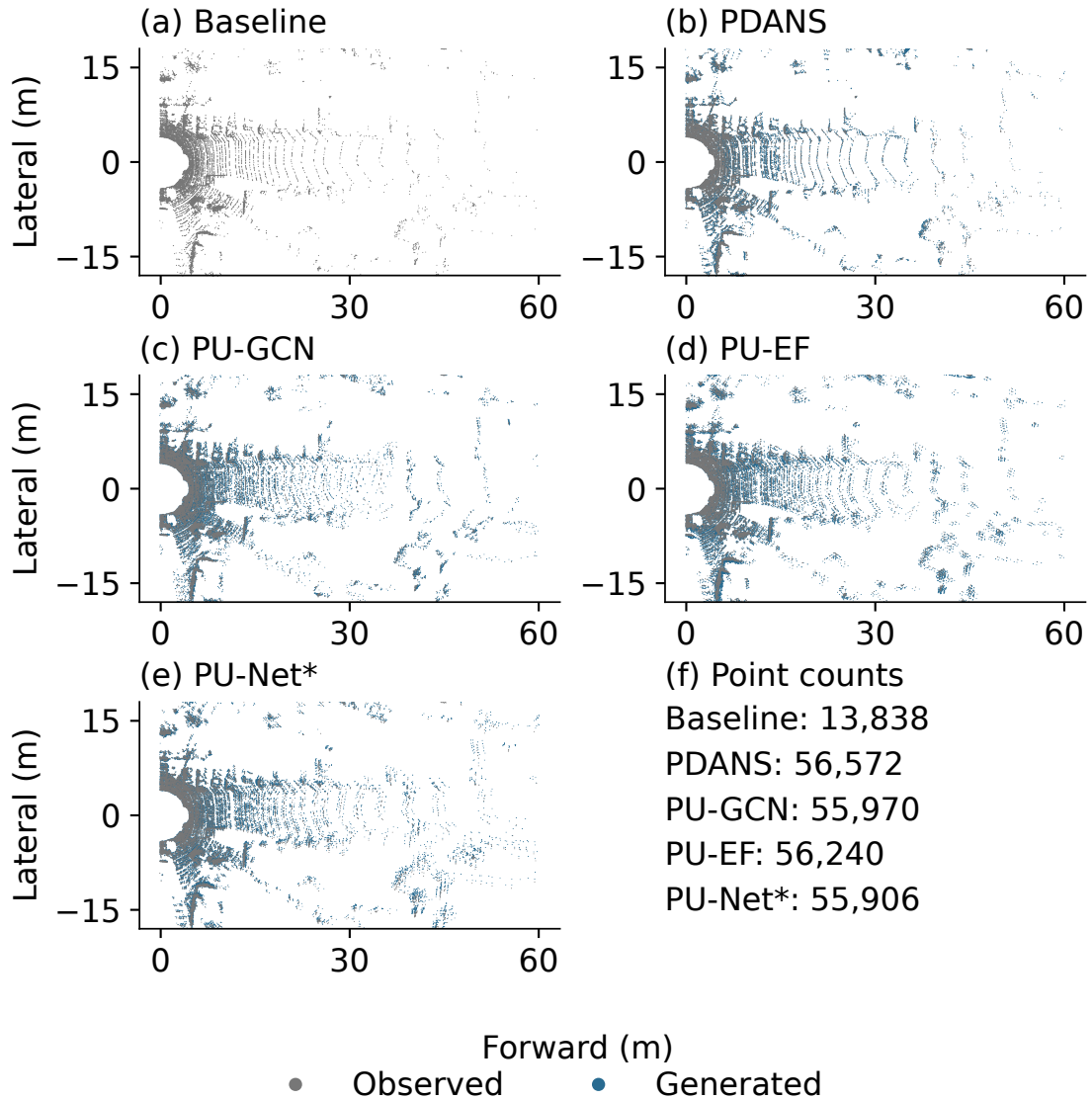


Figure 5.27: The same scene under historical Line B construction. Measured points now come from the sparse baseline. All four stored method outputs are included under the same crop and colour convention as Figure 5.26.

membership in the corresponding observed input, rather than by assigning a method a presumed preservation property.

The separate colours reveal a distinction hidden by an all-blue dense cloud: these historical E1 inputs retain observations at the stored-cloud stage, but much of the displayed density comes from generated rows. In Line B, the measured component is the sparse input, so recovery to the original file size does not restore the original

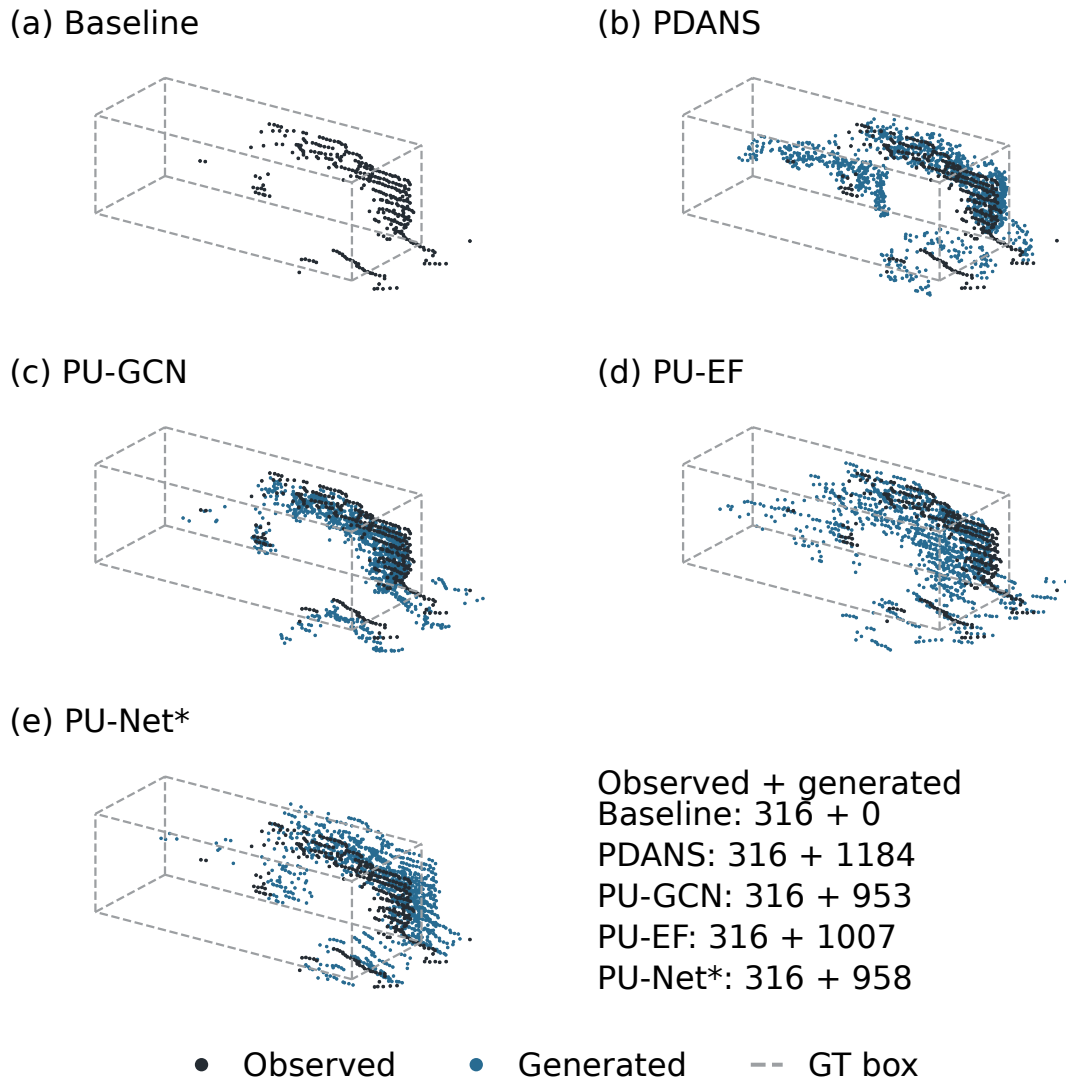


Figure 5.28: Common object-aligned crop for frame 005625, Car audit index 10. All four outputs retain 316 measured rows in this window; generated rows are shown separately. The dashed GT box and view are identical across methods. Historical pipeline labels match Figure 5.26.

measurements. The figure’s point counts refer to the display window, not to detector input cardinality or the number of points inside labelled objects.

5.3.9 Local Surfaces and Measurement Preservation

The whole-scene view cannot resolve the distribution around one object. Figures 5.28 and 5.29 therefore use the same Car in frame 005625 (audit GT index 10) and the same object-aligned coordinates for every method. The crop extends 0.6 m beyond

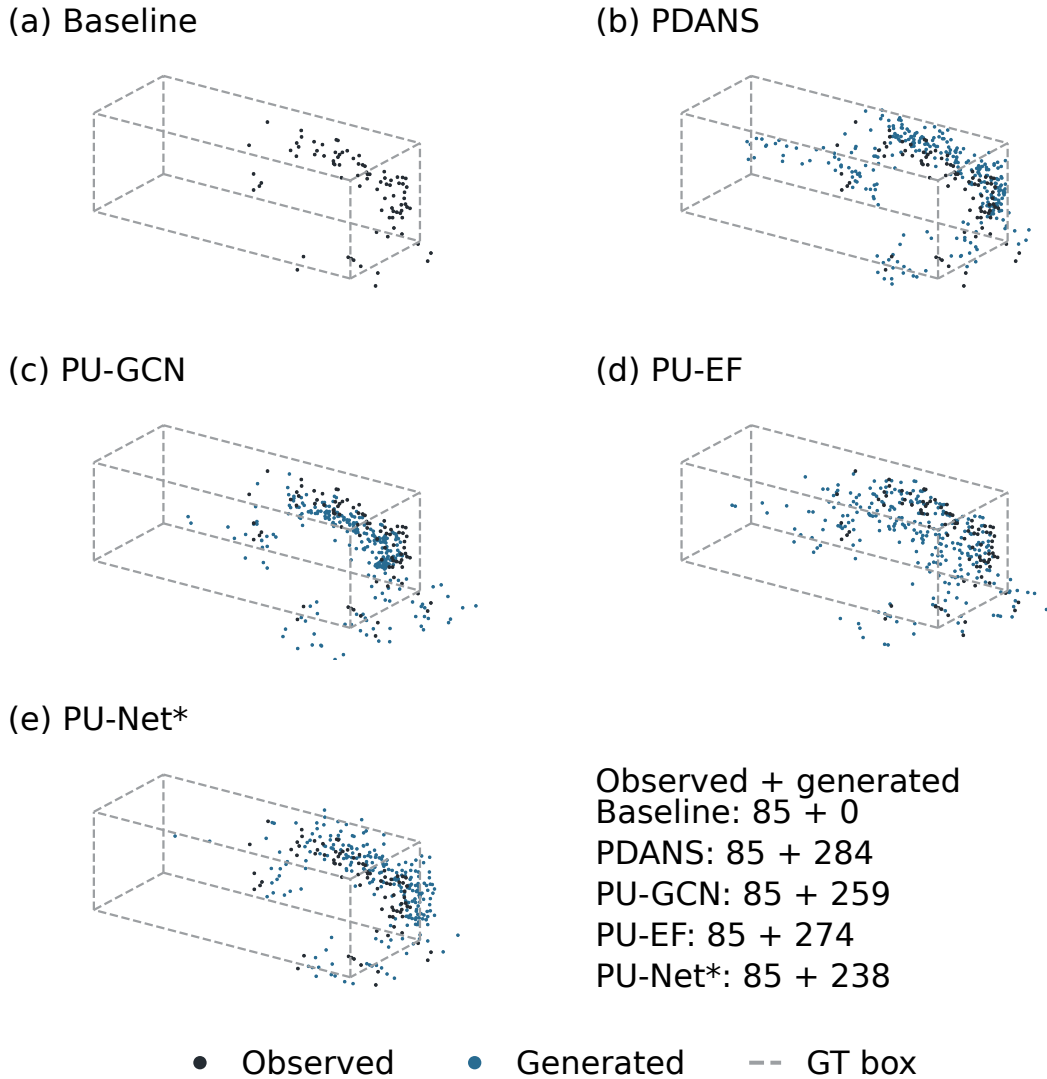


Figure 5.29: The same labelled Car after sparsification. All points within the declared crop are shown; counts distinguish the 85 measured rows from generated rows. Increased crop density alone does not establish restored surface accuracy or improved detection.

the box in length and width and 0.4 m in height. These margins retain context around the measured surface; crop counts are not restricted to the interior of the ground-truth box.

The original crop contained 316 observed rows, compared with 85 after sparsification. In Line A, PDANS added 1,184 generated rows within this window and PU-GCN added 953. In Line B, the respective counts were 284 and 259. The visible point concentration and the incomplete measurement support can therefore be assessed separately. The

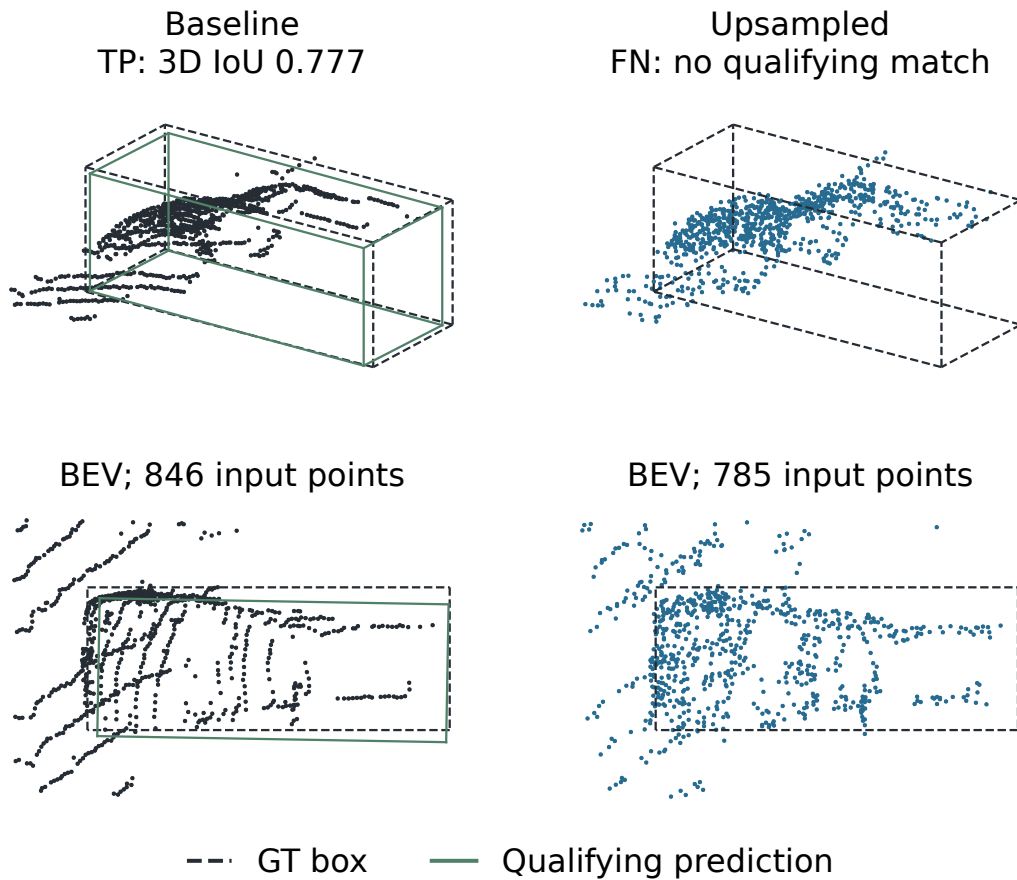


Figure 5.30: Historical lab frame 000590, Line A, PDANS, audit object index 2. The oblique view and BEV show the same effective-input crop. Predictions are read from the retained manifest; only qualifying matches are drawn. The class threshold is 0.70 3D IoU.

dashed box specifies the labelled object’s extent; it is not a surface reconstruction target. Points outside it can be legitimate background because the crop deliberately includes a margin.

5.3.10 Matched Detection Failures and Positive Counterexamples

The four examples below were already identified in the lab event audit. They are selected diagnostic cases, not a random estimate of event frequencies. Each compares the same labelled object before and after upsampling using detector-effective input. Dashed boxes are ground truth; solid boxes are saved qualifying predictions. Un-

qualified predictions are not drawn, so an FN label means no qualifying match, not necessarily an absence of proposals.

The PointRCNN Line A PDANS example lost a Car that initially had a well-aligned prediction. The common local view permits inspection of altered neighbourhood support without confusing this with the global increase in stored point count. Figure 5.30 records 3D IoU 0.777 before upsampling and no qualifying match afterwards. The displayed effective-input crop contains 846 baseline points and 785 upsampled points.

A different PointRCNN example provides a positive counterexample in Line B. The recovered Car crosses the matching threshold despite fewer effective input points in this local crop. Local cardinality alone therefore cannot explain the sign of a detection transition. Figure 5.31 records no qualifying match before upsampling and 3D IoU 0.705 afterwards. The displayed effective-input crop contains 312 baseline points and 271 upsampled points.

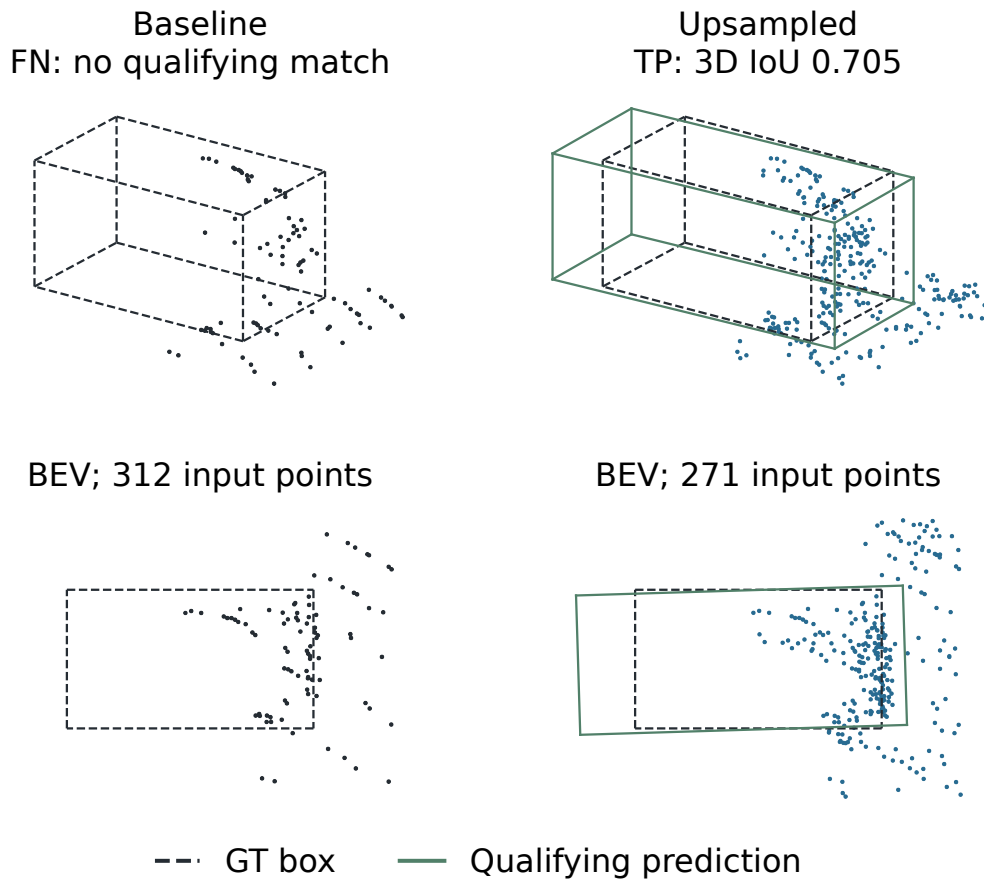


Figure 5.31: Historical lab frame 005625, Line B, PDANS, audit object index 7. The oblique view and BEV show the same effective-input crop. Predictions are read from the retained manifest; only qualifying matches are drawn. The class threshold is 0.70 3D IoU.

The CenterPoint Line B PU-GCN Car is a complementary negative case: more retained crop points coexist with loss of a previously qualifying prediction. This bounds the claim that preserving measurements or adding local density is sufficient for detection. Figure 5.32 records 3D IoU 0.871 before upsampling and no qualifying match afterwards. The displayed effective-input crop contains 100 baseline points and 394 upsampled points.

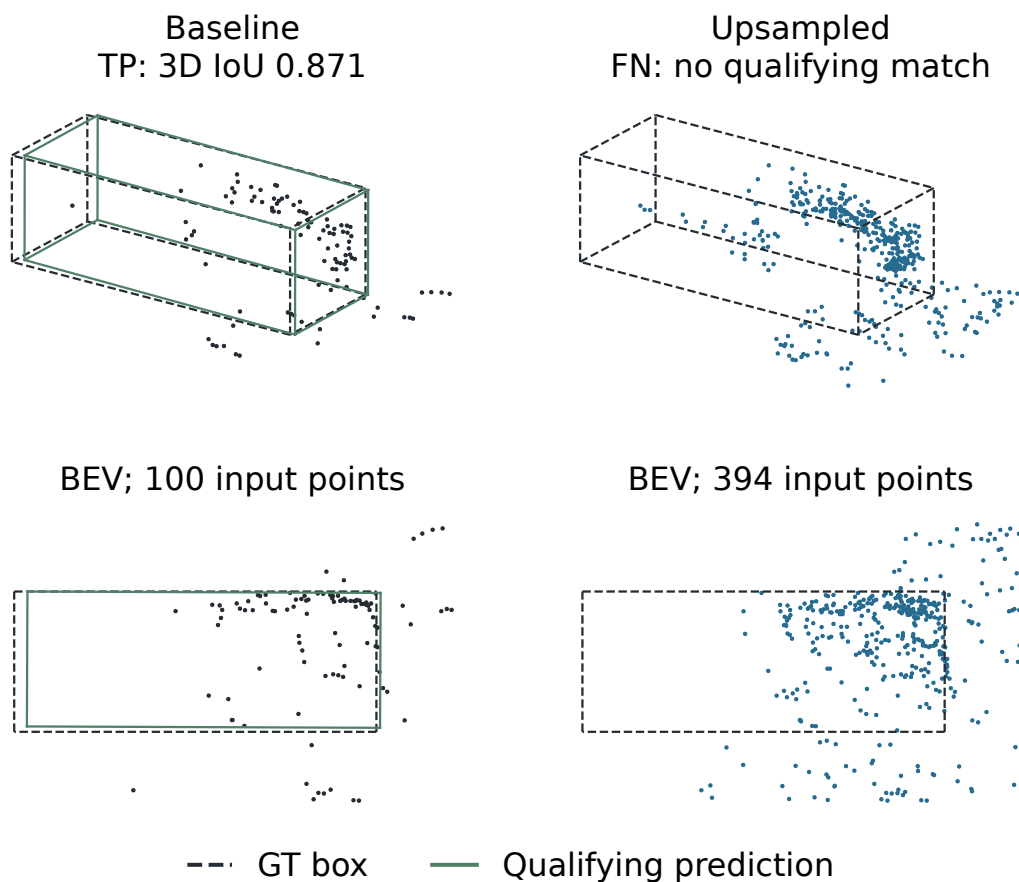


Figure 5.32: Historical lab frame 005625, Line B, PU-GCN, audit object index 10. The oblique view and BEV show the same effective-input crop. Predictions are read from the retained manifest; only qualifying matches are drawn. The class threshold is 0.70 3D IoU.

The CenterPoint Cyclist in the same Line B PU-GCN frame provides a positive case under the class-specific 0.50 IoU threshold. It shows that recoveries can occur in the same representation and scene as Car losses; the full-validation event counts establish which direction dominates. Figure 5.33 records no qualifying match before upsampling and 3D IoU 0.655 afterwards. The displayed effective-input crop contains 19 baseline points and 110 upsampled points.

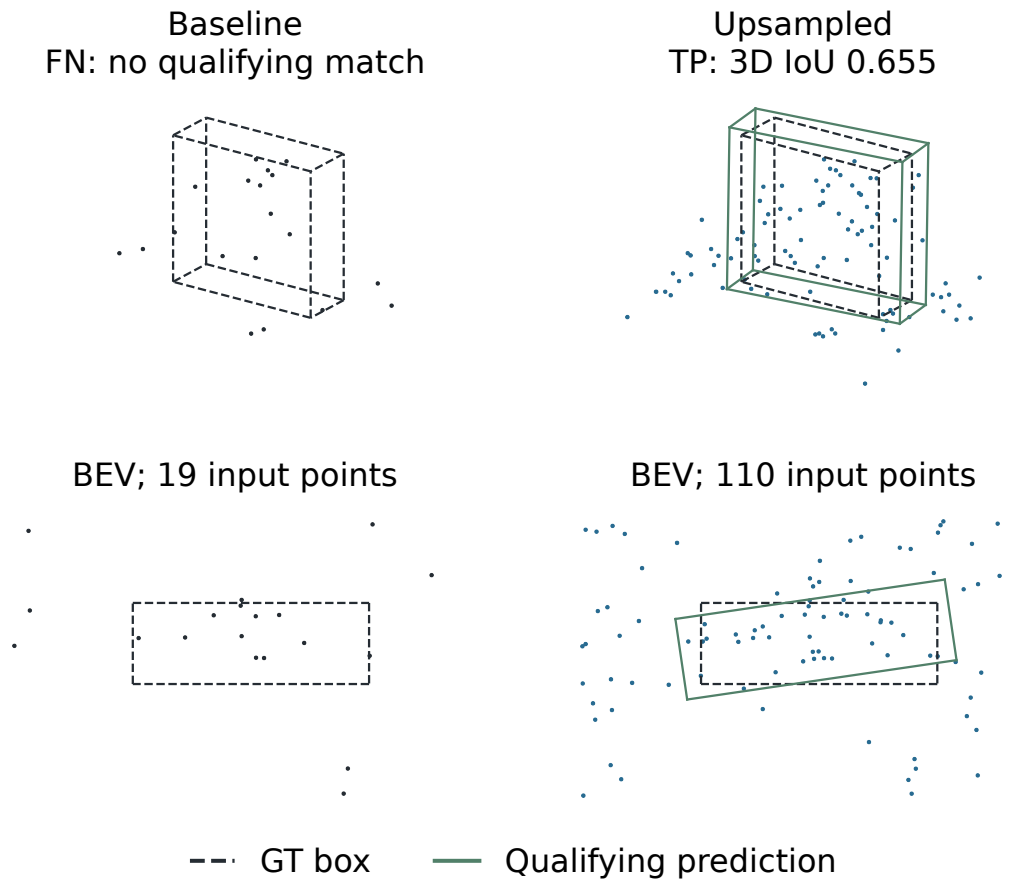


Figure 5.33: Historical lab frame 005625, Line B, PU-GCN, audit object index 4. The oblique view and BEV show the same effective-input crop. Predictions are read from the retained manifest; only qualifying matches are drawn. The class threshold is 0.50 3D IoU.

These cases discriminate several explanations that an AP table cannot. A global fourfold expansion need not increase the number of points entering a particular PointR-CNN crop, and a larger CenterPoint crop can still lose its matched box. Conversely, a positive transition does not require more local effective points. The examples support analysing where generated points enter the representation and how predictions change, while leaving the causal contribution of geometry, sampling, and learned features unresolved. The full-validation final matrix remains the evidence for detector adaptation.

5.4 Joint Discussion

The shared finding is a separation between nominal density, geometric fidelity, and task utility. The HPC classification study demonstrates it without a full-scene detector adapter: PU-GCN leads both Chamfer comparisons, yet PU-Net alone improves Line B classification and no method improves Line A. The lab detection study then shows an additional sensitivity to patch construction, measured-point preservation, and detector input processing. Together, the branches make a geometry-only account of upsampling quality insufficient for the tested downstream tasks.

The experimental designs are complementary but not interchangeable. ModelNet40 measures how well a separately trained PointNet++ learns each representation. Frozen KITTI insertion measures compatibility with a pretrained detector. KITTI adaptation then tests whether representation-specific training can reduce that mismatch. Neither OA and AP nor the two Line A/B scores should be averaged into a universal measure of upsampling quality. Their denominators, target tasks, and training interventions differ.

The tested geometric and learned methods target surface organisation, local feature expansion, or generative reconstruction [3, 4, 5, 6, 7]. Those objectives provide useful geometric priors, but they do not directly optimise the particular PointNet++, PointRCNN, or CenterPoint decisions evaluated here. A point may reduce average surface distance while changing a neighbourhood, occupying a previously empty voxel, or replacing a measurement inside a finite downstream budget.

The reception and voxel probes support this interpretation at different levels of the pipeline. PointNet++ discards much local candidate support at high density; CenterPoint can activate or truncate many additional voxels; PointRCNN resampling changes which points reach the network. These are measured properties, but they do not uniquely decompose every AP or OA deficit. In particular, Line B loses detection performance even without voxel-cap activation, and a full-input PointRCNN path also harms the original baseline. Distribution compatibility extends beyond a single hard input cap.

The object-level analyses strengthen the comparison without equating its different units. On ModelNet40, PU-GCN’s CD advantage holds for more than 93% of test objects in each line, yet it does not produce the best classification result. The mismatch is therefore not explained only by a few geometric outliers. Conversely, Line B PU-Net’s HD advantage is widespread and accompanies its small classification gain, but this co-occurrence does not establish HD as the cause of that gain.

The lab object evidence distinguishes a different set of alternatives. The same historical inputs can lose Cars and recover a Cyclist in one scene; more effective crop points can coexist with a lost box, while a recovered PointRCNN object can have fewer effective crop points. Thus neither an all-failure narrative nor a simple local-count rule fits the retained observations. The loss/recovery totals supply the population context for these deliberately selected examples.

These findings are compatible with the local grouping of PointNet++ and PointRCNN and the voxel-based representation of CenterPoint [8,9,11], but the visualisations do not directly measure learned feature changes. The strongest supported interpretation concerns the complete input and task pipeline. The relative contributions of geometry, sampling, normalisation, and optimisation would require further controlled interventions to separate.

The improvement after detector adaptation is consistent with a mismatch between generated inputs and the original detector’s learned representation. Observation preservation helps separately by retaining the actual sensor rows. Their combination gives the best completed KITTI results, especially for CenterPoint. The extended schedules then tested whether the original three-epoch stopping point was sufficient to explain the remaining loss. All four observed-first conditions improved further and met the specified Car-validation plateau rule, yet their baseline deficits persisted (Section 5.2.5). This strengthens the conclusion within the tested schedules without excluding another optimisation or input design. Because the baseline schedules were not extended to match, and the validation set also selected checkpoints, the differences remain conditional system comparisons rather than an isolated causal effect of upsampling.

This distinction also prevents an inaccurate account of the later PU-GCN work. The new contribution was the completed generation, controlled input construction, detector adaptation, and full-validation evaluation. It was not a newly trained PU-GCN model. Lab PU-EdgeFormer compatibility results and flawed old PU-Net inputs likewise remain evidence about particular implemented pipelines, rather than intrinsic rankings of their published architectures.

The strongest evidence consists of complete paired dataset evaluations, native-cardinality controls, and directly measured implementation effects. The principal limits are one classifier training seed, test-based ModelNet40 checkpoint selection, batch-aggregated classifier metrics, provisional mesh-reference training controls, unequal detector-training budgets, and validation-based detector checkpoint selection. The plateau criterion is specific to the chosen Car metric and the executed sched-

ules. Diagnostic subsets, category correlations, and selected visual cases refine the interpretation but do not increase the number of independent primary experiments.

Within these limits, the practical implication is specific. An upsampling pipeline for downstream perception must be assessed after the actual input adapter and task model, with its observation-preservation rule, reference geometry, and downstream training regime stated explicitly. The work does not rule out useful upsampling. It identifies the conditions under which the tested geometric gains failed to become task gains and the partial recovery achieved by the completed adaptation procedure.

Chapter 6

Conclusion and Outlook

This thesis investigated whether point-cloud upsampling improves downstream perception, using object classification on ModelNet40 and three-dimensional object detection on KITTI. The two-line design separated densification of an available observation from recovery after fourfold sparsification. Five upsampling methods completed the HPC object-level study. The lab scene-level study combined earlier frozen-detector comparisons with a later, fully evaluated PU-GCN input and detector-adaptation experiment. The combined evidence shows that added coordinates, geometric agreement, and useful task information are distinct properties.

6.1 Findings from the Completed Experiments

On ModelNet40, none of the five upsampled Line A representations exceeded the original 1,024-point classifier. Original best reported OA was 91.95%, while the highest upsampled best OA was 91.63% for PU-GCN. After sparsification to 256 points, PU-Net alone improved the selected-checkpoint result, from 90.85% to 91.27%. Its final-epoch gain was smaller, from 90.46% to 90.65%. These are results from separately trained, single-seed classifiers, not repeated-run estimates (Tables 5.1 and 5.2).

The geometric ranking did not reproduce the classification ranking. PU-GCN had the lowest mean equal-cardinality Chamfer distance in both lines, while PU-Net had the best Line B classification and Hausdorff distance. PointNet++ reception measurements further distinguished stored density from the support used by local feature aggregation. PU-GCN’s CD advantage held for more than 93% of test objects in each line, while PU-Net had the lowest Line B HD for about 90%. Thus the geometric and task mismatch was not explained only by a few outliers. The two retained point-cloud examples illustrate that distinction without assigning unrecorded classification outcomes (Sections 5.1.4, 5.1.5 and 5.1.6).

The KITTI study first evaluated 20 three-epoch or frozen-weight arms on all 3,769 validation frames, then extended training for the four observed-first detector–line com-

binations. The extended schedules improved on both frozen observed-first inputs and the earlier adapted checkpoints, while satisfying the specified Car AP plateau criterion. Selected Line A scores reached 71.59 for PointRCNN and 75.97 for CenterPoint, below the original baselines of 81.95 and 79.28. Line B reached 57.05 and 62.90, below the adapted sparse baselines of 68.33 and 68.05 (Table 5.15). Thus the residual deficit persisted beyond the original short adaptation schedule. PU-GCN itself remained fixed throughout.

The integration diagnostics explain why a method name alone is insufficient to interpret a scene-level result. Local patch construction, correct normalisation, measurement preservation, and the detector’s point or voxel budget all changed the evaluated input. Correcting PU-Net normalisation and patch locality substantially improved its diagnostic result. Enlarging PointRCNN input or changing CenterPoint occupancy did not yield a universal remedy. These findings support evaluation of the complete implemented pipeline, with the scope of every control stated explicitly (Sections 5.3.2 and 5.3.3).

6.2 Limits of the Evidence

The ModelNet40 classifiers used one seed and selected their best checkpoint on the test set. Their saved OA and class scores average batch-level ratios; they are not exact object-count proportions. Initial mesh sampling also used Python’s process-dependent hash in its seed. Reusing the archived point sets supports repeatability of the completed runs, whereas exact regeneration from meshes requires additional hash-state control. Reporting epoch 200 alongside best OA reduces dependence on that selection in the presentation, but does not create an independent validation protocol. Four reused training objects limit the mesh-reference classifier controls. Equal-cardinality mesh-reference comparisons also use independently normalised point samples rather than one shared mesh-derived transform. Their distance rankings are conditional on that coordinate convention. P2F used independent cloud and mesh normalisation, and NUC query centres were not perfectly shared across methods; these remain secondary geometry diagnostics. Missing per-object classifier predictions prevent paired prediction-level tests (Sections 4.3.1, 4.10.1, and 4.10.2).

The KITTI extension established a plateau under a 0.2-AP, three-epoch patience rule for Car Moderate 3D AP, rather than global optimisation convergence or convergence of all classes. Checkpoints were selected and reported on the same validation set. Neither the original nor the sparse baseline was extended to an equal training budget;

Line A also has no adapted original baseline. These design limits prevent attributing the system-level differences to generated geometry alone. The earlier PU-Net full-validation pipeline had a normalisation error, and the lab PU-EdgeFormer execution used a compatibility path. Historical evaluator variants and small diagnostic subsets consequently cannot be combined with the final matrix as one architecture ranking. Development also examined validation outcomes; these results are not a new blind test (Sections 5.2.5 and 5.3).

6.3 Implications and Future Work

The practical implication is to assess upsampling after the actual task model and its input adapter. A useful evaluation records visible measurements, generated candidates, strict output, and effective model input separately. It also distinguishes geometry references and training regimes. Such reporting makes a limited recovery gain interpretable without promoting it into a general claim about point-cloud densification.

Further experiments could strengthen the conclusions by separating classifier validation from the fixed test set, repeating training across seeds, regenerating all mesh-reference training controls, evaluating geometry with a shared mesh-derived transform, and retaining per-object predictions. For KITTI, the remaining comparison should use equally trained original, sparse and upsampled inputs, together with independent checkpoint selection and final evaluation. Repeated seeds and alternative optimisation schedules would test the stability of the residual deficit beyond the completed plateau assessment. Native implementations under corrected common patch processing would permit cleaner method comparisons. These are proposed extensions, not experiments performed in this thesis.

Within the executed scope, geometric improvement was insufficient to establish downstream improvement. PU-Net provided a limited object-level recovery benefit, while observation preservation and detector adaptation partially recovered KITTI performance. The evidence supports these conditional findings rather than a universal benefit or universal failure of point-cloud upsampling.

List of Figures

2.1	An unordered point cloud	6
2.2	PointNet++ principle	11
2.3	Point-based and centre-based detection	14
2.4	PU-GCN principle	22
4.1	Local patch processing	46
4.2	Effective detector input	51
4.3	Three-epoch detector adaptation	60
5.1	Classification during training	70
5.2	Later classification trajectories	71
5.3	Line A category changes, part 1	73
5.4	Line A category changes, part 2	74
5.5	Line B category changes, part 1	75
5.6	Line B category changes, part 2	76
5.7	Genuine density controls	77
5.8	PointNet++ neighbourhood reception	78
5.9	Geometric fidelity at equal cardinality	80
5.10	Object-wise geometry winners	82
5.11	Airplane multi-method comparison, Line A	84
5.12	Chair multi-method comparison, Line B	85
5.13	Class-specific adaptation outcome	89
5.14	Recovery and remaining deficits after extended adaptation	94
5.15	Terminal validation after the selected PointRCNN checkpoint	96
5.16	Earlier CenterPoint multi-method comparison	98
5.17	PointRCNN input-budget sensitivity	99
5.18	Paired PDANS patch responses	102
5.19	Normalisation and patch ablation	103
5.20	Voxel-budget audit	104
5.21	Frame-subset size and paired AP variability	107
5.22	PointRCNN lost and recovered objects	113

5.23	CenterPoint lost and recovered objects	115
5.24	PointRCNN loss by object range	116
5.25	CenterPoint loss by object range	117
5.26	Line A multi-method scene comparison	118
5.27	Line B multi-method scene comparison	119
5.28	Line A local multi-method surfaces	120
5.29	Line B local multi-method surfaces	121
5.30	PointRCNN Car loss case	122
5.31	PointRCNN Car recovery case	124
5.32	CenterPoint Car loss case	125
5.33	CenterPoint Cyclist recovery case	126

List of Tables

2.1	Object-level point clouds versus LiDAR scenes	8
4.1	Executed comparison lines. ModelNet40 uses one separately trained classifier per representation; KITTI separates frozen inference and detector adaptation.	38
4.2	Selected recorded integration changes with an effect on execution or evidence quality. Exact code paths, environment details and recovery records are archived with the updated HPC and lab records.	40
4.3	Equal-cardinality CD/HD comparisons on 2,468 test objects per condition.	55
4.4	Initial three-epoch detector-adaptation matrix. F: frozen detector weights; A: adapted detector weights.	59
5.1	ModelNet40 Line A classification, with accuracies in percent. Δ OA is the Best-OA change from Original 1024 in percentage points.	68
5.2	ModelNet40 Line B classification after $1,024 \rightarrow 256$ sparsification and $256 \rightarrow 1,024$ upsampling. The immediate baseline retains its native 256 points. Definitions match Table 5.1.	69
5.3	Direction of reported category-score changes relative to the native baseline. Each row covers all 40 categories. The mean is an unweighted mean of archived category deltas and is not reconstructed OA.	72
5.4	Line A geometry against the independent 4,096-point reference. CD and HD are unsquared distances in normalised coordinates. *P2F uses independently normalised clouds and meshes.	79
5.5	Line B geometry against Original 1024. Definitions match Table 5.4. The sparse 256-point baseline has a separate 256-point mesh reference and is not ranked in this table. The P2F column retains the separate cloud-to-mesh convention.	79
5.6	PointRCNN Line A Car 3D AP_{R40} , 3,769 frames per row. All deltas reference the original frozen baseline. No adapted original baseline was trained, so adapted-row deltas change both input and weights.	87

5.7	PointRCNN Line B Car 3D AP _{R40} . Every row evaluates 3,769 frames. Frozen rows reference the frozen sparse baseline; adapted rows reference the separately adapted sparse baseline.	87
5.8	CenterPoint Line A Car 3D AP _{R40} , 3,769 frames per row. Deltas reference the frozen original baseline. The direct-adapted configuration was not part of the completed matrix.	88
5.9	CenterPoint Line B Car 3D AP _{R40} , 3,769 frames per row. Frozen and adapted arms use their corresponding sparse-baseline control.	88
5.10	PointRCNN BEV AP _{R40} on the final validation set.	90
5.11	PointRCNN Image-Box AP _{R40} on the final validation set.	90
5.12	CenterPoint Pedestrian and Cyclist 3D AP _{R40} on the final validation set.	91
5.13	CenterPoint BEV AP _{R40} on the final validation set.	92
5.14	PointRCNN adaptation screen on 256 frames: Car Moderate 3D AP at IoU 0.70 under the legacy evaluator.	92
5.15	Detector-adaptation comparison: Car Moderate 3D AP _{R40} on 3,769 validation frames. Bold marks extended-schedule results, not the highest scores across input controls.	93
5.16	Completed schedules and selected checkpoints. Final count denotes consecutive terminal epochs without improvement exceeding the running reference by 0.2 AP points (Section 4.11).	95
5.17	Historical PointRCNN E2 3D AP _{R40} for Car. Each row covers all 3,769 frames; the integrated input protocol is shared within E2. Values are percentages.	100
5.18	Historical PointRCNN E2 BEV AP _{R40} for Car. Each row covers all 3,769 frames; the integrated input protocol is shared within E2. Values are percentages.	100
5.19	Historical PointRCNN E2 image-box AP _{R40} for Car. Each row covers all 3,769 frames; the integrated input protocol is shared within E2. Values are percentages.	101
5.20	Historical PointRCNN E3 Car 3D AP _{R40} . Δ Mod. is E3 minus E2 for the same input name, in AP points. All ten rows have 3,769 prediction files.	101
5.21	Low-dose comparison on 256 frames. The last column is PDANS minus observed-fill control, in percentage points.	106

5.22	Sample standard deviations of paired Car Moderate AP contrasts in AP points. All three contrasts use the same sampled frames within a draw.	107
5.23	Recorded full-validation all-valid-input experiment. Values are Car 3D AP from that historical protocol. PU-EF is the compatibility path and PU-Net old retains the faulty adapter.	108
5.24	PointRCNN 256-frame paired patch pilots, Car Moderate 3D AP. Surface denotes surface-c32; the two pilot batches retain separate baselines.	109
5.25	Region-split pilot on 256 frames: Moderate 3D AP _{R40} . Generated rows use surface-c32; PU-Net normalisation is fixed. This is a separate three-class PointRCNN implementation and inference protocol.	109
5.26	PointRCNN V1 selector/component pilot on 256 frames, Car Moderate 3D AP _{R40} . The separate 16-seed probe qualifies these single-evaluation scores.	110
5.27	Completed 256-frame PDANS versions: PointRCNN Car and CenterPoint Car/Pedestrian/Cyclist Moderate 3D AP. CenterPoint Pedestrian/Cyclist baselines are obtained from the recorded scores and baseline deltas.	110
5.28	Line B 256-frame selection pilots, Car Moderate 3D AP. PU-EF is the compatibility implementation and PU-Net uses fixed normalisation. Protocol-specific baselines are not pooled.	111
5.29	Complete retained transition summary. B is baseline matched count, L is lost, R is recovered, and $\Delta = R - L$. PU-EF is the compatibility path and PU-Net* the faulty historical adapter.	113

Bibliography

- [1] A. Geiger, P. Lenz, C. Stiller, and R. Urtasun, “Vision meets robotics: The KITTI dataset,” *Int. J. Robot. Res.*, vol. 32, no. 11, pp. 1231–1237, 2013.
- [2] A. Geiger, P. Lenz, and R. Urtasun, “Are we ready for autonomous driving? the KITTI vision benchmark suite,” in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2012, pp. 3354–3361.
- [3] H. Huang, S. Wu, M. Gong, D. Cohen-Or, U. Ascher, and H. Zhang, “Edge-aware point set resampling,” *ACM Trans. Graph.*, vol. 32, no. 1, pp. 9:1–9:12, 2013.
- [4] L. Yu, X. Li, C.-W. Fu, D. Cohen-Or, and P.-A. Heng, “PU-Net: Point cloud upsampling network,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2018, pp. 2790–2799.
- [5] G. Qian, A. Abualshour, G. Li, A. Thabet, and B. Ghanem, “PU-GCN: Point cloud upsampling using graph convolutional networks,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2021, pp. 11 683–11 692.
- [6] D. Kim, M. Shin, and J. Paik, “PU-EdgeFormer: Edge transformer for dense prediction in point cloud upsampling,” in *Proc. IEEE Int. Conf. Acoust. Speech Signal Process. (ICASSP)*, 2023, pp. 1–5.
- [7] B. Zhang, S. Yang, H. Chen, C. Yang, J. Jia, and G. Jiang, “Point cloud up-sampling using conditional diffusion module with adaptive noise suppression,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2025, pp. 16 987–16 996.
- [8] S. Shi, X. Wang, and H. Li, “PointRCNN: 3D object proposal generation and detection from point cloud,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2019, pp. 770–779.
- [9] T. Yin, X. Zhou, and P. Krähenbühl, “Center-based 3D object detection and tracking,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2021, pp. 11 784–11 793.
- [10] Z. Wu, S. Song, A. Khosla, F. Yu, L. Zhang, X. Tang, and J. Xiao, “3D ShapeNets: A deep representation for volumetric shapes,” in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2015, pp. 1912–1920.
- [11] C. R. Qi, L. Yi, H. Su, and L. J. Guibas, “PointNet++: Deep hierarchical feature learning on point sets in a metric space,” in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2017, pp. 5099–5108.

- [12] C. R. Qi, H. Su, K. Mo, and L. J. Guibas, “PointNet: Deep learning on point sets for 3D classification and segmentation,” in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2017, pp. 652–660.
- [13] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, “Focal loss for dense object detection,” in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, 2017, pp. 2980–2988.
- [14] X. Zhou, D. Wang, and P. Krähenbühl, “Objects as points,” *arXiv preprint arXiv:1904.07850*, 2019.
- [15] Y. Zhou and O. Tuzel, “VoxelNet: End-to-end learning for point cloud based 3D object detection,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2018, pp. 4490–4499.
- [16] Y. Yan, Y. Mao, and B. Li, “SECOND: Sparsely embedded convolutional detection,” *Sensors*, vol. 18, no. 10, p. 3337, 2018.
- [17] R. Li, X. Li, C.-W. Fu, D. Cohen-Or, and P.-A. Heng, “PU-GAN: A point cloud upsampling adversarial network,” in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, 2019, pp. 7203–7212.
- [18] Y. Wang, S. Wu, H. Huang, D. Cohen-Or, and O. Sorkine-Hornung, “Patch-based progressive 3D point set upsampling,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2019, pp. 5958–5967.
- [19] L. Yu, X. Li, C.-W. Fu, D. Cohen-Or, and P.-A. Heng, “EC-Net: An edge-aware point set consolidation network,” in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2018, pp. 386–402.
- [20] H. Fan, H. Su, and L. J. Guibas, “A point set generation network for 3D object reconstruction from a single image,” in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2017, pp. 605–613.
- [21] Y. Wang, Y. Sun, Z. Liu, S. E. Sarma, M. M. Bronstein, and J. M. Solomon, “Dynamic graph CNN for learning on point clouds,” *ACM Trans. Graph.*, vol. 38, no. 5, pp. 146:1–146:12, 2019.
- [22] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2017, pp. 5998–6008.
- [23] J. Ho, A. Jain, and P. Abbeel, “Denoising diffusion probabilistic models,” in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2020, pp. 6840–6851.
- [24] D. P. Huttenlocher, G. A. Klanderman, and W. J. Rucklidge, “Comparing images using the Hausdorff distance,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 15, no. 9, pp. 850–863, 1993.

- [25] A. Simonelli, S. R. Bulò, L. Porzi, M. López-Antequera, and P. Kotschieder, “Disentangling monocular 3D object detection,” in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, 2019, pp. 1991–1999.
- [26] M. Everingham, L. Van Gool, C. K. I. Williams, J. Winn, and A. Zisserman, “The PASCAL visual object classes (VOC) challenge,” *Int. J. Comput. Vis.*, vol. 88, no. 2, pp. 303–338, 2010.
- [27] X. Chen, H. Ma, J. Wan, B. Li, and T. Xia, “Multi-view 3D object detection network for autonomous driving,” in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2017, pp. 1907–1915.
- [28] OpenPCDet Development Team, “OpenPCDet: An open-source toolbox for 3D object detection from point clouds,” 2020.
- [29] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2015.
- [30] Q. McNemar, “Note on the sampling error of the difference between correlated proportions or percentages,” *Psychometrika*, vol. 12, no. 2, pp. 153–157, 1947.
- [31] KITTI Vision Benchmark Suite, “Object detection evaluation: 3D object detection,” evaluation rules, accessed 13 September 2026.
- [32] C. Spearman, “The proof and measurement of association between two things,” *Am. J. Psychol.*, vol. 15, no. 1, pp. 72–101, 1904.
- [33] B. Efron, “Bootstrap methods: Another look at the jackknife,” *Ann. Statist.*, vol. 7, no. 1, pp. 1–26, 1979.

Curriculum Vitae

Jiahui Shen

Academic Background

September 2023–September 2026
M.Sc. in Autonomy Technologies
Friedrich-Alexander-Universität
Erlangen-Nürnberg, Germany



Master's thesis

Investigating the Impact of Point Cloud Upsampling on 3D Object Detection Performance. Implemented preprocessing and evaluation workflows for point-cloud upsampling on ModelNet40 and KITTI. Compared geometric quality with PointNet++ classification and PointRCNN/CenterPoint detection results, and examined detection failures through point-cloud visualizations. Evaluated downstream detector adaptation to PU-GCN inputs while keeping the upsampling checkpoint fixed.

Seminar project

GAN-based Generative Modeling for Simulation Data. Reviewed the use of generative adversarial networks to generate room impulse responses for far-field speech recognition. Compared this approach with analytical room simulation through literature review and analysis of published experimental results. Examined the role of synthetic acoustic data under different room conditions, and prepared a technical report and seminar presentation.

Team project — October 2024–March 2025

Structure-from-Motion Pipeline. Contributed to a pipeline for recovering camera motion and sparse three-dimensional structure from fisheye image sequences using Python and OpenCV. Worked on camera calibration, image undistortion and feature matching using SIFT descriptors, FLANN and RANSAC-based geometric verification. Visualized feature correspondences, camera geometry and reconstructed point clouds to check the pipeline outputs.

October 2018–January 2023
B.Sc. in Aerospace Engineering
Warsaw University of Technology, Warsaw, Poland

Bachelor's thesis

Monte Carlo Simulation of a Flight After Engine Failure During the Take-off Process. Used Monte Carlo simulation to investigate aircraft flight following an engine failure during take-off.